

การระบุโลโก้จากภาพด้วยการเรียนรู้เชิงลึก

LOGO IDENTIFICATION FROM IMAGE BY DEEP LEARNING

ณภัทร ยอดสวัสดิ์
สุรศักดิ์ ต้นสุตเจริญยิ่ง
ศุภวิชญ์ พระสุรรัตน์
สุพิชชา พูลศิริ

รายงานนี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิศวกรรมศาสตรบัณฑิต
สาขาวิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์
สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง
ปีการศึกษา 2567

ปริญญานิพนธ์ปีการศึกษา 2567

สาขาวิชาวิศวกรรมคอมพิวเตอร์

คณะวิศวกรรมศาสตร์ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

เรื่อง การระบุโลโก้จากภาพด้วยการเรียนรู้เชิงลึก

LOGO IDENTIFICATION FROM IMAGE BY DEEP LEARNING

ผู้จัดทำ

- | | |
|---------------------------------|-----------------------|
| 1. นายณภัทร ยอดสวัสดิ์ | รหัสนักศึกษา 64010211 |
| 2. นายสุรศักดิ์ ต้นสุตเจริญยิ่ง | รหัสนักศึกษา 64010935 |
| 3. นายศุภวิชญ์ พระสุรัตน์ | รหัสนักศึกษา 64011289 |
| 4. นางสาวสุพิชชา พูลศิริ | รหัสนักศึกษา 64011308 |

อาจารย์ที่ปรึกษา

(รศ.ดร.เกียรติคุณ เกียรตินัยชนะกิจ)

การระบุโลโก้จากภาพด้วยการเรียนรู้เชิงลึก

นายณภัทร	ยอดสวัสดิ์	64010211
นายสุรศักดิ์	ตันสุคเจริญยิ่ง	64010935
นายสุภาวิชญ์	พระสุรัตน์	64011289
นางสาวสุพิชชา	พูลศิริ	64011308
รศ.ดร.เกียรติกุล	เจียรนัยชนะกิจ	อาจารย์ที่ปรึกษา
ปีการศึกษา 2567		

บทคัดย่อ

Logo Identification เป็นกระบวนการในการวิเคราะห์การปรากฏของโลโก้ภายในภาพด้วยการตรวจจับและจำแนกโลโก้ แบ่งออกเป็นสองขั้นตอนหลัก ได้แก่ การตรวจจับโลโก้และการจำแนกโลโก้ การตรวจจับโลโก้จะทำหน้าที่บอกตำแหน่งของโลโก้ทั้งหมดภายในภาพ ส่วนการจำแนกโลโก้จะทำการระบุแบรนด์ที่สอดคล้องกันกับโลโก้นั้น ๆ ซึ่งระบบจำเป็นต้องตรวจจับและจำแนกโลโก้ได้ด้วยความแม่นยำสูงภายในระยะเวลาที่เหมาะสม เพื่อให้สามารถติดตามและวิเคราะห์ข้อมูลจำนวนมากที่เผยแพร่อย่างต่อเนื่องบนแพลตฟอร์มออนไลน์ในปัจจุบัน โดยวิธีการที่นำเสนอใช้เทคนิคการเรียนรู้เชิงลึกเฉพาะทางในการตรวจจับโลโก้ภายในภาพ และโมเดลการเรียนรู้เชิงลึกอีกรูปแบบในการจำแนกโลโก้ที่ได้จากขั้นตอนก่อนหน้านี้

ผลการทดลองบนชุดข้อมูล FlickrLogos-47 และ 22-Class Thai Logos แสดงให้เห็นว่าวิธีการที่นำเสนอในงานวิจัยฉบับนี้มีประสิทธิภาพเหนือกว่าวิธีอื่น ๆ ทั้งในด้านความแม่นยำของการทดสอบและเวลาที่ใช้ในการประมวลผล

งานวิจัยนี้มีส่วนสำคัญอยู่สองด้าน ได้แก่ การเปรียบเทียบผลการทดลองจากเครื่องมือต่าง ๆ เพื่อกำหนดกลยุทธ์ที่ดีที่สุดสำหรับแต่ละขั้นตอน ได้แก่ การตรวจจับโลโก้และการจำแนกโลโก้ และวิธีการที่ได้นำเสนอในงานวิจัยฉบับนี้มีความแม่นยำที่สูงในระยะเวลาการประมวลผลที่เหมาะสมสำหรับการนำไปใช้งานในเชิงธุรกิจ

Logo Identification from Image by Deep Learning

Mr. Napat	Yordsawad	64010211
Mr. Surasak	Tansudjaroenyong	64010935
Mr. Suppawit	Prasurat	64011289
Ms. Supitcha	Pulsiri	64011308
Assoc.Prof.Dr. Kietikul	Jearanaitanakij	Advisor
Academic Year 2024		

ABSTRACT

Logo identification involves analyzing the presence of brand logos in images by detecting and classifying them. This process comprises two main steps: logo detection and logo classification. Logo detection identifies all logo images and their locations in the picture while logo classification maps those logo images into the corresponding brands. The main challenge in logo monitoring is the system must detect and classify logo images at an acceptable high accuracy within a reasonable short period of time. Our proposed method detects logos from an image by using specific deep learning techniques and classifies logos extracted from the previous step by another deep learning model.

The experimental results on the FlickrLogos-47 and 22-Class Thai Logos dataset indicate superior performance of the proposed method over other approaches both in terms of testing accuracy and execution time.

Our research contributes in two folds. First, we compare the experimental results from various tools to configure the best strategy in each of two steps, i.e., logo detection and logo classification. Second, our proposed method attains a high accuracy in a reasonable execution time suitable for a business deployment.

กิตติกรรมประกาศ

ปริญญานิพนธ์เรื่องการระบุโลโก้จากภาพด้วยการเรียนรู้เชิงลึกฉบับนี้ ประสบความสำเร็จไปได้ด้วยดี เนื่องจากได้รับความกรุณาจาก รศ.ดร.เกียรติกุล เจียรนัยธนกิจ ซึ่งเป็นอาจารย์ที่ปรึกษาที่คอยให้คำปรึกษาและคำแนะนำด้วยความใส่ใจอย่างเต็มที่ พร้อมทั้งเสนอแนวทางการแก้ไขปัญหาและข้อบกพร่องที่เกิดขึ้นอย่างมีประสิทธิภาพ จนโครงงานนี้สามารถสำเร็จลุล่วงตามเป้าหมายที่ได้วางไว้ คณะผู้จัดทำขอขอบพระคุณเป็นอย่างสูง

และยังได้รับความอนุเคราะห์จาก บริษัท ดาต้าเซต จำกัด ที่คอยอำนวยความสะดวกด้านทรัพยากร ให้แนวทางในการพัฒนา ตลอดจนชี้ให้เห็นถึงปัญหาที่อาจเกิดขึ้นตามมา คณะผู้จัดทำขอขอบพระคุณไว้ ณ โอกาสนี้

ขอขอบพระคุณครอบครัวของคณะผู้จัดทำ ที่ให้การสนับสนุน คอยห่วงใยและให้กำลังใจตลอดมา จนเป็นแรงผลักดันให้สามารถทำโครงงานนี้จนเสร็จลุล่วงไปได้ด้วยดี

สุดท้ายนี้ ขอขอบคุณคณะอาจารย์ทุกท่านและทางภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง ที่เอื้อเฟื้อสถานที่ในการทำงาน มอบความรู้และให้โอกาสได้ใช้สิ่งที่เรียนมาต่อยอดผ่านการทำโครงงาน ตลอดจนอำนวยความสะดวกตลอดการทำโครงงานครั้งนี้

ณภัทร ขอดสวัสดิ์

สุรศักดิ์ ดันสุดเจริญยิ่ง

ศุภวิชญ์ พระสุรัตน์

สุพิชชา พูลศิริ

สารบัญ

	หน้า
บทคัดย่อภาษาไทย	I
บทคัดย่อภาษาอังกฤษ	II
กิตติกรรมประกาศ.....	III
สารบัญ	IV
สารบัญตาราง	VI
สารบัญรูป	VII
บทที่ 1 บทนำ	1
1.1 ความเป็นมาของปัญหา	1
1.2 วัตถุประสงค์ของโครงการ.....	2
1.3 ขอบเขตของโครงการ	2
1.4 ขั้นตอนการดำเนินงาน	2
1.5 ประโยชน์ที่คาดว่าจะได้รับ	3
บทที่ 2 ทฤษฎีและงานวิจัยที่เกี่ยวข้อง	4
2.1 ทฤษฎีที่เกี่ยวข้อง.....	4
2.2 งานวิจัยที่เกี่ยวข้อง	15
2.3 เครื่องมือที่ใช้ในการพัฒนา	20
บทที่ 3 การออกแบบและการพัฒนา.....	21
3.1 การเตรียม Dataset และเครื่องมือที่ใช้	21
3.2 การจัดเตรียมข้อมูล.....	23
3.3 การพัฒนา Logo Detection	23
3.4 การพัฒนา Logo Classification	25
3.5 การพัฒนาเว็บแอปพลิเคชัน	33
3.6 การวัดประสิทธิภาพของโมเดล.....	43

สารบัญ (ต่อ)

	หน้า
บทที่ 4 การทดลองและผลการทดลอง.....	45
4.1 การทดลองหาจำนวน Epoch ที่ดีที่สุดในแต่ละโมเดล	45
4.2 การทดลองเทรน Detection Model หลังเทรนกับ 22-Class Thai Logos	46
4.3 การทดลองเทรน Detection Model กับ FlickrLogos-47 Dataset.....	50
4.4 การทดลองเทรน Classification model โดยใช้ Pre-trained CNN model	55
4.5 การทดลองเทรน Classification model โดยใช้ Custom CNN model.....	65
4.6 การทดสอบประสิทธิภาพโดยรวมของ Detection model และ Classification model...	68
4.7 การทดสอบเว็บแอปพลิเคชัน	76
บทที่ 5 บทสรุปและข้อเสนอแนะ.....	78
5.1 สรุปผลการพัฒนา	78
5.2 ปัญหาและแนวทางการแก้ไข	79
5.3 แนวทางการพัฒนาต่อ	79
บรรณานุกรม.....	81
ภาคผนวก.....	85

สารบัญตาราง

ตาราง	หน้า
ตาราง 1.1 แผนการดำเนินงาน	3
ตาราง 2.1 Confusion matrix ขนาด 2*2	7
ตาราง 2.2 เปรียบเทียบ Architectural Features โดยละเอียด	19
ตาราง 3.1 เปรียบเทียบชุดข้อมูล FlickrLogos-47 และ 22-Class Thai Logos	21
ตาราง 3.2 แสดงเลข Class Index ของ 22-Class Thai Logos Dataset.....	22
ตาราง 3.3 รายการความสามารถของผู้ใช้	34
ตาราง 3.4 ตัวชี้วัดประสิทธิภาพ (Metrics).....	43
ตาราง 4.1 ผลการทดสอบ Detection Model หลังเทรนกับ 22-Class Thai Logos.....	50
ตาราง 4.2 ผลการทดสอบ Detection Model หลังเทรนกับ FlickrLogos-47 Dataset	54
ตาราง 4.3 ผลการทดสอบ Hyperparameter tuning classification model กับ 22-Class Thai Logos.....	56
ตาราง 4.4 ผลการทดสอบ Hyperparameter tuning classification model กับ FlickrLogos-47.....	57
ตาราง 4.5 ผลการทดสอบ Fine-tuning classification model กับ 22-Class Thai Logos.....	61
ตาราง 4.6 ผลการทดสอบ Fine-tuning classification model กับ FlickrLogos-47	65
ตาราง 4.7 ผลการทดสอบการเทรน Custom CNN กับ FlickrLogos-47 และ 22-Class Thai Logos.....	68
ตาราง 4.8 ผลการทดสอบประสิทธิภาพของ Classification model กับ 22-Class Thai Logos เมื่อผ่าน YOLOv8	71
ตาราง 4.9 ผลการทดสอบประสิทธิภาพของ Classification model กับ FlickrLogos-47 เมื่อผ่าน YOLOv8	76
ตาราง 4.10 ผลการทดสอบระบบ Logo Management.....	77
ตาราง 4.11 ผลการทดสอบระบบ Logo Management.....	77

สารบัญรูป

รูป	หน้า
รูป 2.1 เปรียบเทียบพื้นที่ซ้อนทับระหว่างกรอบที่เป็นค่าจริงและกรอบที่เกิดจากการทำนาย.....	9
รูป 2.2 Class Activation Map	10
รูป 2.3 ขั้นตอนการทำ Class Activation Mapping	11
รูป 2.4 เปรียบเทียบระหว่าง Relu และ Leaky Relu.....	12
รูป 2.5 Leaky Rely Function	12
รูป 2.6 Squeeze-and-Excitation Block	12
รูป 2.7 โครงสร้าง Convolutional Block Attention Module (CBAM).....	13
รูป 2.8 เปรียบเทียบการเลือก Parameter ระหว่าง Grid Search และ Random Search.....	14
รูป 2.9 กราฟแสดงแนวโน้มของ Gradient เมื่อเกิด Vanishing Gradient	15
รูป 2.10 Timeline การพัฒนาโมเดล Logo Detection ในแต่ละปี.....	15
รูป 2.11 กระบวนการ Pre-training ของภาพ	17
รูป 2.12 Dual assignments แบบสอดคล้องกันสำหรับ NMS-free training	18
รูป 2.13 เปรียบเทียบขนาดและประสิทธิภาพของโมเดล CNN ที่เป็นที่นิยม.....	20
รูป 3.1 การทำ Data Annotation ด้วยโปรแกรม LabelImg	22
รูป 3.2 โครงสร้างของ Classification Model ที่ทำการเพิ่ม CNN Layer	27
รูป 3.3 โครงสร้างของโมเดล EfficientNetV2 ที่ทำการปรับแต่ง Layer	28
รูป 3.4 โครงสร้างของโมเดล MobileNetV3 ที่ทำการปรับแต่ง Layer.....	29
รูป 3.5 โครงสร้างของโมเดล InceptionV3 ที่ทำการปรับแต่ง Layer.....	29
รูป 3.6 โครงสร้างของโมเดล EfficientNetV2 ที่ทำการปรับแต่ง Layer	30
รูป 3.7 โครงสร้างของโมเดล MobileNetV3 ที่ทำการปรับแต่ง Layer.....	30
รูป 3.8 โครงสร้างของโมเดล ConvNeXT ที่ทำการปรับแต่ง Layer	31
รูป 3.9 โครงสร้างของโมเดล Custom CNN.....	31
รูป 3.10 Use Case Diagram ของระบบ	35
รูป 3.11 ภาพรวมการออกแบบเว็บแอปพลิเคชัน	36
รูป 3.12 Modal สำหรับการอัปโหลดรูปภาพ	37
รูป 3.13 ภาพรวมการออกแบบเว็บแอปพลิเคชัน	37
รูป 3.14 หน้า Annotate.....	38
รูป 3.15 หน้า Augmentation.....	39

สารบัญรูป (ต่อ)

รูป	หน้า
รูป 3.16 หน้าสร้าง Dataset	39
รูป 3.17 การเลือกโมเดลที่ต้องการใช้ทำนายผล	40
รูป 3.18 หน้าทำนายผลจากโมเดล	40
รูป 3.19 Sequence Diagram ของการจัดการ Class	41
รูป 3.20 Sequence Diagram ของการทำ Annotation	42
รูป 3.21 Sequence Diagram ของการ Prediction	42
รูป 3.22 ER diagram.....	43
รูป 4.1 กราฟ Class Loss ของ YOLOv8 กับ Flickr Logos-47.....	45
รูป 4.2 กราฟ Box Loss ของ YOLOv8 กับ Flickr Logos-47.....	45
รูป 4.3 กราฟแสดงค่า Metrics ในแต่ละ Epoch ของการเทรน YOLOv8 กับ 22-Class Thai Logos	46
รูป 4.4 ตัวอย่างการทำนายกับชุด Test ของ YOLOv8 กับ 22-Class Thai Logos.....	46
รูป 4.5 กราฟแสดงค่า Metric ในการทำนายกับชุด Test ของ YOLOv8 กับ 22-Class Thai Logos.	47
รูป 4.6 กราฟแสดงค่า Metrics ในแต่ละ Epoch ของการเทรน YOLOv10 กับ 22-Class Thai Logos	47
รูป 4.7 ตัวอย่างการทำนายกับชุด Test ของ YOLOv10 กับ 22-Class Thai Logos.....	48
รูป 4.8 กราฟแสดงค่า Metric ในการทำนายกับชุด Test ของ YOLOv10 กับ 22-Class Thai Logos	48
รูป 4.9 กราฟแสดงค่า Metrics ในแต่ละ Epoch ของการเทรน RT-DETR กับ.....	49
รูป 4.10 ตัวอย่างการทำนายกับชุด Test ของ RT-DETR กับ 22-Class Thai Logos.....	49
รูป 4.11 กราฟแสดงค่า Metric ในการทำนายกับชุด Test ของ RT-DETR กับ 22-Class Thai Logos	50
รูป 4.12 กราฟแสดงค่า Metrics ในแต่ละ Epoch ของการเทรน YOLOv8 กับ FlickrLogos-47	51
รูป 4.13 ตัวอย่างการทำนายกับชุด Test ของ YOLOv8 กับ FlickrLogos-47	51
รูป 4.14 กราฟแสดงค่า Metric ในการทำนายกับชุด Test ของ YOLOv8 กับ FlickrLogos-47	51
รูป 4.15 กราฟแสดงค่า Metrics ในแต่ละ Epoch ของการเทรน YOLOv10 กับ FlickrLogos-47	52
รูป 4.16 ตัวอย่างการทำนายกับชุด Test ของ YOLOv10 กับ FlickrLogos-47	52
รูป 4.17 กราฟแสดงค่า Metric ในการทำนายกับชุด Test ของ YOLOv10 กับ FlickrLogos-47	53
รูป 4.18 กราฟแสดงค่า Metrics ในแต่ละ Epoch ของการเทรน RT-DETR กับ FlickrLogos-47.....	53
รูป 4.19 ตัวอย่างการทำนายกับชุด Test ของ RT-DETR กับ FlickrLogos-47	54

สารบัญรูป (ต่อ)

รูป	หน้า
รูป 4.20 กราฟแสดงค่า Metric ในการทำนายกับชุด Test ของ RT-DETR กับ FlickrLogos-47	54
รูป 4.21 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ EfficientNetV2 กับ 22-Class Thai Logos.....	58
รูป 4.22 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ EfficientNetV2 กับ 22-Class Thai Logos	59
รูป 4.23 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ InceptionResNetV2 กับ 22-Class Thai Logos	59
รูป 4.24 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ InceptionResNetV2 กับ 22-Class Thai Logos.....	60
รูป 4.25 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ MobileNetV3 กับ 22-Class Thai Logos.....	60
รูป 4.26 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ MobileNetV3 กับ 22-Class Thai Logos	61
รูป 4.27 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ EfficientNetV2 กับ FlickrLogos-47	62
รูป 4.28 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ EfficientNetV2 กับ FlickrLogos-47	62
รูป 4.29 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ ConvNext กับ FlickrLogos-47	63
รูป 4.30 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ ConvNext กับ FlickrLogos-47	63
รูป 4.31 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ MobileNetV3 กับ FlickrLogos-47	64
รูป 4.32 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ MobileNetV3 กับ FlickrLogos-47 ..	64
รูป 4.33 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ Custom CNN กับ 22-Class Thai Logos.....	66
รูป 4.34 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ Custom CNN กับ 22-Class Thai Logos	66

สารบัญรูป (ต่อ)

รูป	หน้า
รูป 4.35 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ Custom CNN กับ FlickrLogos-47	67
รูป 4.36 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ Custom CNN กับ FlickrLogos-47..	67
รูป 4.37 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ EfficientNetV2 กับ 22-Class Thai Logos	69
รูป 4.38 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ InceptionV3 กับ 22-Class Thai Logos.....	69
รูป 4.39 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ MobileNetV3 กับ 22-Class Thai Logos	70
รูป 4.40 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ Custom CNN กับ 22-Class Thai Logos.....	71
รูป 4.41 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ EfficientNetV2 กับ FlickrLogos-47	72
รูป 4.42 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ ConvNeXt กับ FlickrLogos-47	73
รูป 4.43 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ MobileNetV3 กับ FlickrLogos-47	74
รูป 4.44 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ Custom CNN กับ FlickrLogos-47	75
รูป 4.45 หน้าทำนายผลจากโมเดล	76

บทที่ 1

บทนำ

1.1 ความเป็นมาของปัญหา

การตรวจสอบโลโก้มีบทบาทสำคัญในการติดตามและวิเคราะห์ชื่อเสียงของแบรนด์ทางออนไลน์ รวมไปถึงการตรวจสอบโซเชียลมีเดีย ฟอรัม และแพลตฟอร์มอื่น ๆ ที่มีการกล่าวถึง แบรนด์ทั้งทางตรงและทางอ้อม ซึ่งช่วยให้องค์กรสามารถมองเห็นภัยคุกคามที่อาจส่งผลกระทบต่อชื่อเสียงของแบรนด์ทั้งในปัจจุบันและอนาคต และยังสามารถตอบสนองต่อปัญหาได้อย่างรวดเร็ว การติดตามนี้ไม่เพียงแต่ช่วยในการวิเคราะห์ความรู้สึกของลูกค้าเกี่ยวกับแบรนด์เท่านั้น แต่ยังช่วยในการรักษาภาพลักษณ์เชิงบวกของแบรนด์และสร้างความไว้วางใจในกลุ่มลูกค้า

อย่างไรก็ตาม ปัญหาที่ไม่ควรมองข้ามคือการแอบอ้างนำโลโก้ไปใช้โดยไม่ได้รับอนุญาต การละเมิดสิทธิ์ในลักษณะนี้พบได้ทั้งในภาคเอกชนและภาครัฐ ซึ่งอาจส่งผลกระทบต่อภาพลักษณ์และชื่อเสียงขององค์กร โดยเฉพาะอย่างยิ่งในยุคที่ข้อมูลข่าวสารมีการแพร่กระจายอย่างรวดเร็ว การใช้โลโก้โดยไม่ได้รับอนุญาตอาจทำให้ผู้บริโภคเกิดความสับสนและไม่ไว้วางใจแบรนด์ได้นอกจากนี้ การละเมิดสิทธิ์ยังอาจนำไปสู่การสูญเสียรายได้และความเสียหายทางกฎหมายที่อาจเกิดขึ้นตามมา เช่น ค่าชดเชยหรือค่าปรับ ซึ่งส่งผลกระทบต่อเสถียรภาพทางการเงินขององค์กร อีกทั้งการติดตามการใช้โลโก้ด้วยวิธีการแบบดั้งเดิมอาจมีความยุ่งยากและใช้เวลานาน ส่งผลให้การพัฒนาระบบที่สามารถตรวจจับโลโก้ได้อย่างแม่นยำและรวดเร็วเป็นสิ่งจำเป็น

เพื่อแก้ไขปัญหาดังกล่าว เทคโนโลยี Computer Vision ที่นับว่าเป็นหัวใจสำคัญของระบบตรวจจับโลโก้อัตโนมัติ โดยอาศัยการเรียนรู้เชิงลึก (Deep Learning) และ Convolutional Neural Network (CNN) รวมไปถึงเทคโนโลยีอื่น ๆ ที่เกี่ยวข้องในการวิเคราะห์ภาพและวิดีโอ ซึ่งจะเรียนรู้ลักษณะเฉพาะของโลโก้ที่ต้องการตรวจจับ เช่น รูปร่าง สี และโครงสร้าง จากนั้นจึงนำสิ่งที่ได้เรียนรู้ไปเปรียบเทียบกับภาพใหม่ ๆ ที่เข้ามา เพื่อตรวจสอบว่ามีโลโก้ที่ตรงกับฐานข้อมูลหรือไม่ ซึ่งเทคโนโลยีนี้มีความแม่นยำสูงและสามารถตรวจจับโลโก้ได้แม้ในสภาพแวดล้อมที่ซับซ้อนและมีการบิดเบือน ทำให้สามารถตอบสนองต่อการเปลี่ยนแปลงในตลาดและป้องกันความเสียหายต่อชื่อเสียงด้วยการติดตามการปรากฏของโลโก้ได้อย่างรวดเร็วและมีประสิทธิภาพ อีกทั้งยังช่วยลดการใช้ทรัพยากรในกระบวนการตรวจสอบ และสามารถเพิ่มความสามารถในการจัดการปัญหาเชิงกลยุทธ์ขององค์กรได้อีกด้วย

1.2 วัตถุประสงค์ของโครงการ

- 1) เพื่อศึกษา สำรวจ และเปรียบเทียบวิธีการตรวจจับโลโก้ด้วยโมเดล Logo Detection และวิธีการจำแนกประเภทโลโก้ด้วยโมเดล Logo Classification ที่มีประสิทธิภาพในปัจจุบัน
- 2) เพื่อพัฒนาและปรับปรุงวิธีการตรวจจับโลโก้ให้มีความแม่นยำและรวดเร็ว
- 3) เพื่อออกแบบและพัฒนาเว็บแอปพลิเคชันสำหรับการเรียกใช้งานโมเดลและอำนวยความสะดวกในการจัดเตรียมข้อมูล
- 4) เพื่อเผยแพร่งานวิจัยสู่สาธารณะในรูปแบบของการตีพิมพ์บทความวิจัยเป็นกรณีศึกษาต่อไป

1.3 ขอบเขตของโครงการ

- 1) สามารถตรวจจับและทำนายแบรนด์ของโลโก้ได้อย่างแม่นยำ
- 2) สามารถเปรียบเทียบประสิทธิภาพของวิธีการตรวจจับโลโก้ที่มีอยู่ในปัจจุบัน
- 3) สามารถปรับปรุงและพัฒนาประสิทธิภาพ ทั้งในด้านของความแม่นยำและความเร็วของการตรวจจับและทำนายแบรนด์ของโลโก้
- 4) รองรับการนำเข้ารูปภาพโลโก้ในรูปแบบไฟล์ .jpg และ .png ได้ โดยรองรับได้สูงสุดไม่ต่ำกว่า 50 ภาพต่อการนำเข้า 1 ครั้ง
- 5) สามารถเลือกโมเดลหรือเปรียบเทียบประสิทธิภาพของโมเดลที่ต้องการเพื่อใช้ในการทำนายผลผ่านหน้าเว็บแอปพลิเคชัน
- 6) ไม่สามารถตรวจจับและทำนายโลโก้ที่ไม่ชัดเจนหรือไม่เป็นที่รู้จักได้

1.4 ขั้นตอนการดำเนินงาน

- 1) ศึกษาค้นคว้าทฤษฎีและงานวิจัยที่เกี่ยวข้อง
- 2) ค้นหาและจัดเตรียม Dataset และเครื่องมือที่ใช้
- 3) จัดเตรียมข้อมูลให้พร้อมสำหรับการเทรนโมเดล
- 4) เปรียบเทียบและเลือกโมเดลที่จะนำมาพัฒนา
- 5) พัฒนาโมเดล Logo Detection
- 6) พัฒนา Logo Classifier
- 7) พัฒนาเว็บแอปพลิเคชัน
- 8) ทดสอบและวัดประสิทธิภาพ
- 9) ประเมินผลและสรุปผล
- 10) เขียนต้นฉบับบทความวิจัย

ตาราง 1.1 แผนการดำเนินงาน

INDEX	ACTIVITY	ASSIGNED	START	END	DURATION	COMPLETION	Jul-24			Aug-24			Sep-24			Oct-24			Nov-24			Dec-24			Jan-25			Feb-25			Mar-25										
			DATE	DATE	(Week)	(%)	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
1	ศึกษาหน้าคว่ำ																																								
1.1	ศึกษาเรื่อง Transformer	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	1-Jul	7-Jul		1	20%																																		
1.2	ศึกษาเรื่อง Transfer Learning	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	1-Jul	7-Jul		1	20%																																		
1.3	ศึกษาการวัดประสิทธิภาพของโมเดล	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	1-Jul	7-Jul		1	20%																																		
1.4	ศึกษาเรื่อง Image Augmentation	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	1-Jul	7-Jul		1	5%																																		
1.5	ศึกษา YOLOv8	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	8-Jul	14-Jul		1	5%																																		
1.6	ศึกษา DETR	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	8-Jul	14-Jul		1	5%																																		
1.7	ศึกษา RT-DETR	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	8-Jul	14-Jul		1	5%																																		
1.8	ศึกษา MobileNetV2	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	15-Jul	21-Jul		1	15%																																		
1.9	ศึกษา InceptionV4	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	15-Jul	21-Jul		1	15%																																		
1.10	ศึกษา EfficientNet	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	15-Jul	21-Jul		1	15%																																		
1.11	ศึกษา ViT	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	15-Jul	21-Jul		1	15%																																		
1.12	ศึกษา ConvNeXt	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	15-Jul	21-Jul		1	15%																																		
2	ค้นหา Dataset และเครื่องมือที่จะใช้																																								
2.1	การเตรียม Dataset	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	22-Jul	4-Aug		2	10%																																		
2.2	การทำ Data Annotation	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	22-Jul	4-Aug		2	0%																																		
3	การจัดเตรียมข้อมูล																																								
3.1	การทำ Image Augmentation	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	5-Aug	7-Aug		0.5	0%																																		
3.2	การจัดเตรียมข้อมูลให้พร้อมสำหรับการเทรนโมเดล	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	5-Aug	7-Aug		0.5	0%																																		
4	การพัฒนา Logo Detection																																								
4.1	การพัฒนาโมเดลผ่าน YOLOv8	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	8-Aug	21-Aug		2	25%																																		
4.2	การพัฒนาโมเดลผ่าน DETR	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	22-Aug	8-Sep		2.5	25%																																		
4.3	การพัฒนาโมเดลผ่าน RT-DETR	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	9-Sep	18-Sep		1.5	25%																																		
5	การพัฒนา Logo Classifier																																								
5.1	การพัฒนาโมเดลผ่าน MobileNetV2	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	19-Sep	6-Oct		2.5	0%																																		
5.2	การพัฒนาโมเดลผ่าน InceptionV4	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	7-Oct	23-Oct		2.5	0%																																		
5.3	การพัฒนาโมเดลผ่าน EfficientNet	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	24-Oct	10-Nov		2.5	0%																																		
5.4	การพัฒนาโมเดลผ่าน ViT	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	11-Nov	27-Nov		2.5	0%																																		
5.5	การพัฒนาโมเดลผ่าน ConvNeXt	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	28-Nov	15-Dec		2.5	0%																																		
6	การพัฒนา Web Application																																								
6.1	ออกแบบ User Interface	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	1-Jul	9-Feb		34	100%																																		
6.2	พัฒนา Web Application	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	1-Jul	9-Feb		34	0%																																		
7	การทดสอบและวัดประสิทธิภาพ																																								
7.1	ทดสอบและวัดประสิทธิภาพของโมเดล	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	10-Feb	23-Feb		2	0%																																		
7.2	ทดสอบและวัดประสิทธิภาพของ Web Application	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	10-Feb	23-Feb		2	0%																																		
8	Testing and Integration																																								
8.1	การประเมินผลและสรุปผล	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	10-Feb	23-Feb		2	0%																																		
9	เขียนต้นฉบับทบทวนวิจัย	ณัฏพร, สุวัฑิตี, ศุภวิชญ์, สุทธิชา	24-Feb	23-Mar		4	0%																																		

1.5 ประโยชน์ที่คาดว่าจะได้รับ

- 1) สามารถลดเวลาสำหรับการติดตามโลโก้ที่ปรากฏในสื่อออนไลน์ช่องทางต่าง ๆ
- 2) สามารถลดเวลาในการจัดเตรียมข้อมูลและแสดงผลภาพการตรวจจับโลโก้และจำแนกโลโก้ผ่านหน้าเว็บแอปพลิเคชันได้อย่างชัดเจน
- 3) สามารถเปรียบเทียบประสิทธิภาพของวิธีในการตรวจจับโลโก้และจำแนกโลโก้แต่ละโมเดลได้อย่างชัดเจน ผ่านชุดข้อมูลที่ใช้ในการวัดประสิทธิภาพสำหรับการตรวจจับและจำแนกโลโก้อย่างแพร่หลาย เพื่อเป็นแนวทางในการเลือกโมเดลที่เหมาะสมในการทำ Logo Detection และ Logo Classification
- 4) เผยแพร่งานวิจัยสู่สาธารณะในรูปแบบของการตีพิมพ์บทความวิจัย

บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

2.1 ทฤษฎีที่เกี่ยวข้อง

2.1.1 Convolutional Neural Network (CNN)

Convolutional Neural Network (CNN) เป็นโครงสร้างของโครงข่ายประสาทเทียมที่ถูกออกแบบมาเพื่อการจำแนกประเภทและการตรวจจับวัตถุในภาพ โดยมีการนำเอาหลักการของการทำงานของเซลล์ประสาทในสมองมนุษย์มาใช้ในการประมวลผลข้อมูลภาพ เพื่อให้สามารถระบุลักษณะและลักษณะของวัตถุในภาพได้อย่างแม่นยำ ซึ่งจะประกอบด้วยชั้นต่าง ๆ ดังนี้

2.1.1.1 Convolutional Layer

- 1) Convolutional layer มีหน้าที่ในการสกัดลักษณะเด่น (Feature extraction) จากภาพ โดยใช้การทำ convolution ระหว่างภาพข้อมูลและก้อนกรอง (Filter/Kernel) ที่มีขนาดเฉพาะ
- 2) การทำ convolution ช่วยให้โครงข่ายสามารถตรวจจับลักษณะเด่นเช่น เส้นขอบ ลายเส้น หรือลักษณะเด่นอื่น ๆ ในภาพ

2.1.1.2 Pooling Layer

- 1) Pooling layer เป็นส่วนที่ใช้ในการลดขนาดของข้อมูลภาพที่ผ่านการสกัดลักษณะเด่น (Feature map) ลงเพื่อลดการคำนวณและความซับซ้อนของโมเดล
- 2) รูปแบบการทำ pooling ที่พบบ่อยได้แก่ max pooling และ average pooling

2.1.1.3 Activation Layer

- 1) Activation function เป็นฟังก์ชันที่ใช้ในการเพิ่มความเชื่อมั่นในการตัดสินใจของโมเดล
- 2) ส่วนใหญ่จะใช้ ReLU (Rectified Linear Unit) เป็น activation function ใน CNN เนื่องจากมีประสิทธิภาพและคำนวณได้รวดเร็ว

2.1.1.4 Fully Connected Layer

- 1) Fully connected layer เป็นส่วนที่ใช้ในการรวมลักษณะเด่นที่ถูกสกัดจากภาพให้กับแต่ละกลุ่มวัตถุในภาพ
- 2) มักถูกใช้ในการวิเคราะห์ลักษณะของวัตถุในภาพ และประมวลผลเพื่อการจำแนกประเภท

2.1.1.5 Output Layer

- 1) Output layer เป็นส่วนที่ใช้ในการกำหนดค่าผลลัพธ์สุดท้ายของโมเดล CNN

2.1.2 Transfer Learning

Transfer Learning เป็นเทคนิคใน Machine Learning ที่นำความรู้ที่ได้จากการเทรนโมเดลจากงานเดิมไปประยุกต์ใช้กับงานใหม่ที่เกี่ยวข้องกัน โดยพื้นฐานแล้วการ Transfer Learning คือการนำความรู้ที่โมเดลได้ถูกเทรนจากงานที่มีข้อมูลจำนวนมาก ไปใช้กับงานใหม่ที่มีจำนวนข้อมูลน้อย โดยสามารถใช้รูปแบบ (pattern) ที่ทำการเทรนมาก่อนหน้ามาเป็นพื้นฐานแทนที่จะทำการเทรนใหม่ทั้งหมด

เทคนิคนี้มักถูกนำไปใช้ในงานด้าน Computer vision และ Natural language processing เช่น การวิเคราะห์ความรู้สึก (sentiment analysis) เนื่องจากงานเหล่านี้ต้องการความสามารถในการคำนวณเป็นอย่างมาก

2.1.2.1 Pre-training

เป็นการเทรนโมเดล Machine Learning ด้วยข้อมูลจำนวนมากก่อนนำมา fine-tuning เพื่อให้โมเดลมีความสามารถทั่วไปและสามารถใช้งานในหลาย ๆ งานได้โดยไม่ต้องเทรนใหม่ตั้งแต่ต้น การ pre-training จะช่วยลดเวลาในการเทรนโมเดล

2.1.2.2 Fine-tuning

เป็นกระบวนการที่ใช้ในการปรับแต่งโมเดล Machine Learning โดยใช้ชุดข้อมูลที่เฉพาะของงานหรือปัญหาที่ต้องการแก้ไข เป้าหมายของ fine-tuning คือการปรับพารามิเตอร์ของโมเดลให้เหมาะสมกับงานหรือปัญหาที่ต้องการแก้ไขโดยเฉพาะ ซึ่งการ fine-tuning จะช่วยเพิ่มประสิทธิภาพของโมเดลในการทำนายหรือการจำแนกข้อมูลในงานหรือปัญหาที่เฉพาะเจาะจงได้มากขึ้นโดยไม่ต้องเริ่มต้นการเทรนโมเดลใหม่ทั้งหมด

2.1.3 Transformer

สถาปัตยกรรม Transformer เป็นหนึ่งในรูปแบบที่สำคัญที่สุดของโครงข่ายประสาทเทียมในปัจจุบัน อาศัยทฤษฎีและแนวคิดหลักที่น่าสนใจหลายประการ กลไกการจดจำ (Attention Mechanism) ถือเป็นหัวใจสำคัญที่ทำให้ Transformer แตกต่างจากโครงข่ายประสาทเทียมแบบดั้งเดิม โดยคำนวณคะแนนความสนใจเพื่อกำหนดน้ำหนักความสำคัญระหว่างองค์ประกอบของข้อมูลอินพุตช่วยให้สามารถจับความสัมพันธ์ระยะไกลได้อย่างมีประสิทธิภาพ

จุดเด่นสำคัญของ Transformer คือความสามารถในการคำนวณแบบคู่ขนานได้ทั้งหมด ซึ่งต่างจากการประมวลผลแบบลำดับของ RNN ทำให้มีประสิทธิภาพสูงมากสำหรับข้อมูลขนาดใหญ่และซีเควนซ์ยาว อันเป็นผลมาจากคุณสมบัติของกลไกการจดจำที่ไม่จำเป็นต้องอิงกับการคำนวณจากขั้นตอนก่อนหน้า นอกจากนี้ การเข้ารหัสเชิงตำแหน่ง (Positional Encoding) ก็เป็น

ส่วนสำคัญที่ทำให้โมเดลสามารถจำแนกข้อมูลอินพุตตามตำแหน่งได้ เนื่องจาก Transformer ไม่มี การประมวลผลตามลำดับ

สถาปัตยกรรม Transformer ยังมีบทบาทสำคัญในการพัฒนาทฤษฎีการเรียนรู้การ นำเสนอข้อมูลที่มีนัยสำคัญ โดยกลไกการจดจำและประสิทธิภาพการประมวลผลแบบคู่ขนานช่วย ให้โมเดลสร้างการนำเสนอข้อมูลที่สะท้อนความสัมพันธ์และรายละเอียดในข้อมูลได้อย่างลึกซึ้งจึง ได้รับการยอมรับและนำไปประยุกต์ใช้อย่างแพร่หลายในหลายสาขา ทั้งการประมวลผล ภาษาธรรมชาติ แปลภาษา สร้างข้อความ ตลอดจนการนำไปใช้ในด้าน Computer vision

2.1.4 Image Augmentation

Image Augmentation เป็นเทคนิคที่ใช้เพื่อขยายชุดข้อมูล โดยจะนำ Image Augmentation มาใช้ก็ต่อเมื่อมีข้อมูลปริมาณน้อย เนื่องจากในการทำ Machine Learning ปริมาณ ข้อมูลมีผลต่อค่าความแม่นยำของโมเดล อีกทั้งยังอาจส่งผลให้เกิดการ overfit ของโมเดลขึ้นหากชุด ข้อมูลมีขนาดเล็กเกินไป พารามิเตอร์ในการ augmentation ภาพ มีดังนี้

2.1.4.1 rotation_range

มุมการหมุนภาพแบบสุ่มในช่วงที่กำหนด (degrees)

2.1.4.2 width_shift_range

การเลื่อนภาพซ้าย-ขวา

2.1.4.3 height_shift_range

การเลื่อนภาพขึ้น-ลง

2.1.4.4 brightness_range

ช่วงความสว่างที่จะเปลี่ยนแปลง

2.1.4.5 shear_range

การบิดภาพ

2.1.4.6 zoom_range

การซูมภาพ

2.1.4.7 channel_shift_range

เปลี่ยนแปลงค่าสีแต่ละช่องสี

2.1.4.8 horizontal_flip

ทำการพลิกซ้าย-ขวา

2.1.4.9 vertical_flip

ทำการพลิกขึ้น-ลง

2.1.4.10 rescale

ปรับค่าสีของภาพด้วยค่า scaling factor

2.1.4.11 fill_mode

วิธีการสำหรับการเติมข้อมูลหลังจากการเลื่อนหรือการชุม

2.1.5 Confusion Matrix

Confusion matrix คือเครื่องมือที่ใช้ประเมินประสิทธิภาพของ Classification Machine Learning รูปแบบต่าง ๆ ร่วมกับชุดข้อมูลทดสอบที่ทราบค่าหรือผลลัพธ์ที่แท้จริงแล้ว ด้วยการประเมินค่าจริง (Actual value) เทียบกับค่าที่ได้จากผลลัพธ์ของ Machine Learning Model หรือค่าที่คาดการณ์ไว้ (Predicted Value) แล้วสรุปค่าออกมาในรูปแบบของตารางเมทริกซ์ ประกอบด้วย

ตาราง 2.1 Confusion matrix ขนาด 2*2

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positive (TP)	False Positive (FP)
Predicted Negative (0)	False Negative (FN)	True Negative (TN)

- 1) ค่า True Positives (TP) คือ จำนวนข้อมูลที่ถูกจำแนกได้อย่างถูกต้องในกลุ่มของประเภทที่ต้องการ (Positive class) หรือกลุ่มที่สนใจ ในกรณีนี้คือข้อมูลที่ถูกทำนายว่าเป็น Positive หรือเป็นแบบนี้จริง และข้อมูลจริงก็เป็นแบบนี้จริง หรือก็คือการทำนายถูกต้องว่าเป็นสิ่งที่ประเมินไว้
- 2) True Negatives (TN) คือ จำนวนข้อมูลที่ถูกจำแนกถูกต้องในกลุ่มของประเภทที่ไม่ใช่เป้าหมาย (Negative class) หรือกลุ่มที่ไม่ได้สนใจ ในกรณีนี้คือข้อมูลที่ถูกทำนายว่าเป็น Negative และข้อมูลจริงก็เป็น Negative นั่นคือการทำนายถูกต้องว่าไม่เป็นสิ่งที่ประเมินไว้
- 3) False Positives (FP) คือ จำนวนข้อมูลที่ถูกทำนายผิดในกลุ่มของประเภทที่สนใจ ในกรณีนี้คือข้อมูลที่ถูกทำนายว่าเป็น Positive แต่ข้อมูลจริงกลับเป็น Negative นั่นคือการทำนายผิดว่าเป็นสิ่งที่ทำการประเมินไว้ หรือ False
- 4) False Negatives (FN) คือ จำนวนข้อมูลที่ถูกจำแนกผิดในกลุ่มของประเภทที่ไม่สนใจ ในกรณีนี้คือข้อมูลที่ถูกทำนายว่าเป็น Negative แต่ข้อมูลจริงกลับเป็น Positive นั่นคือการทำนายผิดว่าไม่เป็นสิ่งที่ประเมินไว้

จากการทำ Confusion Matrix สามารถนำไปใช้ประเมินประสิทธิภาพของโมเดล Machine Learning ผ่านการคำนวณค่าพารามิเตอร์ที่ใช้ประเมิน 4 ค่า ได้แก่

2.1.5.1 Accuracy

ค่าความแม่นยำเป็นสัดส่วนของข้อมูลทั้งหมดที่ถูกจำแนก (Classify) ถูกต้องโดยโมเดล

$$Accuracy = \frac{True\ Positive + True\ Negative}{True\ Positive + True\ Negative + False\ Positive + False\ Negative} \quad (2.1)$$

2.1.5.2 Precision

ค่าความแม่นยำในการทำนายกลุ่มเป้าหมาย โดยพิจารณาเฉพาะ True Positive (TP)

$$Precision = \frac{True\ Positive}{True\ Positive + False\ Positive} \quad (2.2)$$

2.1.5.3 Recall

ความสามารถของโมเดลในการตรวจจับเหตุการณ์ที่สนใจ โดยพิจารณาจากข้อมูลที่เป็น Actually Positive

$$Recall = \frac{True\ Positive}{True\ Positive + False\ Negative} \quad (2.3)$$

2.1.5.4 F1-Score

ค่าประสิทธิภาพโดยรวมระหว่างความแม่นยำในการทำนายในกลุ่มเป้าหมาย (Precision) และความสามารถในการตรวจจับเหตุการณ์ที่สนใจ (Recall) มาหาค่าเฉลี่ยแบบ Harmonic Mean

$$F1 - Score = \frac{2(Precision \times Recall)}{Precision + Recall} \quad (2.4)$$

2.1.6 AUC (Area Under the Curve) / ROC (Receiver Operating Characteristic)

AUC (Area Under the Curve) และ ROC (Receiver Operating Characteristic) เป็นเครื่องมือที่ใช้ในการประเมินประสิทธิภาพของโมเดลในการจำแนกประเภท โดยเฉพาะในกรณีที่มีข้อมูลที่ไม่สมดุล

2.1.6.1 ROC Curve

แสดงความสัมพันธ์ระหว่างอัตราการตรวจจับจริง (True Positive Rate) และอัตราการตรวจจับเท็จ (False Positive Rate) ของโมเดลในระดับต่าง ๆ ของเกณฑ์การตัดสินใจ

คำนวณค่า Average Precision (AP) สำหรับแต่ละคลาสวัตถุ โดยเป็นค่าเฉลี่ยของ Precision คำนวณที่ค่า Recall ต่างๆ โดยใช้เทคนิคการคำนวณพื้นที่ใต้กราฟของ Precision-Recall Curve

2.1.6.2 AUC

ค่าที่ได้จากพื้นที่ใต้ ROC Curve ซึ่งมีค่าตั้งแต่ 0 ถึง 1 โดยค่าที่ใกล้เคียง 1 แสดงถึงประสิทธิภาพที่ดีของโมเดลในการจำแนกประเภท ในขณะที่ค่าที่ต่ำกว่า 0.5 แสดงว่าโมเดลมีประสิทธิภาพต่ำกว่าการสุ่ม

2.1.7 IoU (Intersection over Union)

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}} \quad (2.5)$$

การวัดประสิทธิภาพของโมเดลสำหรับการทำ Object Detection โดยคำนวณจากเปอร์เซ็นต์ของพื้นที่ซ้อนทับกันระหว่างกรอบที่เป็นค่าจริง (Ground-truth Bounding Box) และกรอบที่เกิดจากการทำนาย (Predicted Bounding Box) กับพื้นที่ทั้งหมด



รูป 2.1 เปรียบเทียบพื้นที่ซ้อนทับระหว่างกรอบที่เป็นค่าจริงและกรอบที่เกิดจากการทำนาย

จากรูป 2.1 จะสังเกตได้ว่าหากมีค่า IoU สูง หมายความว่าโมเดลมีประสิทธิภาพในการตรวจจับวัตถุได้ดี โดยส่วนใหญ่จะวัดค่าประสิทธิภาพของโมเดลที่ค่า $IoU = 0.5$ และ ที่ค่า $0.5 < IoU < 0.95$ แล้วจึงนำค่าที่ได้ในแต่ละช่วงมาเฉลี่ยกัน

2.1.8 mAP (mean Average Precision)

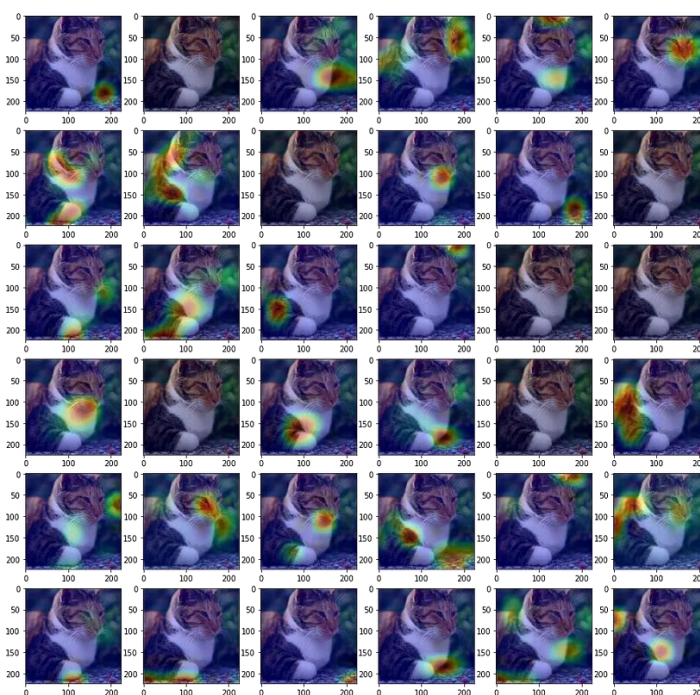
mAP หรือ mean Average Precision เป็นค่าวัดประสิทธิภาพมาตรฐานที่ใช้ในการวัดประสิทธิภาพของโมเดลสำหรับการทำ Object Detection โดยเป็นการประเมินทั้งความแม่นยำในการตรวจจับวัตถุ (Precision) และความครอบคลุมของวัตถุที่สามารถตรวจจับได้ (Recall) ซึ่งมีขั้นตอนการคำนวณ ดังนี้

- 1) คำนวณค่า Precision และ Recall สำหรับแต่ละคลาสวัตถุ (Class)

- 2) คำนวณค่า Average Precision (AP) สำหรับแต่ละคลาสวัตถุ โดยเป็นค่าเฉลี่ยของ Precision คำนวณที่ค่า Recall ต่างๆ โดยใช้เทคนิคการคำนวณพื้นที่ใต้กราฟของ Precision-Recall Curve
- 3) คำนวณค่า mAP จากค่าเฉลี่ยของ AP ทุกคลาสวัตถุ โดยคำนวณจากการนำค่า AP ของแต่ละคลาสมารวมกันและหารด้วยจำนวนคลาสทั้งหมด

การคำนวณ mAP จะต้องกำหนดค่า IoU Threshold เพื่อระบุว่าการตรวจจับใดจะถูกนับเป็น True Positive หรือ False Positive โดยทั่วไปมักใช้ค่า IoU Threshold ที่ 0.5 และสามารถใช้อื่นๆมาคิดได้เช่นกัน โดยค่า mAP ที่ได้จะอยู่ระหว่าง 0 ถึง 1 โดยค่าที่สูงแสดงถึงประสิทธิภาพการตรวจจับวัตถุที่ดี

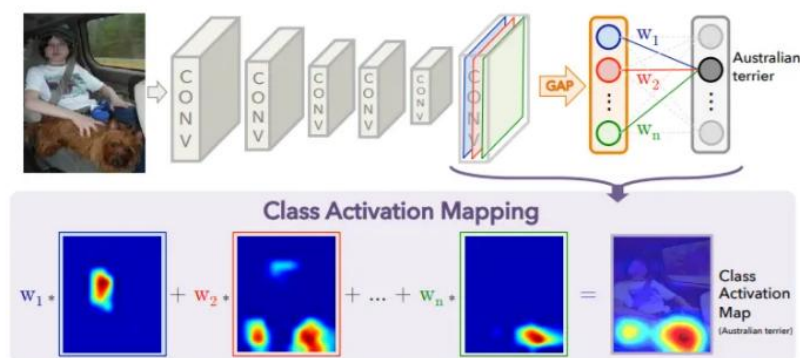
2.1.9 Class Activation Map (CAM)



รูป 2.2 Class Activation Map

Class Activation Map เป็นการหา feature ที่สำคัญที่ได้จากการทำแต่ละ node ในระหว่างการทำ CNN (Convolutional Neural Network) โดยการนำ CNN ส่วนมากจากประกอบด้วย Convolutional layer จำนวนหนึ่งตามด้วย Maxpooling layer และมี output หลังจากการทำสองขั้นนี้เป็นสองมิติ สิ่งที่ทำให้สูญเสียความเป็นสองมิติไปนั้นก็คือ Fully connected layer จึงทำการตัดขั้นนี้ทิ้ง เพื่อรักษาความเป็นสองมิติ และจะใส่ Global Average Pooling (GAP) มาแทนที่ มีขั้นตอนดังนี้

- 1) หาค่าเฉลี่ยในแต่ละ feature map ซึ่งมีจำนวนเท่ากับจำนวน channel ของชั้นก่อนหน้า เป็นการยุบจากข้อมูลสองมิติให้เหลือเพียงค่าเดียว แต่เนื่องจากมีหลาย map จึงได้จำนวน node ก่อนที่จะไปชั้นคำตอบสุดท้ายเท่ากับจำนวน channel
- 2) นำค่าที่ได้ไปคูณเข้ากับค่า weight เพื่อนำไปสู่ชั้นคำตอบ



รูป 2.3 ขั้นตอนการทำ Class Activation Mapping

ภายหลังการเทรน จะสามารถหาบริเวณที่ต้องการได้จากการนำ output ในชั้นสุดท้ายที่เป็น feature map สองมิติในแต่ละ channel มาคูณกับค่า weight แล้วนำมาบวกกัน

2.1.10 Logo Identification

การระบุโลโก้เป็นการหาโลโก้ที่มีอยู่ภายในภาพที่กำหนด ว่ามีอยู่ที่ตำแหน่งไหนจำนวนเท่าไร และเป็นโลโก้ใด ซึ่งการระบุโลโก้แบ่งออกเป็น 2 ส่วน ได้แก่ Logo Detection และ Logo Classification

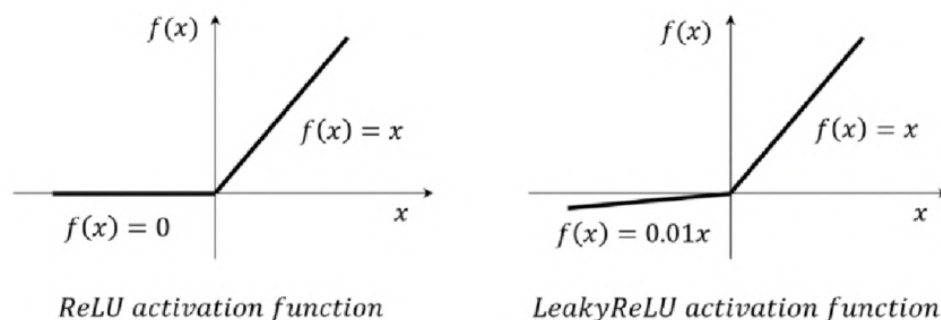
2.1.10.1 Logo Detection

การตรวจจับโลโก้เป็นกระบวนการค้นหาและระบุตำแหน่งของโลโก้ภายในภาพหรือวิดีโอ โดยการกำหนดกรอบ (Bounding Box) ในบริเวณที่อาจมีโลโก้อยู่ ซึ่งเป็นขั้นตอนเริ่มต้นของการระบุโลโก้ก่อนนำไปสู่การจำแนกโลโก้

2.1.10.2 Logo Classification

การจำแนกโลโก้เป็นกระบวนการต่อจากการตรวจจับโลโก้ โดยจะทำการจำแนกว่าภายในบริเวณที่มีการตรวจจับโลโก้เกิดขึ้น จะระบุว่าเป็นโลโก้ของแบรนด์ใด โดยใช้ Machine Learning เข้ามาช่วยในการจำแนกโลโก้แบรนด์ต่าง ๆ

2.1.11 Leaky ReLU



รูป 2.4 เปรียบเทียบระหว่าง Relu และ Leaky Relu

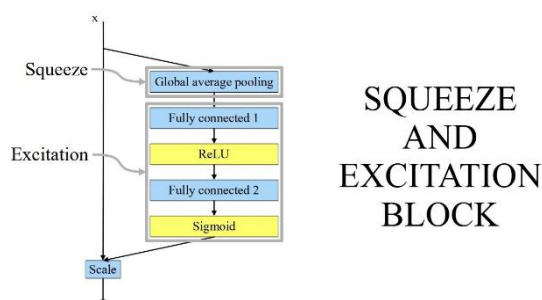
Leaky ReLU (Leaky Rectified Linear Unit) เป็นฟังก์ชันการเปิดใช้งานที่ถูกพัฒนาขึ้นเพื่อแก้ไขปัญหาของ ReLU ที่เรียกว่า "Dying ReLU" ซึ่งเกิดขึ้นเมื่อเซลล์ประสาทบางตัวไม่สามารถเรียนรู้ได้ในระหว่างการฝึกอบรม เนื่องจากมีค่าเป็นศูนย์ตลอดเวลา ฟังก์ชัน Leaky ReLU จะอนุญาตให้มีการไหลของค่าลบเล็กน้อยผ่านเซลล์ประสาทเมื่ออินพุตมีค่าน้อยกว่าศูนย์ โดยมีรูปแบบการคำนวณ ดังนี้

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases}$$

รูป 2.5 Leaky Rely Function

โดยที่ α เป็นค่าคงที่เล็กน้อย เช่น 0.01 ที่กำหนดให้กับค่าลบ เพื่อให้แน่ใจว่าเซลล์ประสาทยังคงสามารถเรียนรู้ได้แม้ในช่วงที่อินพุตมีค่าต่ำ

2.1.12 Squeeze-and-Excitation (SE)



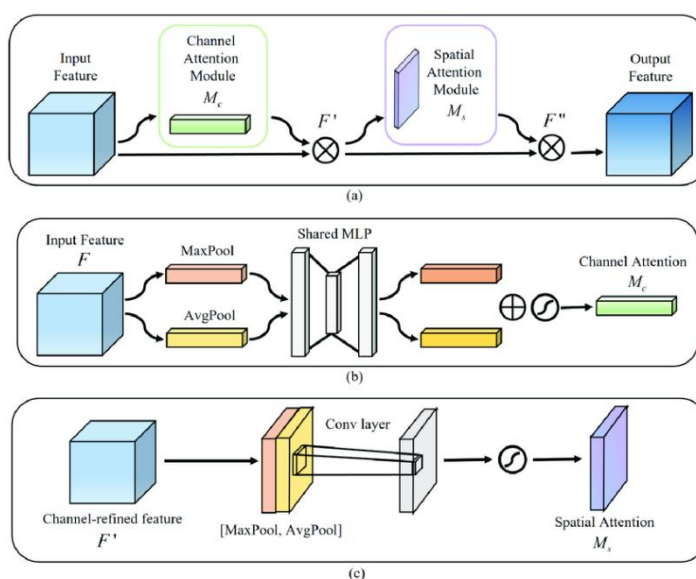
รูป 2.6 Squeeze-and-Excitation Block

Squeeze-and-Excitation (SE) เป็นโมดูลที่ออกแบบมาเพื่อปรับปรุงความสามารถในการแสดงผลของโครงข่ายประสาทเทียม โดยการทำงานของ SE จะมุ่งเน้นไปที่การปรับค่าความสำคัญของฟีเจอร์ในระดับช่อง (channel-wise) ผ่านกลไกการให้ความสนใจ ซึ่งประกอบด้วยขั้นตอนหลัก ดังนี้

- 1) Squeeze คือ การใช้ Global Average Pooling เพื่อลดมิติของข้อมูลจากรูปแบบ $C \times H \times W$ ไปเป็น $C \times 1 \times 1$ เพื่อสกัดข้อมูลที่สำคัญจากแต่ละช่อง
- 2) Excitation คือ การสร้างเวกเตอร์การปรับน้ำหนักสำหรับแต่ละช่องโดยใช้ Multi-Layer Perceptron (MLP) เพื่อเรียนรู้ความสำคัญของช่องต่าง ๆ จากนั้นจึงนำค่าที่ได้ไปคูณกับฟีเจอร์เดิมเพื่อให้เกิดการปรับน้ำหนักที่เหมาะสม

การใช้ SE Block ช่วยเพิ่มประสิทธิภาพในการจำแนกประเภทและการตรวจจับวัตถุในภาพ โดยเฉพาะในงานด้าน Computer Vision ที่ต้องการความสามารถในการรับรู้ลักษณะที่สำคัญอย่างมีประสิทธิภาพ

2.1.13 Convolutional Block Attention Module (CBAM)



รูป 2.7 โครงสร้าง Convolutional Block Attention Module (CBAM)

Convolutional Block Attention Module (CBAM) เป็นโมดูลที่ออกแบบมาเพื่อเพิ่มประสิทธิภาพของโครงข่ายประสาทเทียมโดยใช้กลไกการให้ความสนใจในสองมิติ ได้แก่ ช่อง (channel) และพื้นที่ (spatial) โดย CBAM จะทำงานตามขั้นตอน ดังนี้

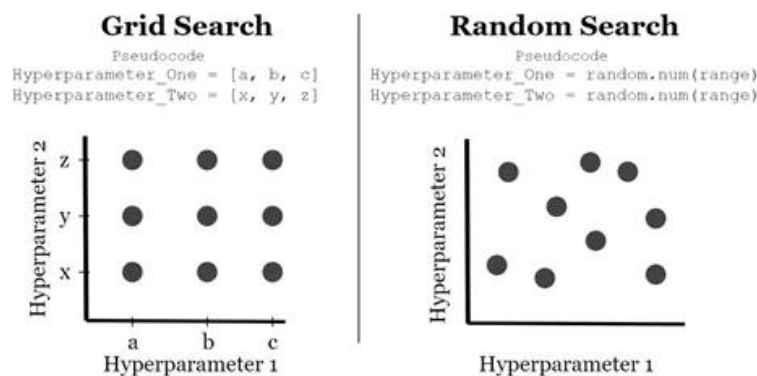
- 1) Channel Attention: เริ่มต้นด้วยการสร้างแผนที่ความสนใจในระดับช่อง โดยใช้ข้อมูลจาก Global Average Pooling และ Max Pooling เพื่อให้ได้ฟีเจอร์ที่ดีที่สุดในการสร้างแผนที่ความสนใจ

- 2) Spatial Attention: หลังจากได้แผนที่ความสนใจในระดับช่องแล้ว CBAM จะสร้างแผนที่ความสนใจในระดับพื้นที่ โดยใช้ข้อมูลจากพีเจอร์เดิมเพื่อกำหนดจุดที่ควรให้ความสำคัญมากขึ้น

ผลลัพธ์สุดท้ายจะเป็นพีเจอร์ที่ได้รับการปรับแต่งโดยการคูณแผนที่ความสนใจกับพีเจอร์เดิม ทำให้โมเดลสามารถจับข้อมูลที่สำคัญได้ดีขึ้นและลดเสียงรบกวนในข้อมูล

2.1.14 Hyperparameter Tuning

Hyperparameter Tuning คือกระบวนการปรับแต่งค่าพารามิเตอร์ภายนอกของโมเดล (Hyperparameters) เพื่อให้โมเดลมีประสิทธิภาพสูงสุด โดย Hyperparameter เหล่านี้ไม่ถูกเรียนรู้จากข้อมูลโดยตรง แต่ถูกกำหนดขึ้นก่อนการฝึกโมเดล เช่น ค่า Learning Rate, Batch Size, Number of Layers, และ Regularization Strength เป็นต้น โดยมักจะมีหลายวิธีที่นิยมใช้ในการปรับแต่ง Hyperparameter ดังนี้



รูป 2.8 เปรียบเทียบการเลือก Parameter ระหว่าง Grid Search และ Random Search

2.1.14.1 Grid Search

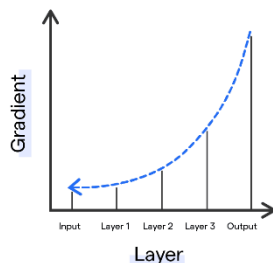
การหาค่าพารามิเตอร์ โดยมีวิธีการ คือ กำหนดค่าพารามิเตอร์ทั้งหมดที่ต้องการทดลองและทำการทดลองประเมินประสิทธิภาพหรือความแม่นยำของโมเดลไว้ในทุกชุดของพารามิเตอร์ เพื่อหาพารามิเตอร์ที่ดีที่สุดสำหรับโมเดลและข้อมูลที่ทำกรเทรนในขณะนั้น

2.1.14.2 Random Search

การทำงานของ Random Search คล้ายกับการทำ Grid Search แต่แทนที่จะลองใช้พารามิเตอร์ที่กำหนดไว้ล่วงหน้าในกริดทุกชุด Random Search จะทำการสุ่มเลือกค่าพารามิเตอร์จากกริดที่สร้างขึ้น ดังนั้นการทำ Random Search จะไม่รับประกันว่าเราจะได้โมเดลที่มีประสิทธิภาพที่สุดเหมือนกับ Grid Search แต่วิธีมีประสิทธิภาพสูงในการใช้งานจริงเนื่องจากใช้เวลาในการสร้างโมเดลที่น้อยมาก

2.1.15 Vanishing Gradient

Vanishing Gradient Problem



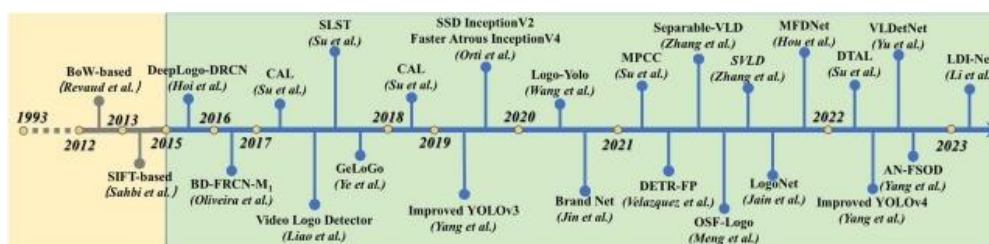
รูป 2.9 กราฟแสดงแนวโน้มของ Gradient เมื่อเกิด Vanishing Gradient

Vanishing Gradient เป็นปัญหาที่เกิดขึ้นในกระบวนการฝึกโครงข่ายประสาทเทียมแบบหลายชั้น (Deep Neural Networks – DNNs) โดยเฉพาะในโครงข่ายที่มีหลายชั้น เช่น Recurrent Neural Networks (RNNs) และ Deep Feedforward Networks

ปัญหานี้เกิดขึ้นเมื่อทำการอัปเดตน้ำหนักของโมเดลด้วย Backpropagation ซึ่งใช้ Gradient Descent ในการคำนวณค่าที่ใช้ปรับพารามิเตอร์ของโมเดล หาก Gradient มีค่าน้อยมาก (ใกล้ศูนย์) เมื่อย้อนกลับไปยังชั้นต้น ๆ ของโมเดล น้ำหนักในชั้นลึก ๆ จะถูกอัปเดตน้อยมากซึ่งส่งผลทำให้โมเดลเรียนรู้ได้ช้าหรืออาจหยุดเรียนรู้ไปเลย

2.2 งานวิจัยที่เกี่ยวข้อง

2.2.1 งานวิจัย “Deep Learning for Logo Detection: A Survey”



รูป 2.10 Timeline การพัฒนาโมเดล Logo Detection ในแต่ละปี

เอกสารวิจัยนี้เป็นการสำรวจโมเดลการตรวจจับโลโก้ตั้งแต่ปี ค.ศ. 2012 ถึงปลายปี ค.ศ. 2023 โดยเน้นไปที่ประสิทธิภาพของการตรวจจับโลโก้โดยใช้เทคนิคการเรียนรู้เชิงลึก (deep learning) ในงานวิจัยนี้ได้นำเสนอชุดข้อมูลที่ใช้ในการเทรนโมเดลต่าง ๆ รวมถึงวิธีการประมวลผล

ที่โมเดลแต่ละตัวใช้ และ Loss function ของโมเดลแต่ละตัว รวมถึงการวิเคราะห์ข้อดีและข้อเสียของแต่ละโมเดล เพื่อให้เกิดความเข้าใจในประสิทธิภาพและประสิทธิผลของโมเดลแต่ละตัว

โดยเริ่มต้นจากการนำเสนอชุดข้อมูลที่ใช้ในการเทรนโมเดล โดยแบ่งออกเป็นปีที่ใช้ในการสร้างชุดข้อมูลและชุดข้อมูลที่ถูกใช้ในการทดสอบโมเดล เพื่อให้ผู้อ่านเข้าใจถึงแหล่งข้อมูลที่ใช้ในการวิจัย ต่อมาเป็นการอธิบายเกี่ยวกับวิธีการประมวลผลที่ใช้ในแต่ละโมเดล รวมถึงการสร้างและเทรนโมเดล และการปรับแต่งพารามิเตอร์ต่าง ๆ ในโมเดลเพื่อให้ได้ประสิทธิภาพที่สูงสุดในส่วนของ Loss function ได้ทำการอธิบายถึงวิธีการใช้ Loss function เพื่อปรับค่าพารามิเตอร์ของโมเดลในระหว่างการเทรนและการเลือกใช้ Loss function ที่เหมาะสมกับงานตรวจสอบโลโก้ นอกจากนี้ยังมีการวิเคราะห์ข้อดีและข้อเสียของแต่ละโมเดล เพื่อช่วยให้ผู้อ่านเข้าใจถึงความแม่นยำและประสิทธิภาพของแต่ละโมเดลในงานวิจัย

สุดท้ายเป็นการสรุปผลลัพธ์ของการวิจัยทั้งหมดในรูปแบบของตาราง เพื่อให้เกิดความเข้าใจและสามารถเปรียบเทียบความแม่นยำของโมเดลแต่ละตัวได้โดยง่าย

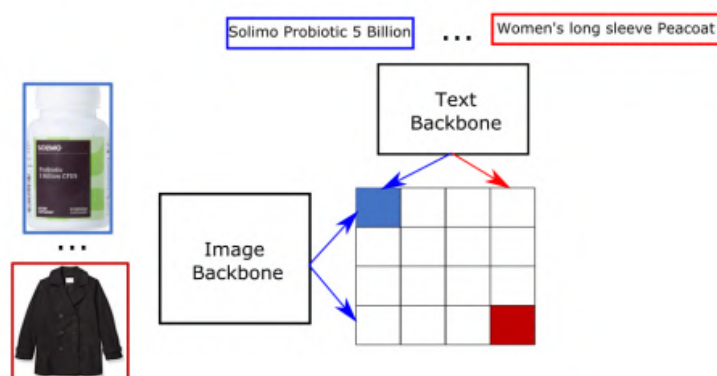
2.2.2 งานวิจัย “Comparative Analysis of Deep Learning Models for Brand Logo

Classification in Real-World Scenarios”

เอกสารวิจัยนี้เป็นงานวิจัยที่นำเสนอการทำ image classification ที่มีความแม่นยำสูงผ่านการใช้ transformer ที่ทำให้โมเดลสามารถเลือกเฉพาะส่วนที่เจาะจงของข้อมูล input ได้ ซึ่งทำให้สามารถเพิ่มความสามารถในการจับภาพและประมวลผลข้อมูลในส่วนที่เกี่ยวข้องพร้อมทั้งเพิ่มหลักการ dual attention ซึ่งถูกพัฒนามาเป็น Davit ซึ่งเป็นโมเดลที่นักวิจัยคิดค้นขึ้น พร้อมทั้งเปรียบเทียบประสิทธิภาพกับโมเดลที่เป็น Convolutional Neural Network ที่นิยมใช้กันปกติในงาน image classification

จากผลลัพธ์จากงานวิจัย ระบุว่า Davit ให้ผลลัพธ์ ในแง่ของความแม่นยำที่สูงมากแต่แลกมาด้วยความต้องการพลังในการคำนวณอย่างมหาศาล ขณะที่ Convolutional Neural Network ยังคงได้เปรียบในแง่ของการตอบสนองแบบ real time แต่อย่างไรก็ตามด้วยความสามารถในการประมวลผลและแนวโน้มที่จะมีการเพิ่มประสิทธิภาพให้แก่ vision transformer architectures สูงขึ้นอย่างต่อเนื่อง โมเดลที่มีฐานจาก transformer จึงมีแนวโน้มที่จะเป็นส่วนหลักในงาน deep learning ได้ในอนาคต

2.2.3 งานวิจัย “Image-Text Pre-Training for Logo Recognition”



รูป 2.11 กระบวนการ Pre-training ของภาพ

เอกสารนี้เป็นวิจัยที่กล่าวถึงการพยายามปรับปรุงการเพิ่มประสิทธิภาพของการทำ logo detection ที่ลักษณะของโลโก้ประกอบไปด้วยทั้งตราสัญลักษณ์และข้อความ โดยจากงานวิจัยนี้ได้นำเสนอ 2 วิธีหลัก ๆ โดยวิธีแรกได้มีการนำเทคนิค Image-Text Pre-training ในการสอนโมเดล ซึ่งเป็นการเทรนโมเดลด้วยข้อมูลคู่ภาพและข้อความคำอธิบายภาพ จะช่วยให้โมเดลเรียนรู้ความสัมพันธ์ระหว่างข้อความและภาพได้ดีขึ้น ทำให้มีความไวต่อข้อความและตัวอักษรในตราสัญลักษณ์มากขึ้น และวิธีที่ 2 คือการทำ ProxyNCAHN++ Loss มาใช้ร่วมกัน โดยเป็นการรวมตัวอย่างภาพที่เป็น Hard Negative ของตราสัญลักษณ์คลาสอื่น ๆ เข้ามาในการเทรน จะช่วยให้โมเดลสามารถจำแนกความแตกต่างระหว่างตราสัญลักษณ์ต่าง ๆ ได้ดียิ่งขึ้น โดยเฉพาะอย่างยิ่งกับตราสัญลักษณ์ที่มีลักษณะคล้ายคลึงกัน หรือมีบางส่วนของตัวอักษร/ข้อความคล้ายกัน

จากผลลัพธ์งานวิจัยระบุว่า เมื่อนำ Image-Text Pre-training และ ProxyNCAHN++ Loss มาใช้ร่วมกันโดยมีการทดสอบกับหลายชุดข้อมูลมาตรฐาน ผลปรากฏว่าวิธีการใหม่สามารถเพิ่มความแม่นยำในการจดจำ ได้ประมาณ 3-6% สำหรับตราสัญลักษณ์ที่มีข้อความประกอบสำคัญ แสดงถึงประโยชน์ที่จะนำไปใช้งานกับการพัฒนาโมเดลของเรา ซึ่งช่วยเพิ่มประสิทธิภาพในการรับมือกับตราสัญลักษณ์ที่หลากหลายและมีความซับซ้อนมากขึ้น

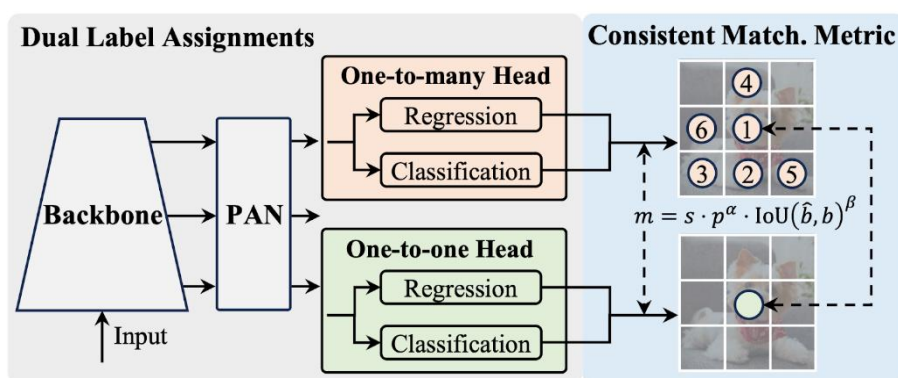
2.2.4 งานวิจัย “DETRs Beat YOLOs on Real-time Object Detection”

เอกสารงานวิจัยนี้นำเสนอถึง RT-DETR ที่มีประสิทธิภาพการทำ object detection ได้ดีกว่า YOLO ทั้งในด้านเร็วและความแม่นยำ และ RT-DETR ยังสามารถปรับความเร็วได้อย่างยืดหยุ่นผ่านการเปลี่ยนชิ้นในส่วน of decoder โดยที่ไม่ต้องทำการเทรนซ้ำ ซึ่งเหมาะสำหรับงานในลักษณะ real time

ทางนักวิจัยได้กล่าวไว้เกี่ยวกับการทำ Non-maximum suppression (NMS) ส่งผลเสียต่อความแม่นยำและความเร็วของการทำ object detection แบบ real time ทำให้นักวิจัยได้เสนอการแก้ไขปัญหามาจากการทำ hybrid encoder ที่สามารถช่วยลดความซ้ำซ้อนในการคำนวณ โดยแยกการคำนวณออกเป็น intra-scale interaction ที่ใช้ attention mechanism ในการประมวลผลและ cross-scale fusion ที่ใช้ CCFM module ซึ่งประกอบด้วยชั้น convolution เพื่อรวมลักษณะของรูป และเมื่อแบ่งส่วนการคำนวณเป็น 2 ส่วนจึงทำให้ลดภาระในการคำนวณของ encoder อย่างมาก

จากผลลัพท์งานวิจัยระบุไว้ว่า RT-DETR แบบ Extreme ให้ค่า AP 54.8 % และความเร็ว 74 FPS ซึ่งดีกว่า YOLO ทั้งในด้านความแม่นยำและความเร็ว ส่วน RT-DETR แบบ Large ให้ค่า AP 53.0% บน COCO val 2017 และความเร็ว 114 FPS บน GPU Tesla T4

2.2.5 งานวิจัย “YOLOv10: Real-Time End-to-End Object Detection”



รูป 2.12 Dual assignments แบบสอดคล้องกันสำหรับ NMS-free training

เอกสารงานวิจัยนี้เป็นเอกสารที่กล่าวถึงการพัฒนาของโมเดล YOLOv10 ซึ่งเป็นโมเดลสำหรับ Object Detection ที่ถูกพัฒนาอยู่ในตระกูล YOLO (You Only Look Once) โดยมุ่งหวังที่จะปรับปรุงทั้งประสิทธิภาพและความสามารถในการใช้งานของโมเดล YOLO

โดย YOLOv10 มีการปรับปรุงโมเดลโดยการพัฒนาโมเดลโดยปราศจาก NMS ซึ่งงานวิจัยได้นำเสนอวิธีการในการแทนใหม่ที่เรียกว่า Dual Assignment Strategy ซึ่งไม่จำเป็นต้องใช้ Non-Maximum Suppression (NMS) ใน Post processing วิธีนี้ช่วยให้เพิ่มความแม่นยำให้สูงขึ้นและลดเวลาในการทำนายลงอย่างมาก นอกจากนี้งานวิจัยนี้เสนอแนวทางการออกแบบที่ครอบคลุม (Holistic Model Design) ซึ่งส่งผลให้ YOLOv10 มีประสิทธิภาพและความแม่นยำมากขึ้นโดยที่โมเดลมีขนาดที่เล็กลง ซึ่งเป็นการลดขนาดของโมเดลที่ทันสมัย

จากการวัดประสิทธิภาพของโมเดล YOLOv10 แสดงให้เห็นว่ามีความแม่นยำและความเร็วเหนือกว่าโมเดลก่อนหน้านี้ ตัวอย่างเช่น YOLOv10-S ถูกกล่าวหาว่ามีความเร็วในการทำงาน

มากกว่า RT-DETR-R18 ถึง 1.8 เท่า ในขณะที่มีจำนวนพารามิเตอร์น้อยกว่า แสดงให้เห็นถึงประสิทธิภาพในการคำนวณที่ดีขึ้น โดยที่ลดการใช้ทรัพยากร

2.2.6 งานวิจัย “YOLOV5, YOLOV8 AND YOLOV10: THE GO-TO DETECTORS FOR REAL-TIME VISION”

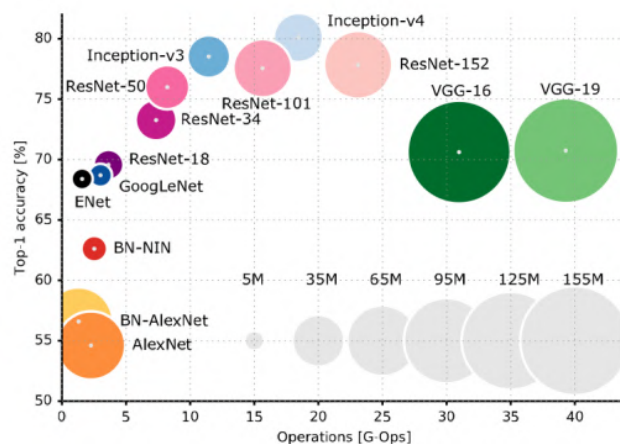
ตาราง 2.2 เปรียบเทียบ Architectural Features โดยละเอียด

Feature	YOLOv5	YOLOv8	YOLOv10
Backbone	CSPDarknet	CSPDarknet (enhanced)	CSPDarknet (further optimized)
Neck	PANet	PANet (improved)	PANet (with efficiency optimizations)
Head	Anchor-based	Anchor-free	Anchor-free with dual assignments
NMS	Required	Required	NMS-free
Activation Function	Leaky ReLU	SiLU (Swish)	Advanced activation (not specified)
Feature Pyramid	FPN	Modified FPN	Enhanced FPN with spatial-channel decoupling
Loss Function	CIoU loss	Task-specific losses	Consistent Dual Assignment loss
Data Augmentation	Mosaic, Cutout	Mosaic, Cutout, Mixup	Advanced augmentation techniques
Training Strategy	Single-stage	Single-stage	Two-stage with dual assignments

เอกสารงานวิจัยนี้เป็นการศึกษาเกี่ยวกับการพัฒนาของโมเดลตระกูล YOLO ในแต่ละเวอร์ชัน โดยเน้นการเปรียบเทียบสามรุ่นหลัก ได้แก่ YOLOv5, YOLOv8 และ YOLOv10 เพื่อแสดงให้เห็นถึงความแตกต่างและการพัฒนาในโครงสร้างสถาปัตยกรรมของแต่ละรุ่น

โดยแสดงให้เห็นถึงการปรับปรุงที่สำคัญ เช่น การใช้ dual-pathway approach ใน YOLOv10 ที่ช่วยลดเวลาในการประมวลผลและเพิ่มความแม่นยำ รวมถึงการกำจัด Non-Maximum Suppression (NMS) เพื่อเพิ่มประสิทธิภาพในการตรวจจับวัตถุ

2.2.7 งานวิจัย “Gesture Recognition for Robotic Control Using Deep Learning”



รูป 2.13 เปรียบเทียบขนาดและประสิทธิภาพของโมเดล CNN ที่เป็นที่นิยม

เอกสารงานวิจัยนี้เป็นการศึกษาเกี่ยวกับการใช้โมเดล Convolutional Neural Networks (CNNs) ในการจดจำท่าทางจากภาพถ่ายหรือวิดีโอ โดยมีการเปรียบเทียบโมเดล CNN หลายรูปแบบซึ่งมีคุณสมบัติและประสิทธิภาพที่แตกต่างกัน งานวิจัยนี้มีความเกี่ยวข้องกับงานของทางคณะผู้จัดทำซึ่งเน้นการทำ Image Classification โดยสามารถนำผลการศึกษาไปเป็นแนวทางและปรับใช้ในการเลือกโมเดลที่เหมาะสมที่สุดกับบริบทของงานได้ อาทิเช่น หากต้องการความแม่นยำสูงและมีทรัพยากรเพียงพอ เช่น การประมวลผลบนเซิร์ฟเวอร์หรือคลาวด์ สามารถเลือกใช้ ConvNeXt ซึ่งเหมาะสำหรับข้อมูลที่ซับซ้อนและต้องการผลลัพธ์ที่แม่นยำ หรือหากต้องการลดภาระด้านทรัพยากร หรือใช้งานในระบบเรียลไทม์ เช่น อุปกรณ์พกพา สามารถเลือกใช้ MobileNet ซึ่งมีขนาดเล็กและทำงานได้รวดเร็ว

2.3 เครื่องมือที่ใช้ในการพัฒนา

- 1) Pre-Trained Model ของ Detection Model ที่ใช้ในการเทรนกับชุดข้อมูล
- 2) Pre-Trained Model ของ Classification Model ที่ใช้ในการเทรนกับชุดข้อมูล
- 3) NVIDIA RTX A4000 Graphics Card

บทที่ 3

การออกแบบและการพัฒนา

3.1 การเตรียม Dataset และเครื่องมือที่ใช้

3.1.1 การเตรียม Dataset

Dataset เป็นสิ่งสำคัญที่สุดในงาน Machine Learning เนื่องจากมีบทบาทสำคัญในการเทรนโมเดลและการทดสอบความแม่นยำของโมเดล โดย Dataset ที่มีคุณภาพจะส่งผลให้โมเดลที่ทำการเทรนมีประสิทธิภาพในการทำนายสูง ซึ่งจากที่คณะผู้จัดทำได้ทำการสำรวจ พบว่ามี Dataset ที่มีข้อมูลโลโก้เป็นจำนวนมาก ทั้ง Dataset ที่เปิดเป็นสาธารณะและเป็นส่วนตัว ทางคณะผู้จัดทำจึงเลือกนำเสนอ Dataset ที่มีการเปิดเผยสู่สาธารณะ ได้แก่ FlickrLogos-47 และ Dataset ที่ทางคณะผู้จัดทำได้เก็บรวบรวมมา คือ 22-Class Thai Logos ซึ่งเป็นชุดข้อมูลที่มีทั้งหมด 22 คลาส โดยเป็นโลโก้ขององค์กรที่สามารถพบเห็นได้ในประเทศไทย และทำการรวบรวมจากสื่อออนไลน์ต่าง ๆ ดังนี้

ตาราง 3.1 เปรียบเทียบชุดข้อมูล FlickrLogos-47 และ 22-Class Thai Logos

Dataset	Logos	Images	Year
FlickrLogos-47	47	2,235	2011
22-Class Thai Logos	22	1,170	2024

จากตารางดังกล่าวระบุข้อมูลของ FlickrLogos-47 และ 22-Class Thai Logos ได้แก่ ชื่อ, จำนวนโลโก้, จำนวนรูป, และปีที่เผยแพร่ คณะผู้จัดทำได้มีการวางแผนที่จะนำโลโก้ Dataset ที่มีอยู่ในสาธารณะมาใช้ประกอบการเทรนโมเดล เพื่อให้โมเดลสามารถเรียนรู้การตรวจจับ โลโก้ได้ และเพื่อเป็นการประหยัดเวลาในการเตรียม Dataset

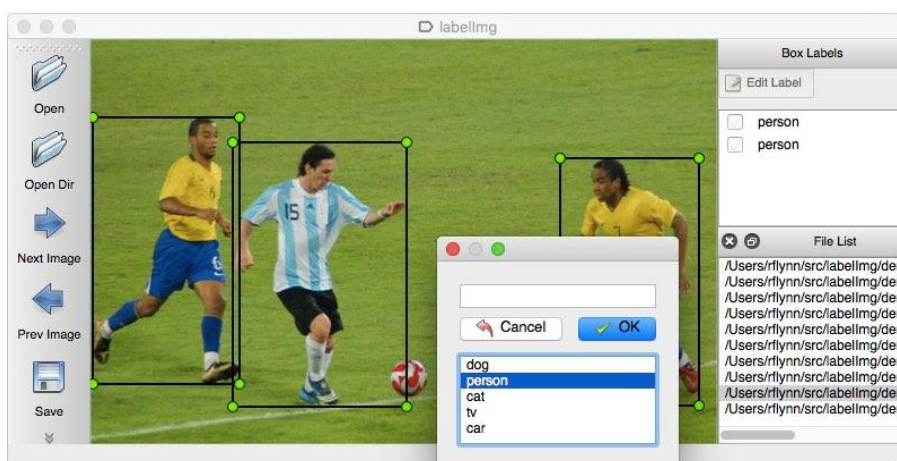
โดยจาก Dataset ที่กล่าวมาเราจะนำไปใช้ในการทดสอบทั้งในการทำ Logo Detection และ Logo Classification ซึ่งอาจจะมีรูปแบบการวัดผลที่แตกต่างกันไปตามความเหมาะสม อีกทั้งคณะผู้จัดทำต้องการให้โมเดลมีความสามารถในการจำแนกโลโก้บางส่วนที่มีอยู่ในประเทศไทยขององค์กรภาครัฐและเอกชน จึงมีความจำเป็นต้องรวบรวมข้อมูลรูปโลโก้จากอินเทอร์เน็ตเพิ่มเติม

ด้วยเหตุนี้ทางคณะผู้จัดทำจึงได้ทำการรวบรวม 22-Class Thai Logos จากทางอินเทอร์เน็ต โดยแบ่งเป็น 22 คลาส แต่ละคลาสประกอบด้วยรูปที่มีโลโก้คลาสนั้นประกอบอยู่ เป็นจำนวน 50-60 รูปต่อ 1 คลาส

3.1.2 การทำ Data annotation

ในการทำ Object Detection และ Image Classification ผ่านการเทรน CNN โมเดลนั้น เป็นการสอนโมเดลแบบ Supervised learning ซึ่งเป็นการสอนโมเดลโดยใช้ข้อมูลที่มีคำตอบ (Labeled data) ซึ่งในงาน Object Detection จะเรียกว่า Annotated Data โดยปกติจะระบุตำแหน่งของวัตถุในรูปภาพผ่าน Bounding Boxes ที่จะระบุตำแหน่งและขนาดของวัตถุที่ถูกระบุไว้ว่าเป็นวัตถุที่ต้องการใช้เป็นคำตอบ

จาก dataset ที่ปรากฏในตาราง 3.1 ส่วนใหญ่จะมีการทำ data annotation มาเรียบร้อยแล้วจึงสามารถนำมาใช้ในการสอนโมเดลได้ทันที แต่กับข้อมูลที่ทำการรวบรวมเองเพื่อใช้สำหรับการ fine tune โมเดลจำเป็นต้องมีการทำ data annotation โดยการทำ data annotation คณะผู้จัดทำเลือกใช้ LabelImg ซึ่งเป็นเครื่องมือที่นิยมใช้ในการทำ data annotation สำหรับงานที่เป็นรูปภาพ



รูป 3.1 การทำ Data Annotation ด้วยโปรแกรม LabelImg

โดย LabelImg สามารถบันทึกไฟล์ annotation ได้หลายรูปแบบเช่น PASCAL VOC, YOLO และ CreateML เป็นต้น และนอกจากที่ LabelImg สามารถระบุ Bounding Boxes ได้แล้ว ยังสามารถระบุคลาสของวัตถุที่อยู่ใน Bounding Boxes ได้เพื่อนำไปใช้ในโมเดลที่สามารถทำได้ทั้ง Detection และ Classification ได้

ทางคณะผู้จัดทำได้มีการทำ Data Annotation กับ 22-Class Thai Logos โดยกำหนดให้มีรูปแบบเป็น YOLO format และกำหนดเลข Class Index ดังนี้

ตาราง 3.2 แสดงเลข Class Index ของ 22-Class Thai Logos Dataset

0. DDC	1. GSB	2. MEA
3. GHB	4. PEA	5. SCB
6. EGAT	7. BBL	8. NBTC

9. TTB	10. CP	11. KTB
12. CP-ALL	13. UOB	14. 7-Eleven
15. BAY	16. SET	17. KBANK
18. SEC	19. Dataxet	20. MR.DIY
21. TNN		

3.2 การจัดเตรียมข้อมูล

3.2.1 การทำ Image Augmentation

ทำ Image Augmentation ช่วยให้โมเดลสามารถเรียนรู้รูปแบบที่หลากหลายของวัตถุได้ดีขึ้นและทำให้มีความแม่นยำในการทำนายวัตถุแม้จะอยู่ในการแปลงของสภาพแวดล้อมมากขึ้น โดยเป็นเทคนิคสำคัญที่ช่วยเพิ่มปริมาณและความหลากหลายของข้อมูล โดยการสร้างภาพใหม่จากภาพเดิม

3.2.2 การทำ Image Resizing, Normalization, และ Data Partitioning

Image Resizing เป็นการเปลี่ยนขนาดของข้อมูลให้สอดคล้องกับ input ของโมเดล หลังจากนั้นทำการ Normalization ซึ่งทำให้ขนาด pixel ของรูปภาพอยู่ในช่วง 0-1 ซึ่งสามารถทำให้โมเดลนั้น convergence ได้ง่ายมากยิ่งขึ้น และทำการแบ่งข้อมูลให้พร้อมสำหรับการนำเข้าโมเดล โดยการแบ่งข้อมูลออกเป็น 3 ส่วน ได้แก่ Training Set, Validation Set, และ Testing Set

3.3 การพัฒนา Logo Detection

3.3.1 การเปรียบเทียบและเลือกโมเดลที่จะนำมาพัฒนา

3.3.1.1 YOLOv8

โมเดล YOLOv8 เป็นการพัฒนาต่อยอดมาจากเวอร์ชันก่อนหน้าอย่าง YOLOv7 เพื่อให้มีประสิทธิภาพการทำงานที่สูงขึ้นในแง่ความแม่นยำ ความเร็ว และการใช้ทรัพยากร

โดยโมเดล YOLOv8 มีการเปลี่ยน Backbone มาใช้เป็น ConvNeXt ซึ่งมีการนำแนวคิดของ transformer มาใช้ทำให้โมเดลมีประสิทธิภาพยิ่งขึ้น ในขณะที่ให้ยังคงขนาดของโมเดลให้เล็กและใช้ทรัพยากรน้อย ทำให้ประสิทธิภาพในการทำ real-time ก็ดีขึ้นด้วย และยังมีการเปลี่ยน Neck เป็น PAFPN++ (Parallel Alternating FPN++) ซึ่งจะทำให้โมเดลมีการจัดการภาพที่มีรูปแบบหลากหลายได้ดีขึ้น ซึ่งส่งผลให้ประสิทธิภาพในการตรวจจับวัตถุดีขึ้นกว่าเวอร์ชันก่อนหน้า และในส่วนของ Head ยังมีการเปลี่ยนมาใช้ anchor-free จากเดิมที่ต้องกำหนดขนาดและสัดส่วนของ anchors ให้เหมาะสมก่อน หากการตั้งค่าไม่ถูกต้องจะส่งผลต่อประสิทธิภาพของ

โมเดล ไปใช้การทำนายค่า Parameter ต่าง ๆ ของวัตถุ ทั้งตำแหน่ง ขนาด และคลาส ทำให้โมเดลมีความยืดหยุ่นกว่าเวอร์ชันก่อนหน้านี้

การเทรนโมเดล YOLOv8 เป็นกระบวนการที่สะดวกและรวดเร็ว เนื่องจากโมเดล YOLOv8 นั้น มีการ Pre-training อย่างเต็มรูปแบบมาก่อนแล้ว จึงสามารถนำโมเดลนั้นมาทำการ Fine-tuning ต่อได้ด้วย Dataset ที่ทำการเตรียมไว้ได้โดยตรง โดยไม่ต้องเริ่มต้นใหม่ทั้งหมด เพื่อให้โมเดลมีความแม่นยำและเหมาะสมกับงานหรือปัญหาที่เฉพาะเจาะจงได้อย่างมีประสิทธิภาพ

3.3.1.2 YOLOv10

โมเดล YOLOv10 มีการเปลี่ยน Backbone มาใช้เป็น CSPNet (Cross Stage Partial Network) ซึ่งจะช่วยลดความซับซ้อนของโมเดลลง ทำให้ลดเวลาในการเทรนโมเดล และมีการนำ PAN (Path Aggregation Network) มาใช้ซึ่งจะช่วยในการทำให้โมเดลสามารถตรวจจับได้ทั้งในหลากหลายขนาด อีกทั้งยังนำการใช้ NMS (Non-Maximum Suppression) ออกเพื่อทำการลดเวลาในการประมวลผล และมีการคำนวณ Convolutions ที่มี Kernel ขนาดใหญ่ เพื่อทำการเพิ่มประสิทธิภาพของโมเดลโดยที่ยังรักษาการใช้ทรัพยากรไว้

การเทรนโมเดล YOLOv10 เป็นกระบวนการที่สะดวกและรวดเร็วคล้ายกับโมเดล YOLOv8 ที่มีการ Pre-training อย่างเต็มรูปแบบมาก่อนแล้ว แต่มีขนาดโมเดลที่เล็กลง โดยยังคงประสิทธิภาพเอาไว้ได้

3.3.1.3 RT-DETR

โมเดล Real-Time Detection Transformer (RT-DETR) พัฒนามาจาก DETR ซึ่งมีการปรับปรุงความเร็วในการตรวจจับวัตถุในรูปให้เร็วกว่าโมเดลเดิม

โดยโมเดล RT-DETR มีโครงสร้างแบบ Feature Pyramid Network (FPN) ซึ่งจะช่วยให้สามารถรวบรวม features ได้จากหลายความละเอียด จะทำให้ตรวจจับวัตถุขนาดเล็กและใหญ่ได้อย่างมีประสิทธิภาพ และมีการเปลี่ยน Backbone มาใช้ ResNet-C ซึ่งเป็น ResNet ที่มีการลด channels ลงทำให้โมเดลมีความเร็วที่เพิ่มขึ้น อีกทั้งยังมีการทำ Parallel Decoding เป็นการประมวลผลแบบขนาน ทำให้ลดเวลาได้ถึง 4 เท่า

การเทรนโมเดล RT-DETR เป็นกระบวนการที่สะดวกและรวดเร็วเช่นเดียวกับโมเดล YOLOv8 เนื่องจากมีผู้พัฒนาคนเดียวกันนั่นก็คือ Ultralytics RT-DETR นั้นมีการ Pre-training อย่างเต็มรูปแบบมาก่อนแล้ว จึงสามารถนำโมเดลนั้นมาทำการ Fine-tuning ต่อได้ด้วย Dataset ที่ทำการเตรียมไว้ได้โดยตรง ไม่ต้องเริ่มต้นใหม่ทั้งหมด เพื่อให้โมเดลมีความแม่นยำและเหมาะสมกับงานหรือปัญหาที่เฉพาะเจาะจงได้อย่างมีประสิทธิภาพ

3.3.2 การเทรนโมเดล

ทำการเทรนโมเดลด้วยข้อมูล Training Set กับ Validation Set ที่ได้เตรียมไว้

3.4 การพัฒนา Logo Classification

3.4.1 การเปรียบเทียบและเลือกโมเดลที่จะนำมาพัฒนา

3.4.1.1 MobileNetV3

MobileNetV3 เป็น โมเดล Convolutional Neural Network (CNN) ที่ได้รับการออกแบบให้มีขนาดเล็กและใช้ทรัพยากรคอมพิวเตอร์น้อย เพื่อให้สามารถทำงานได้อย่างมีประสิทธิภาพบนอุปกรณ์เคลื่อนที่ นอกจากนี้ ยังถูกออกแบบให้ใช้พลังงานต่ำและมี latency ต่ำ ทำให้เหมาะกับการใช้งานในสภาพแวดล้อมที่มีทรัพยากรจำกัด MobileNetV3 สามารถประยุกต์ใช้ได้ในงานที่หลากหลาย เช่น Object Detection, Image Classification และ Semantic Segmentation

โมเดล MobileNetV3 ได้รับการพัฒนาต่อยอดจาก MobileNetV2 โดยมีการนำ Hard-Swish ซึ่ง Activation Function มาใช้แทน ReLU เพื่อช่วยให้โมเดลสามารถเรียนรู้ได้เร็วขึ้นและลดปัญหา Overfitting ได้ดีขึ้น โดยเฉพาะในกรณีของชุดข้อมูลขนาดใหญ่ นอกจากนี้ MobileNetV3 ยังมีการนำ Squeeze-and-Excitation Block (SE Block) มาใช้เพื่อเพิ่ม weight ให้กับฟีเจอร์ที่สำคัญและลด weight ของฟีเจอร์ที่ไม่สำคัญ ซึ่งช่วยเพิ่มประสิทธิภาพการเรียนรู้ของโมเดล

อีกหนึ่งจุดเด่นของ MobileNetV3 คือการนำ Neural Architecture Search (NAS) มาใช้ในการออกแบบโครงสร้างของโมเดล ซึ่งช่วยปรับแต่ง nodes ภายในโมเดลโดยอัตโนมัติ ทำให้ลดเวลาในการพัฒนา เนื่องจากนักพัฒนาไม่จำเป็นต้องปรับแต่งโครงสร้างของโมเดลเองมากนัก ส่งผลให้ MobileNetV3 มีประสิทธิภาพสูงขึ้นและสามารถปรับให้เหมาะสมกับงานเฉพาะทางได้ดียิ่งขึ้น

3.4.1.2 InceptionV3

InceptionV3 เป็น โมเดล Convolutional Neural Network (CNN) ที่ได้รับการพัฒนาต่อยอดจาก InceptionV2 โดยมีการปรับปรุงโครงสร้างให้มีประสิทธิภาพมากขึ้นทั้งในด้านความแม่นยำและความสามารถในการเรียนรู้ฟีเจอร์ที่ซับซ้อน โดยโมเดลนี้ได้รับการออกแบบให้สามารถทำงานได้ดีในด้าน Image Classification บนชุดข้อมูลขนาดใหญ่

หนึ่งในการเปลี่ยนแปลงสำคัญของ InceptionV3 คือการนำแนวคิด Factorized Convolutions มาใช้เพื่อช่วยลดความซับซ้อนของการคำนวณและจำนวนพารามิเตอร์ของโมเดล เดิมทีการใช้ Convolution ขนาดใหญ่ เช่น 5×5 นั้นมีการใช้ทรัพยากรที่สูง ทั้งในด้านการคำนวณและการใช้หน่วยความจำ ดังนั้น InceptionV3 ได้ปรับให้ใช้การแยก Convolution ขนาดใหญ่ออกเป็นชุดของ Convolution ที่มีขนาดเล็กลง เช่น การแทนที่ 5×5 Convolution ด้วย 3×3 Convolution สองชุด ซึ่งช่วยลดจำนวนพารามิเตอร์และเพิ่มความเร็วในการประมวลผลโดยไม่สูญเสียประสิทธิภาพ

อีกทั้งยังมีการใช้ Label Smoothing ซึ่งเป็นเทคนิคที่ช่วยกระจายค่าความน่าจะเป็นของคลาสเป้าหมายออกไปเล็กน้อย แทนที่จะกำหนดค่า 100% ให้กับคลาสเดียว วิธีนี้ช่วย

ลดปัญหา Overfitting ทำให้โมเดลสามารถเรียนรู้ได้อย่างมีประสิทธิภาพมากขึ้นเมื่อใช้งานกับชุดข้อมูลขนาดใหญ่

InceptionV3 ยังคงใช้แนวคิด Auxiliary Classifiers ที่เคยมีใน InceptionV2 โดยช่วยให้โมเดลสามารถเรียนรู้ได้เร็วขึ้นและลดปัญหา Gradient Vanishing นอกจากนี้ยังมีการใช้ Batch Normalization ในทุกชั้นของโมเดลเพื่อช่วยให้การฝึกมีความเสถียรมากขึ้น ลดการพึ่งพาการตั้งค่า Learning Rate ที่แม่นยำ และช่วยให้โมเดลสามารถเรียนรู้ได้เร็วขึ้น

3.4.1.3 Inception-ResNetV2

Inception-ResNetV2 เป็นโมเดล Convolutional Neural Network (CNN) ที่ผสมโครงสร้างของ InceptionV3 เข้ากับ Residual Connections จาก ResNet เพื่อเพิ่มประสิทธิภาพในการเรียนรู้และลดปัญหา Vanishing Gradient การใช้ Residual Connections ช่วยให้โมเดลสามารถเรียนรู้ฟีเจอร์ที่ซับซ้อนได้อย่างมีประสิทธิภาพ

โมเดลนี้ยังคงแนวคิด Inception Module ที่แบ่ง Convolution ออกเป็นหลายขนาด และใช้ Factorized Convolutions เพื่อลดความซับซ้อน นอกจากนี้ยังมีการใช้ Normalization Scaling เพื่อปรับสมดุลของค่าใน Residual Connections และ Batch Normalization เพื่อช่วยให้การฝึกโมเดลมีประสิทธิภาพมากขึ้น

Inception-ResNetV2 มีความแม่นยำสูงกว่ารุ่นก่อนหน้า แต่มีความซับซ้อนและต้องใช้ทรัพยากรในการประมวลผลมากขึ้น ทำให้เหมาะสำหรับงานที่ต้องการความแม่นยำสูง เช่น การวิเคราะห์ภาพทางการแพทย์ และระบบจดจำวัตถุที่ซับซ้อน

3.4.1.4 EfficientNetV2

EfficientNetV2 เป็นโมเดล CNN ที่พัฒนาต่อยอดจาก EfficientNet โดยเน้นการปรับสเกลของโมเดล (depth, width, resolution) อย่างสมดุลผ่าน Compound Scaling เพื่อเพิ่มประสิทธิภาพในการเรียนรู้และลดการใช้ทรัพยากร นอกจากนี้ยังใช้ Swish Activation และ Squeeze-and-Excitation Blocks เพื่อช่วยให้โมเดลดึงฟีเจอร์ที่สำคัญได้ดีขึ้น

โมเดลนี้ได้รับการออกแบบด้วย Neural Architecture Search (NAS) เพื่อให้โครงสร้างมีประสิทธิภาพสูงสุด EfficientNetV2 เหมาะสำหรับงานที่ต้องการทั้งความแม่นยำสูงและการใช้ทรัพยากรต่ำ เช่น Image Classification และ Object Detection

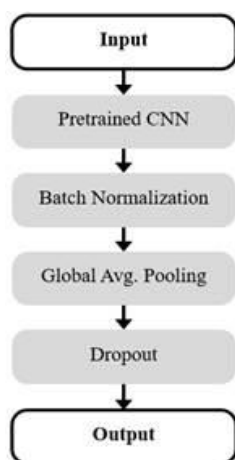
3.4.1.5 ConvNeXt (Convolutional Neural Networks with Exceptional Transformers)

ConvNeXt เป็นโมเดลที่ใช้สถาปัตยกรรมแบบผสมทั้ง CNN และ Transformer ที่ถูกเรียงสลับกันซึ่งทำให้มีข้อดีของทั้ง 2 วิธี โดย CNN จะมีข้อดีในด้านการตรวจจับข้อมูลในภาพเชิงพื้นที่ ในขณะที่ Transformer จะเรียนรู้ความสัมพันธ์ระหว่างข้อมูลเหล่านั้น และ

ConvNeXt ยังมีการใช้ Depthwise Convolution ซึ่งเป็นเทคนิคเดียวกับ MobileNetV2 ซึ่งจะทำให้ประหยัดทรัพยากรและมีประสิทธิภาพสูงกว่า CNN ทั่วไป

3.4.2 Transfer learning

ในการเทรน Classification Model ที่เป็นโมเดล Pre-Trained จะใช้แนวทางการปรับแต่งโมเดลโดยการ freeze base layers ของแต่ละโมเดล Pre-Trained และทำการเพิ่ม Convolutional Neural Network (CNN) เพื่อให้เหมาะสมกับการทำ Logo Classification โดยชั้นที่เพิ่มเข้าไปในแต่ละโมเดล Pre-Trained จะมีโครงสร้างที่คล้ายกัน ซึ่งประกอบด้วยองค์ประกอบสำคัญ ได้แก่ Batch normalization ซึ่งจะช่วยให้การกระจาย input ในแต่ละชั้นลดลง ทำให้ลดเวลาในการเทรนโมเดล และทำให้โมเดลมีการเรียนรู้ฟีเจอร์ของแต่ละคลาสได้ดีมากยิ่งขึ้น อีกทั้งยังมีการเพิ่ม Global Average Pooling (GAP) ซึ่งจะช่วยในการเรียนรู้ฟีเจอร์ของโลโก้ได้ดีขึ้น และช่วยลดจำนวนพารามิเตอร์ทำให้เกิด Overfitting ได้ยากขึ้น อีกทั้งยังมีการนำ Dropout ซึ่งจะทำให้การสุ่ม drop บาง neurons ทำให้โมเดลไม่พึ่งพาบาง neurons มากเกินไป อันเป็นผลให้โมเดลเกิดการเรียนรู้มากกว่าการจดจำ โดยโมเดลจะเก่งกับภาพที่ไม่เคยเห็นและลดการเกิด Overfitting โดยจะมี layer ทั้งหมด ดังรูป



รูป 3.2 โครงสร้างของ Classification Model ที่ทำการเพิ่ม CNN Layer

3.4.3 Hyperparameter tuning

ในการเทรน Classification Model ซึ่งเป็นโมเดล Convolutional Neural Network (CNN) ได้ทำ Hyperparameters Tuning เพื่อเพิ่มประสิทธิภาพของโมเดล โดยพิจารณาค่าที่มีผลกระทบต่อการเรียนรู้ของโมเดล ได้แก่ Learning Rate และ Dropout Rate

เนื่องจากข้อจำกัดด้านเวลาและทรัพยากรที่มีอยู่ จึงได้เลือกปรับแต่งเฉพาะสองค่าดังกล่าว โดยใช้วิธี Grid Search เพื่อค้นหาชุดค่าที่เหมาะสมที่สุดสำหรับแต่ละโมเดล ซึ่งช่วงค่าที่ใช้ในการทดลองมี ดังนี้

1) Learning Rate: 0.0001, 0.0003, 0.001, 0.003, 0.01

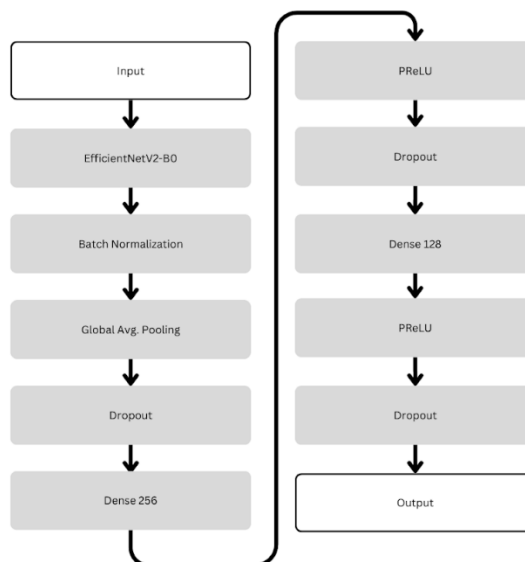
2) Dropout Rate: 0.1, 0.2, 0.3, 0.4

โดยจะทำ Hyperparameters Tuning กับ Classification Model ซึ่งเป็น Pre-Trained ที่ดีที่สุด 3 โมเดล จากทั้งหมด 5 โมเดลในแต่ละ Dataset ซึ่งโมเดลทั้งหมดได้รับการปรับแต่งด้วยชุดค่าข้างต้นเพื่อให้เกิดความสอดคล้องในการเปรียบเทียบผลลัพธ์

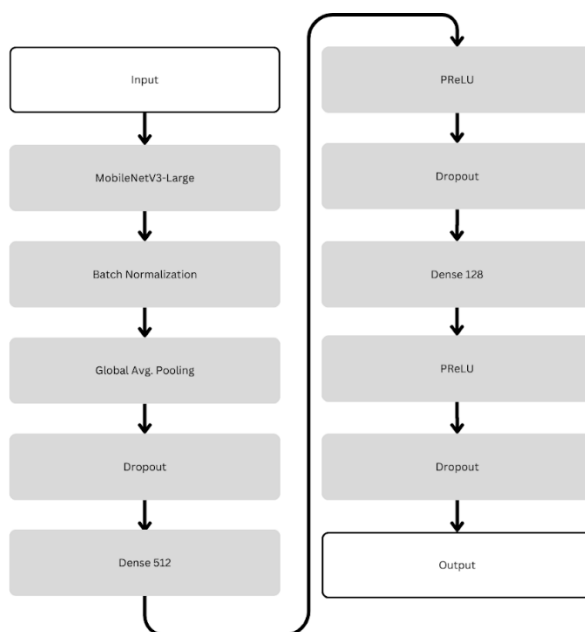
3.4.4 การเทรนโมเดล

ทำการเทรนโมเดลด้วยข้อมูล Training Set และ Validation Set ที่ได้เตรียมไว้ โดยหลังจากได้โมเดล Pre-Trained ที่ดีที่สุด 3 โมเดล จาก 5 โมเดลในแต่ละ dataset จากนั้นขั้นตอนการทำ Hyperparameters Tuning จะนำโมเดลมาทำการปรับแต่ง layer เพิ่มเติม เพื่อให้โมเดลมีประสิทธิภาพที่ดีที่สุด โดยจะได้ layer ของแต่ละโมเดล ในแต่ละ dataset ดังนี้

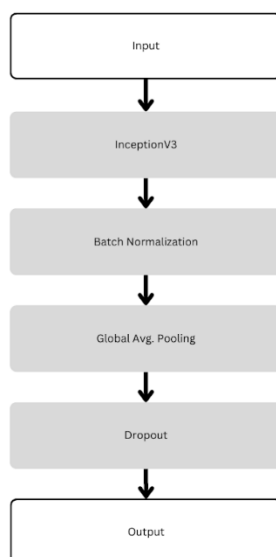
3.4.4.1 22-Class Thai Logos



รูป 3.3 โครงสร้างของโมเดล EfficientNetV2 ที่ทำการปรับแต่ง Layer

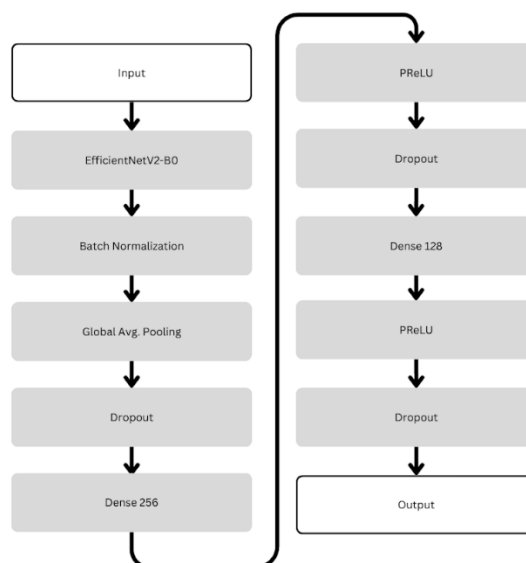


รูป 3.4 โครงสร้างของโมเดล MobileNetV3 ที่ทำการปรับแต่ง Layer

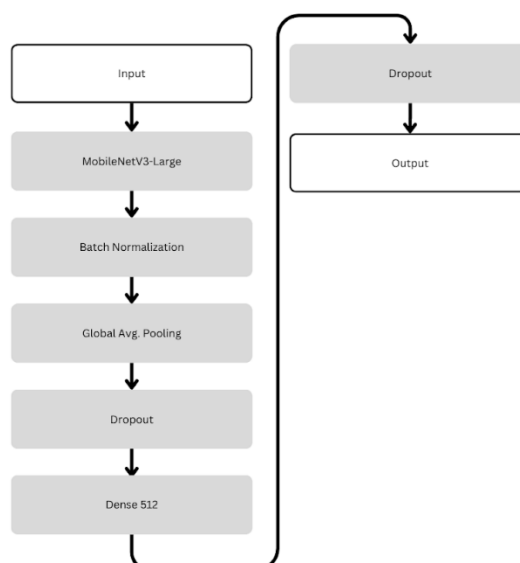


รูป 3.5 โครงสร้างของโมเดล InceptionV3 ที่ทำการปรับแต่ง Layer

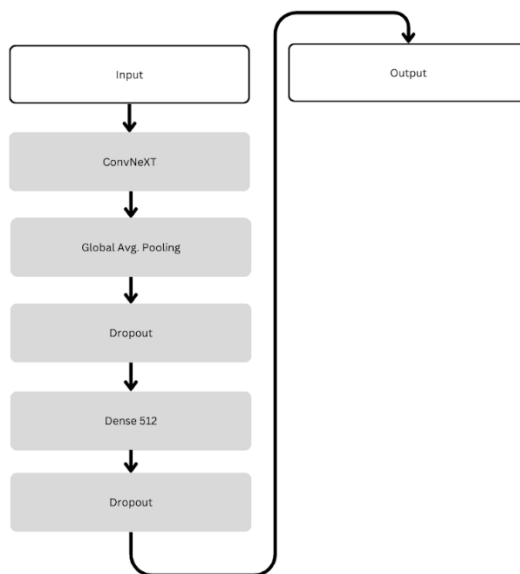
3.4.4.2 FlickrLogos-47 Dataset



รูป 3.6 โครงสร้างของโมเดล EfficientNetV2 ที่ทำการปรับแต่ง Layer



รูป 3.7 โครงสร้างของโมเดล MobileNetV3 ที่ทำการปรับแต่ง Layer

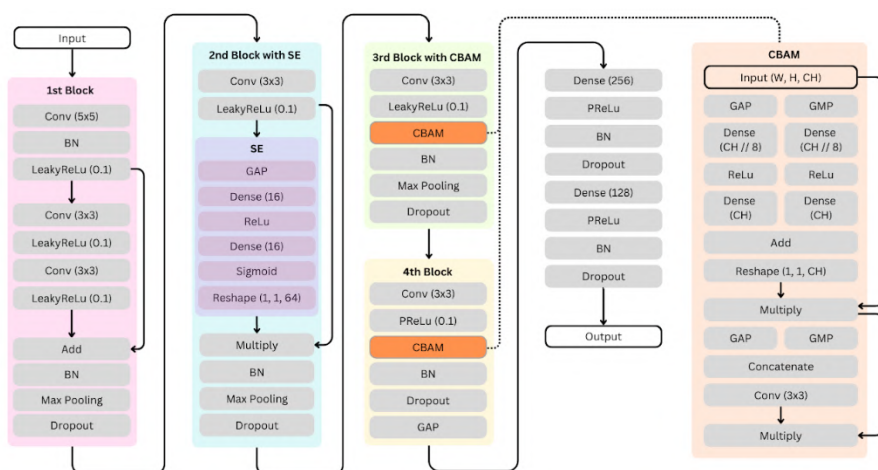


รูป 3.8 โครงสร้างของโมเดล ConvNeXT ที่ทำการปรับแต่ง Layer

3.4.5 การออกแบบและพัฒนา Custom CNN

3.4.5.1 การออกแบบ Custom CNN

ทำการออกแบบ Custom CNN ให้เหมาะสมกับการทำ Logo Classification โดยจะนำส่วนที่น่าสนใจในแต่ละโมเดล Pre-trained มาใช้ร่วมกัน และดึงประสิทธิภาพของ Custom CNN ให้สูงที่สุดโดยให้มีปริมาณพารามิเตอร์ของโมเดลน้อยที่สุด โดยจะได้โครงสร้างโมเดล ดังรูป



รูป 3.9 โครงสร้างของโมเดล Custom CNN

จากรูป 3.9 มีการกำหนด Input size ของรูปให้มีขนาด 128×128 pixels เนื่องจากเป็นขนาดที่ใกล้เคียงกับ Objects ที่ถูกตรวจจับจากโมเดล Logo Detection และเป็นขนาดที่เหมาะสมกับการใช้งาน convolutional filters ได้อย่างมีประสิทธิภาพ โดยโมเดลจะมีโครงสร้างของ Custom CNN ซึ่งมีทั้งหมด 4 Blocks ดังนี้

- 1) ใน Block ที่ 1 มีการนำ Convolution ขนาด 5×5 ซึ่งเป็นขนาดใหญ่เพื่อใช้สำหรับฟีเจอร์ต่าง ๆ ของโลโก้ เช่น เส้นขอบ, เส้นโค้ง, เส้นตรง, และรูปร่างพื้นฐานในโลโก้ และนำ Convolution ขนาด 3×3 ซ้ำหลายชั้น ซึ่งมีประสิทธิภาพที่มากกว่าการใช้ Convolution ขนาดใหญ่เพียงชั้นเดียว ทำให้โมเดลสามารถเรียนรู้ฟีเจอร์ที่ซับซ้อนขึ้นได้ และเป็นการลดจำนวนพารามิเตอร์เมื่อเทียบกับการใช้ filter ขนาดใหญ่ อีกทั้งยังมี Residual Connection (Add Layer) เพื่อลดปัญหา Vanishing Gradient และลดการสูญเสียค่า gradient ของ weight ใน layer แรก ๆ ของโมเดล
- 2) ใน Block ที่ 2 มีส่วนที่สำคัญคือ Squeeze-and-Excitation (SE) Module โดยจะเริ่มจากการทำ Global Average Pooling (GAP) กับทุก Channel ของ Feature Maps เพื่อเปลี่ยนให้เป็นเวกเตอร์ $1 \times 1 \times C$ จากนั้นใช้ Fully Connected Layers เพื่อเรียนรู้ความสำคัญของแต่ละ Channel และใช้ Sigmoid เพื่อปรับ weight แต่ละ Channel แล้วจึงใช้ Multiply Operation คูณ weight ของแต่ละ Channel กลับไปที่ Feature Map ทำให้ Channel ที่สำคัญได้รับความสนใจมากขึ้น ส่วน Channel ที่ไม่สำคัญจะถูกลดความสำคัญลง จากกระบวนการดังกล่าวช่วยให้โมเดลมีประสิทธิภาพดียิ่งขึ้นโดยไม่เพิ่มพารามิเตอร์มากเกินไป และโมเดลจะโฟกัสไปที่ฟีเจอร์ที่สำคัญของภาพ
- 3) ใน Block ที่ 3 มีส่วนที่สำคัญคือ Convolutional Block Attention Module (CBAM) ซึ่งจะช่วยให้โมเดลโฟกัสไปที่โลโก้ได้ดียิ่งขึ้นโดยการใช้ Attention Mechanism 2 แบบ ได้แก่ Channel Attention เป็นการเรียนรู้ว่าฟีเจอร์ใดสำคัญ และ Spatial Attention เป็นการเรียนรู้ว่าบริเวณใดของภาพสำคัญ มีการทำงานเป็นลำดับโดยการนำ Feature Map จาก CNN มาเข้า Channel Attention แล้วนำผลลัพธ์จาก Channel Attention ไปเข้า Spatial Attention และนำ Feature Map ที่ปรับปรุงแล้วไปใช้ต่อใน CNN โดยใน block นี้จะทำการโฟกัสฟีเจอร์หลักของโลโก้ เช่น เส้นขอบ และรูปร่าง

- 4) ใน Block ที่ 4 มีการนำ Convolutional Block Attention Module (CBAM) มาใช้อีกครั้งเพื่อทำการโฟกัสไปที่รายละเอียดของโลโก้ เช่น ลวดลายภายในโลโก้ และรายละเอียดของตัวอักษร ซึ่งช่วยลดการสูญเสียฟีเจอร์ที่สำคัญ เนื่องจากเมื่อมีการผ่านชั้น Convolution, Pooling, และ Dropout อาจทำให้ฟีเจอร์บางอย่างสูญหายไป และยังช่วยให้โมเดลสามารถแยกรายละเอียดของโลโก้ที่มีความคล้ายกันได้ดียิ่งขึ้น

3.4.5.2 การเทรนโมเดล Custom CNN

ทำการเทรน โมเดลด้วยข้อมูล Training Set และ Validation Set ชุดเดียวกัน กับโมเดล Pre-Trained โดยมีทั้งหมด 2 Datasets ได้แก่ FlickrLogos-47 และ 22-Class Thai Logos หลังจากนั้นจึงทำการทดสอบประสิทธิภาพของโมเดลด้วยข้อมูล Testing Set

3.5 การพัฒนาเว็บแอปพลิเคชัน

3.5.1 ภาพรวมของระบบ

เว็บแอปพลิเคชันนี้ต้องการจำลองการเรียกใช้งาน โมเดลด้วย UI สำหรับการใช้งาน ภายในองค์กรเท่านั้น เพื่อให้ง่ายและสะดวกต่อการใช้งาน โดยการใช้งานสามารถแบ่งออกเป็น 2 ส่วนหลัก ได้แก่

3.5.1.1 Logo Management System

การเตรียมข้อมูลให้พร้อมสำหรับการเทรนหรือทำนายผลโมเดล

3.5.1.2 Logo Prediction System

การนำข้อมูลที่เตรียมไว้หรือดึงข้อมูลเกี่ยวกับแบรนด์ที่ต้องการจาก โซเชียลมีเดียและทำนายผลด้วยโมเดล ไม่ว่าจะเป็นการเปรียบเทียบประสิทธิภาพระหว่างโมเดล หรือการทดสอบโมเดลในลักษณะต่าง ๆ

โดยเป็นการทำงานที่แยกส่วนกัน กล่าวคือ หากมี Dataset ที่ได้เตรียมไว้เรียบร้อยแล้วก็สามารถเลือกใช้งานในส่วนของ Logo Prediction ได้เลย

3.5.2 ความสามารถในการใช้งานระบบ

ตาราง 3.3 รายการความสามารถของผู้ใช้

ID	Requirement	Description
R01	ระบบรับข้อมูลภาพโลโก้	ผู้ใช้งานสามารถอัปโหลดภาพโลโก้ได้จากไฟล์รูปภาพในรูปแบบ JPG และ PNG
R02	ระบบการจัดการคลาส	ผู้ใช้งานสามารถเพิ่ม, แก้ไข, และลบคลาสจากระบบได้
R03	การสร้างและบริหาร Dataset	ผู้ใช้งานสามารถสร้าง Dataset ของโลโก้ด้วยการทำ Annotation และ/หรือ Augmentation เพื่อใช้สำหรับการเทรนหรือทดสอบโมเดลได้
R04	การคัดเลือกโมเดลสำหรับการทำนาย	ผู้ใช้งานสามารถเลือกโมเดลสำหรับการตรวจจับโลโก้และการจำแนกโลโก้ได้
R05	ระบบดึงข้อมูลภาพจากโซเชียลมีเดีย	ผู้ใช้งานสามารถดึงข้อมูลภาพโลโก้จาก X.com ตามคีย์เวิร์ดโลโก้ที่ต้องการและเตรียมข้อมูลสำหรับการทำนายผล
R06	การทำนายผลโลโก้จากภาพ	ผู้ใช้งานสามารถใช้โมเดลในการวิเคราะห์และทำนายผลโลโก้จากภาพที่อัปโหลดหรือนำเข้าจากโซเชียลมีเดีย
R07	การแสดงผลโลโก้ที่ตรวจพบ	ระบบต้องสามารถแสดงกรอบรอบโลโก้พร้อมชื่อแบรนด์ที่ตรวจพบในภาพ
R08	ระบบแจ้งเตือนความผิดพลาด	หากเกิดข้อผิดพลาดในการอัปโหลดภาพ, ดึงข้อมูลจากโซเชียลมีเดีย หรือการทำงานของโมเดล ระบบจะแสดงข้อความแจ้งเตือน

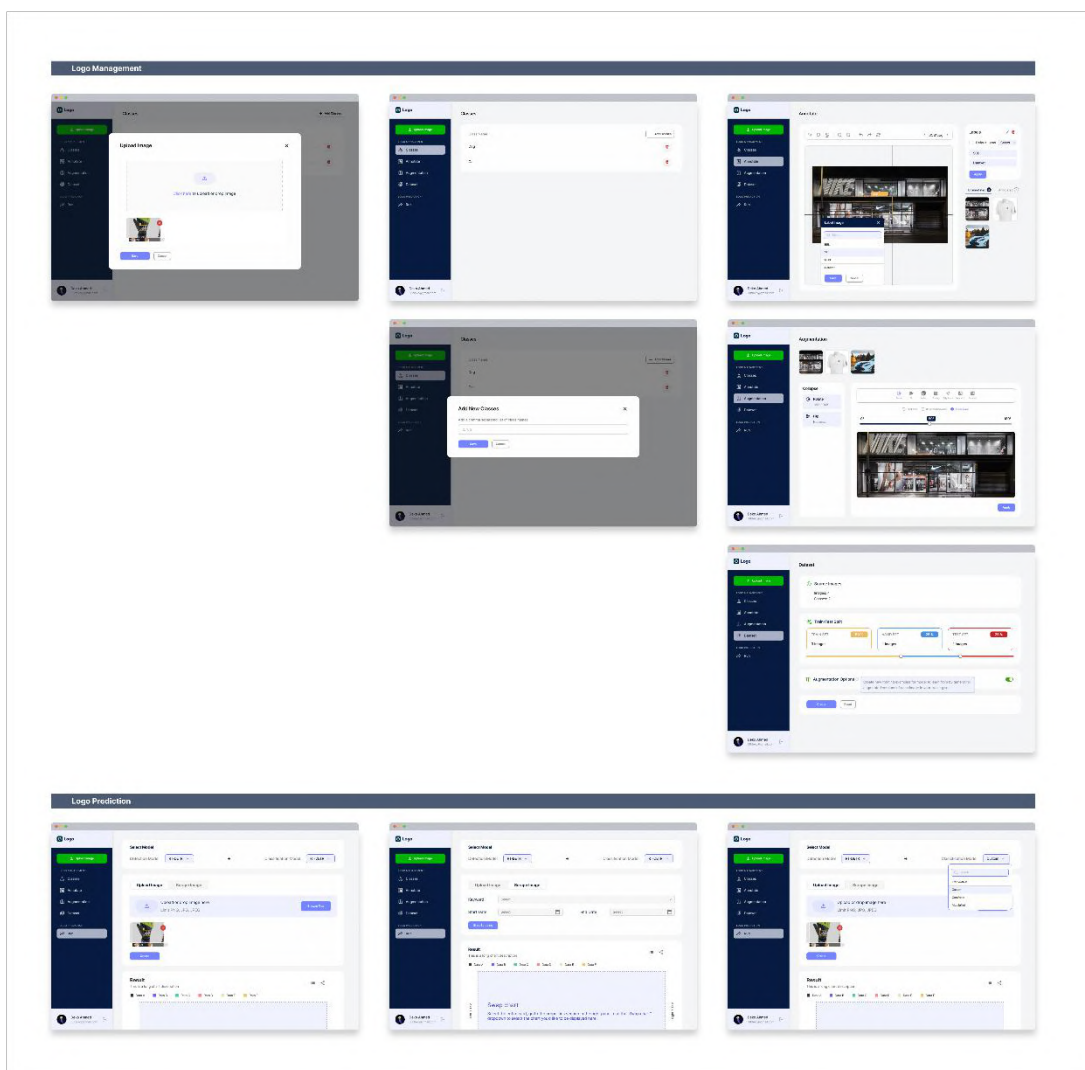
3.5.3 การออกแบบ Use Case Diagram



รูป 3.10 Use Case Diagram ของระบบ

3.5.4 การออกแบบส่วนติดต่อผู้ใช้ (User Interface)

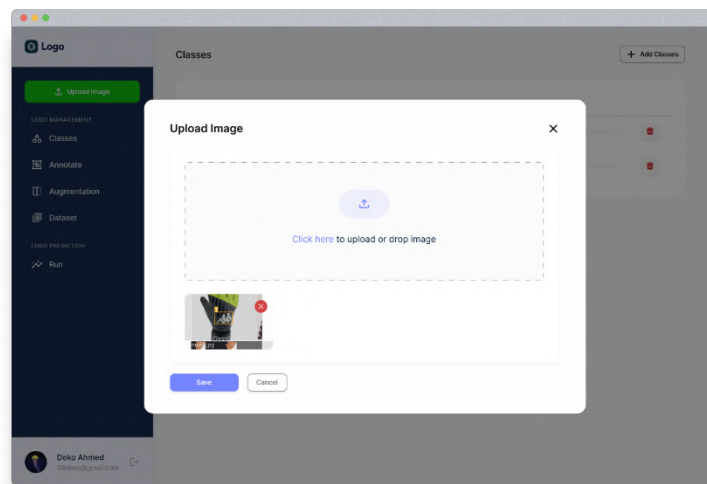
การออกแบบส่วนติดต่อผู้ใช้ของเว็บแอปพลิเคชันโดยคำนึงถึงผู้ใช้ทั่วไปที่ไม่มี
ความรู้เฉพาะทางมากนัก ให้สามารถใช้งานได้อย่างสะดวกและเข้าใจง่าย



รูป 3.11 ภาพรวมการออกแบบเว็บแอปพลิเคชัน

3.5.4.1 Modal สำหรับการอัปโหลดรูปภาพ

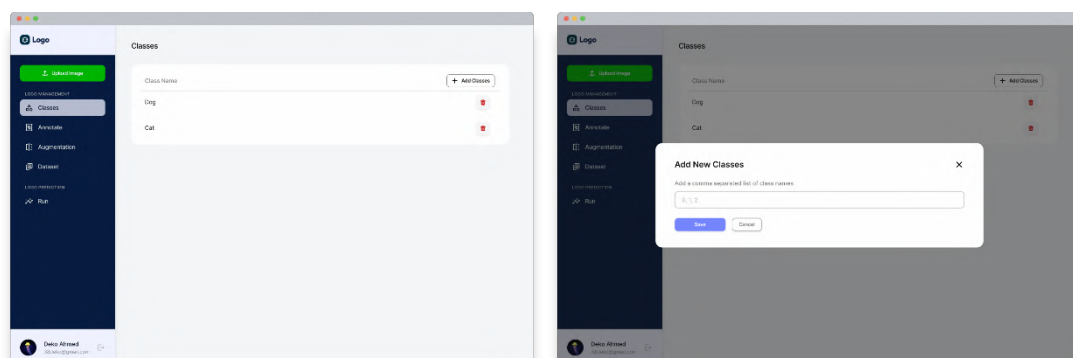
การเพิ่มรูปภาพเพื่อนำไปทำ Logo Management โดยสามารถคลิกเพื่อเลือกรูปภาพจากโฟลเดอร์ภายในเครื่องหรือลากและวางรูปที่ต้องการได้



รูป 3.12 Modal สำหรับการอัปโหลดรูปภาพ

3.5.4.2 หน้า Class

หน้า Class เป็นการจัดการกับ Class ที่ต้องการใช้งานสำหรับ Label โดยจะแสดง Class ทั้งหมดที่มีอยู่ สามารถเพิ่มและลบ Class ออกได้



ก)

ข)

รูป 3.13 ภาพรวมการออกแบบเว็บแอปพลิเคชัน

ก) หน้า Class

ข) Modal สำหรับการเพิ่ม Class

3.5.4.3 หน้า Annotate

หน้า Annotate เป็นการ Label โลโก้ที่ปรากฏอยู่ในภาพ ซึ่งจะอยู่ในรูปแบบของ YOLO Format โดยแบ่งการแสดงผลออกเป็น 3 ส่วน คือ

1) Canvas

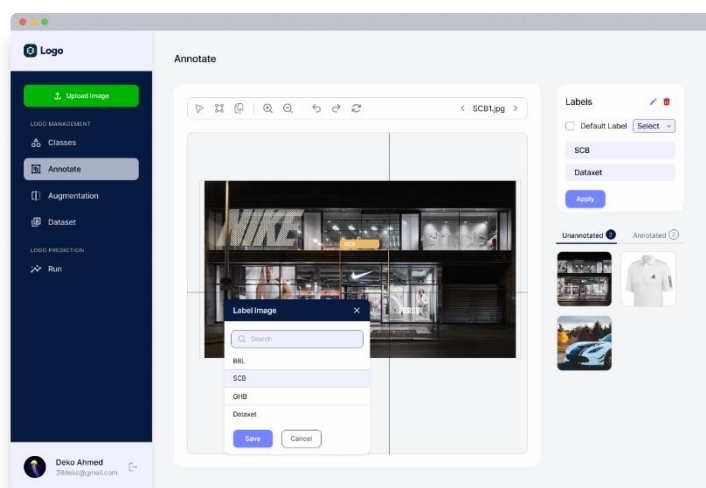
สามารถตรวจจับตำแหน่งของ cursor และแสดงเส้นตัดเพื่อให้่ายสำหรับการระบุตำแหน่งของโลโก้ภายในภาพ โดยสามารถเลือกใช้เครื่องมือสำหรับการทำ Annotation จาก Toolbar

2) Labels

แสดงผล class name ที่ได้ทำการ Annotation ทั้งหมดของรูปภาพที่เลือก เมื่อกดปุ่ม apply จะเป็นการบันทึกลง Dataset (หากรูปภาพที่เลือกเป็นภาพ Unannotated จะเปลี่ยนเป็น Annotated) และมีฟิเจอร์สำหรับการใช้งาน default class

3) Images

แสดงผลในรูปแบบของ Tab แบ่งออกเป็น Unannotated กับ Annotated โดย Unannotated คือรูปที่ได้จากการอัปโหลด ส่วน Annotated คือรูปที่มีการทำ Label เรียบร้อยแล้ว



รูป 3.14 หน้า Annotate

3.5.4.4 หน้า Augmentation

หน้า Augmentation เป็นการ Label โลโก้ที่ปรากฏอยู่ในภาพ ซึ่งจะอยู่ในรูปแบบของ YOLO Format โดยแบ่งการแสดงผลออกเป็น 3 ส่วน คือ

1) Images

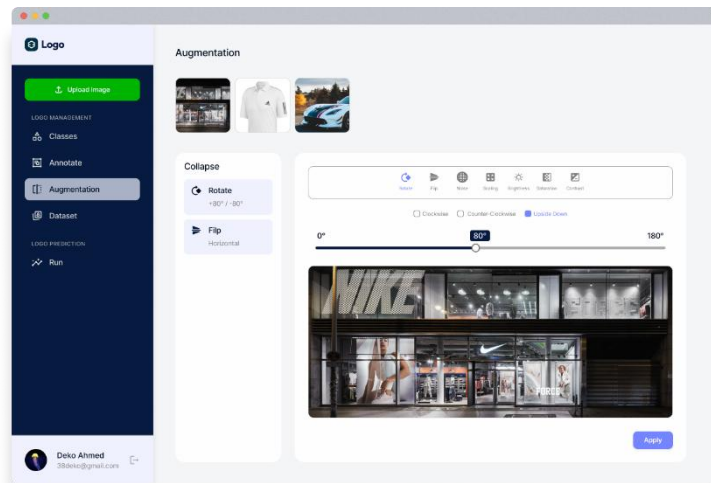
แสดงรูปภาพที่มีใน Dataset โดยสามารถกดเลือกที่รูปเพื่อแสดงภาพตัวอย่าง

2) Collapse

แสดงประเภทของการ Augment และค่า Augment ที่ถูกเลือกทั้งหมดในปัจจุบัน สามารถยกเลิกได้โดยการกดปุ่มกากบาทด้านขวาบนของกล่อง

3) Canvas

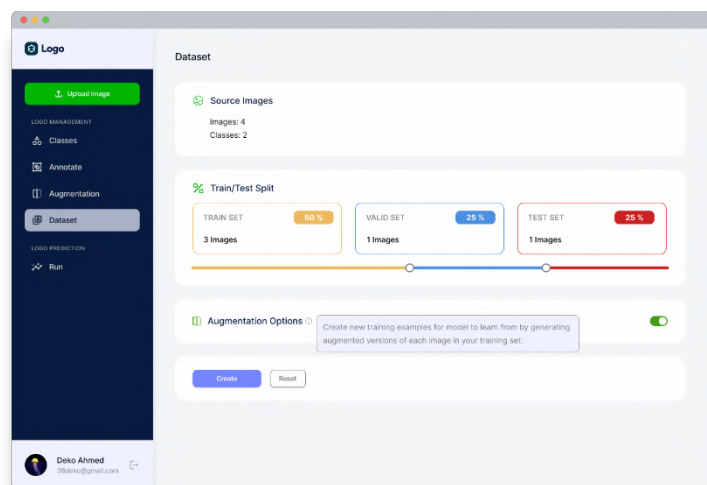
แสดงภาพตัวอย่างที่ได้จาก Augmented ที่เลือก สามารถเลือกใช้เครื่องมือสำหรับการทำ Augment จาก Toolbar โดยมีตัวเลือกให้ปรับได้ คือ Rotate, Flip, Noise, Scaling, Brightness, Saturation, และ Contrast



รูป 3.15 หน้า Augmentation

3.5.4.5 หน้าสร้าง Dataset

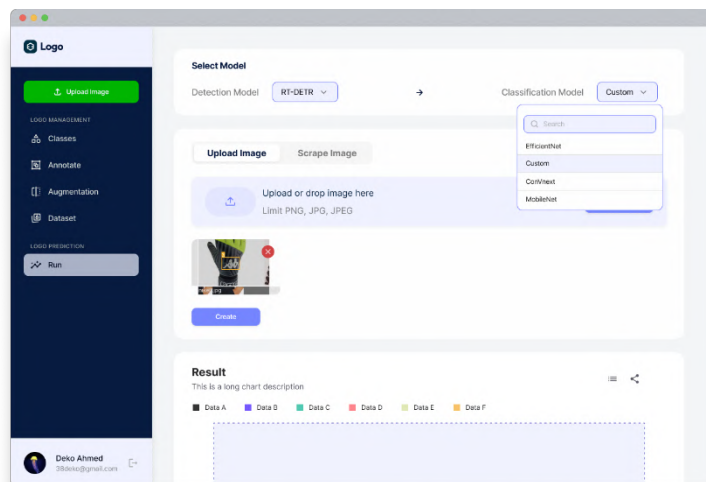
แสดงรายละเอียดของ Dataset ในขณะนั้น โดยจะบอกจำนวนของรูปภาพ ที่ทำการดี label เรียบร้อยแล้วและจำนวนคลาสทั้งหมดในระบบ สามารถปรับขนาดในแต่ละ Segment ของ Dataset ซึ่งประกอบด้วย Training set, Test set, และ Validation set อีกทั้งยังสามารถเลือก เปิด/ปิด การใช้ Augmentation ที่ทำการตั้งไว้ในหน้า Augment ได้



รูป 3.16 หน้าสร้าง Dataset

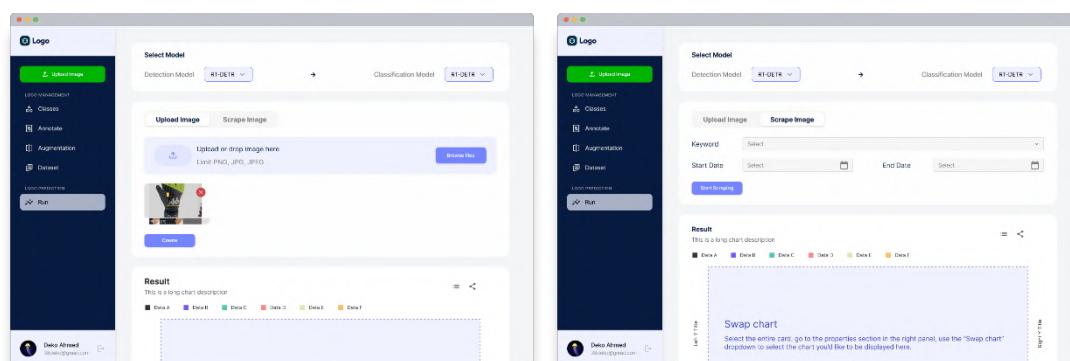
3.5.4.6 หน้าทำนายผลจากโมเดล

หน้า Run จะแสดงผลที่ทำนายได้จาก โมเดลที่เลือกทั้ง Detection Model และ Classification Model



รูป 3.17 การเลือกโมเดลที่ต้องการใช้ทำนายผล

โดยสามารถอัปโหลดภาพและเมื่อกดปุ่มทำนาย ผลลัพธ์ที่ได้จะแสดงเป็นภาพที่ถูกติกรอบพร้อมระบุ Class หรือหากต้องการที่จะนำเข้าข้อมูลรูปภาพจากโซเชียลมีเดียก็สามารถเลือกคีย์เวิร์ดของแบรนด์ที่ต้องการและเลือกระยะเวลาเพื่อนำเข้ารูปภาพและทำนายผลก็ได้เช่นกัน



ก)

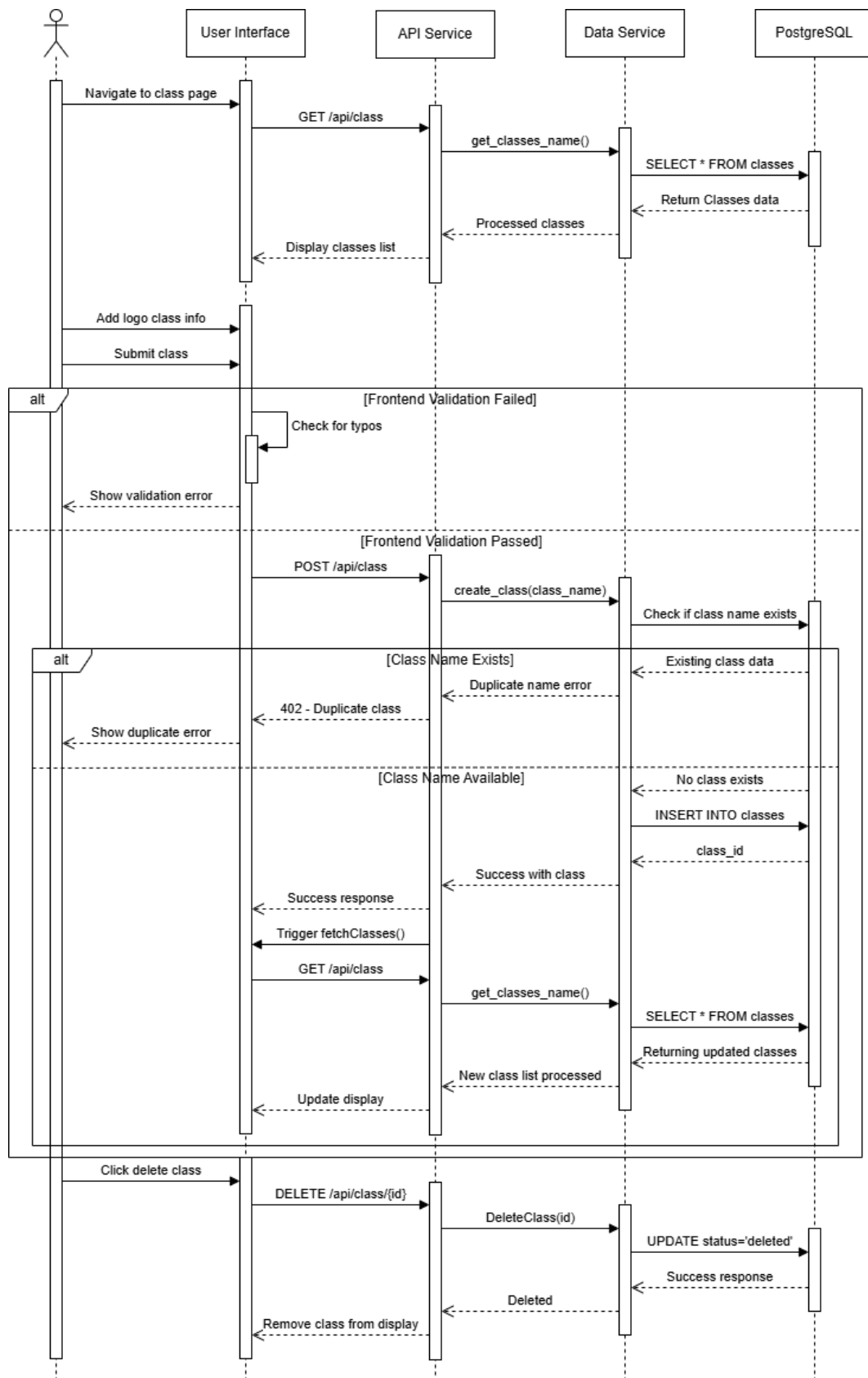
ข)

รูป 3.18 หน้าทำนายผลจากโมเดล

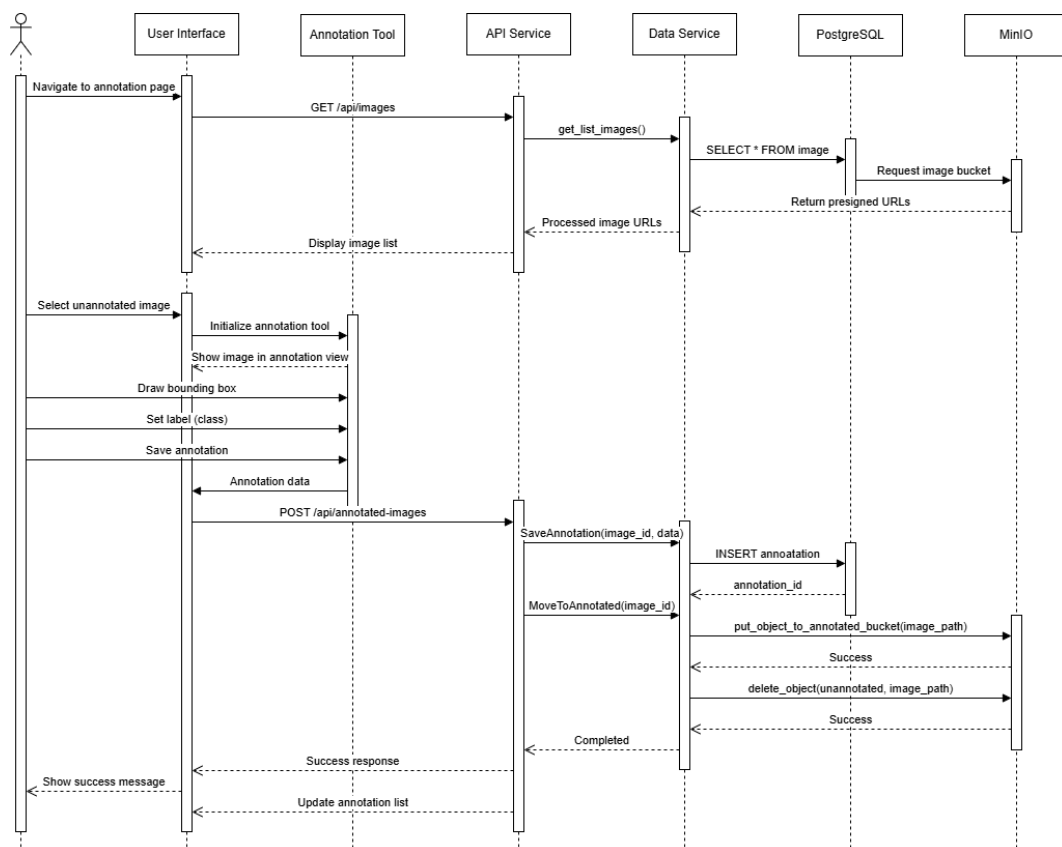
ก) อัปโหลดรูปภาพเพื่อทำนายผล

ข) การดึงข้อมูลภาพจากโซเชียลมีเดียเพื่อทำนายผล

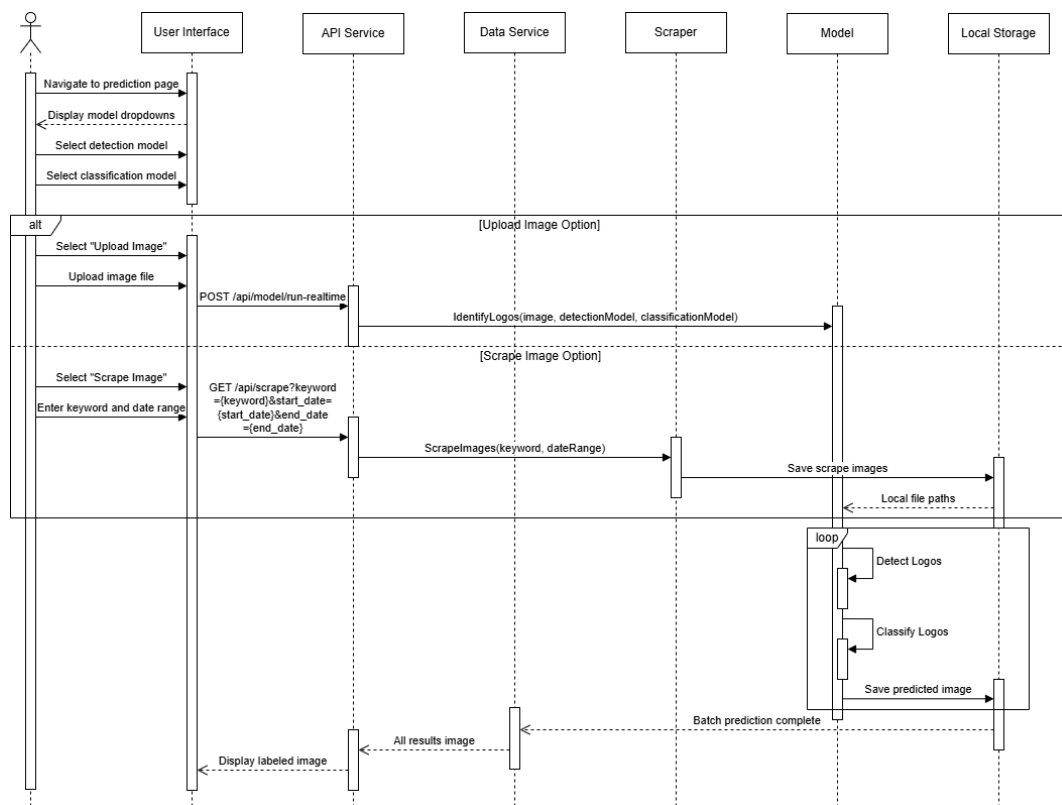
3.5.5 การออกแบบ Sequence Diagram



รูป 3.19 Sequence Diagram ของการจัดการ Class

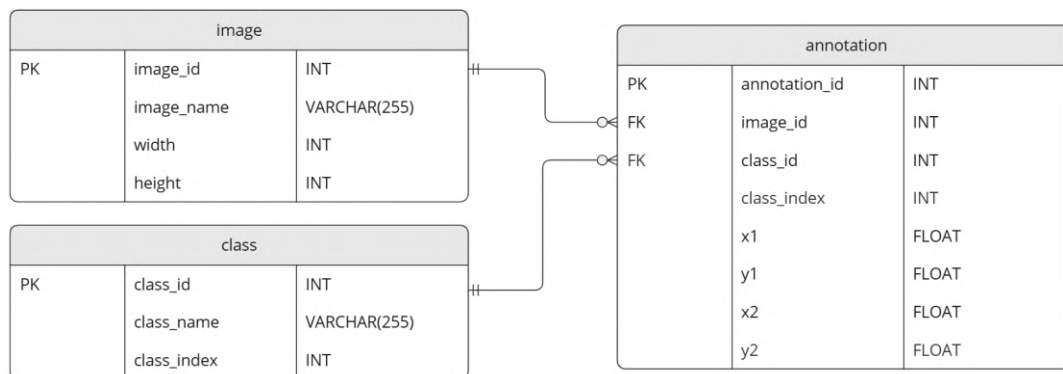


รูป 3.20 Sequence Diagram ของการทำ Annotation



รูป 3.21 Sequence Diagram ของการ Prediction

3.5.6 การออกแบบฐานข้อมูล



รูป 3.22 ER diagram

3.6 การวัดประสิทธิภาพของโมเดล

ตาราง 3.4 ตัวชี้วัดประสิทธิภาพ (Metrics)

ตัวชี้วัดประสิทธิภาพ	คำอธิบาย
Accuracy	ค่าความแม่นยำเป็นสัดส่วนระหว่างข้อมูลที่ทำนายถูกต้องทั้งหมดกับข้อมูลทั้งหมด
Precision	ค่าความแม่นยำในการทำนายในกลุ่มเป้าหมายเป็นสัดส่วนระหว่างข้อมูลที่ทำนายกลุ่มเป้าหมายถูกต้องกับการทำนายของกลุ่มเป้าหมายทั้งหมด
Recall	ค่าความถูกต้องเป็นอัตราส่วนของตัวอย่างที่ทำนายได้ถูกต้องต่อจำนวนตัวอย่างทั้งหมดที่เป็นจริงในกลุ่มที่ต้องการจะตรวจจับ
F1-Score	ค่าเฉลี่ยระหว่างความแม่นยำในการทำนายในกลุ่มเป้าหมาย (Precision) และความสามารถในการตรวจจับเหตุการณ์ที่เราสนใจ (Recall)
Intersection over Union (IoU)	วัดความแม่นยำในการตรวจจับวัตถุในภาพ โดยการหาพื้นที่ที่ทับซ้อนระหว่างกรอบจากการทำนายวัตถุและกรอบของวัตถุจริง
Mean Average Precision (mAP)	วัดประสิทธิภาพในการตรวจจับวัตถุหรือวัตถุต่างๆ โดยคำนวณค่าเฉลี่ยของ precision ที่ค่า recall เปลี่ยนแปลงไป ซึ่งเป็นวิธีที่ใช้รวมผลลัพธ์การตรวจจับของทุกคลาสเข้าด้วยกันเพื่อให้ได้ค่า ความแม่นยำของระบบทั้งหมดโดยรวม
Runtime	เวลาที่ใช้ในการตรวจจับและตรวจสอบโลโก้ จากรูปที่ใส่เข้าไป

ตัวชี้วัดประสิทธิภาพ	คำอธิบาย
Confusion Matrix	วัดประสิทธิภาพในการจำแนกประเภทของวัตถุ (Classification) ในประเภทต่าง ๆ โดยประกอบด้วย ค่าหลัก 4 ค่า ได้แก่ True Positive (TP): ทำนายว่าเป็นบวก และค่าจริงก็เป็นบวก True Negative (TN): ทำนายว่าเป็นลบ และค่าจริงก็เป็นลบ False Positive (FP): ทำนายว่าเป็นบวก แต่ค่าจริงเป็นลบ (Type I Error) False Negative (FN): ทำนายว่าเป็นลบ แต่ค่าจริงเป็นบวก (Type II Error)
Receiver Operating Characteristic (ROC)	วัดประสิทธิภาพในการจำแนกประเภทของวัตถุ (Classification) ในประเภทต่างๆ โดยการแสดงความสัมพันธ์ระหว่าง True Positive Rate (TPR) และ False Positive Rate (FPR)
Area Under the Curve (AUC)	วัดประสิทธิภาพของโมเดลจำแนกประเภท โดยคำนวณจากพื้นที่ใต้กราฟ ROC มีค่าอยู่ระหว่าง 0-1 ยิ่งค่า AUC ใกล้ 1 มาก โมเดลจะยิ่งมีความสามารถในการแยกแยะระหว่างคลาสต่าง ๆ ได้ดีขึ้น

ทำการทดสอบประสิทธิภาพของโมเดล ดังนี้

- 1) ใช้ข้อมูลชุดทดสอบ (Testing set) ที่ทำการแบ่งไว้ในขั้นตอนการเตรียม data เพื่อทดสอบความแม่นยำของโมเดล
- 2) วัดผลของโมเดล Logo Detection โดยดูจากตัวชี้วัดประสิทธิภาพดังตารางข้างต้น โดยมีค่า Mean Average Precision (mAP) และค่า Intersection over Union (IoU) เป็นหลัก
- 3) วัดผลของโมเดล Logo Classification โดยจะวัดจาก Testing set ที่ถูก crop จากค่า ground truth ที่ทำการ annotation ไว้ โดยจะทำการวัดประสิทธิภาพโดยรวมของโมเดลจากค่า Accuracy เป็นหลัก และดูผลประกอบกับค่า F1-score, Precision, Recall และ AUC และวัดประสิทธิภาพของแต่ละ class โดยดูได้จาก Confusion Matrix

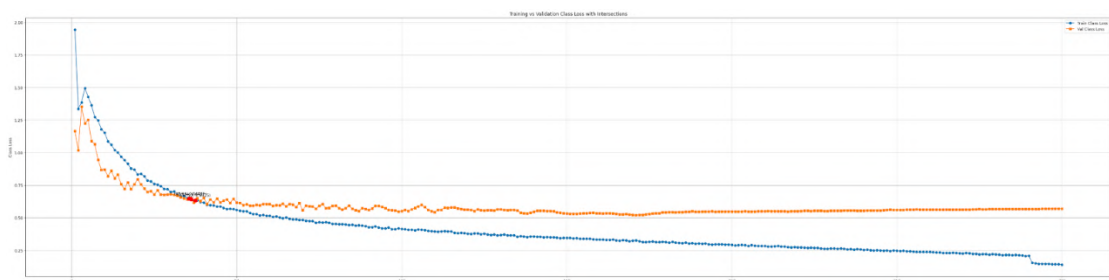
บทที่ 4

การทดลองและผลการทดลอง

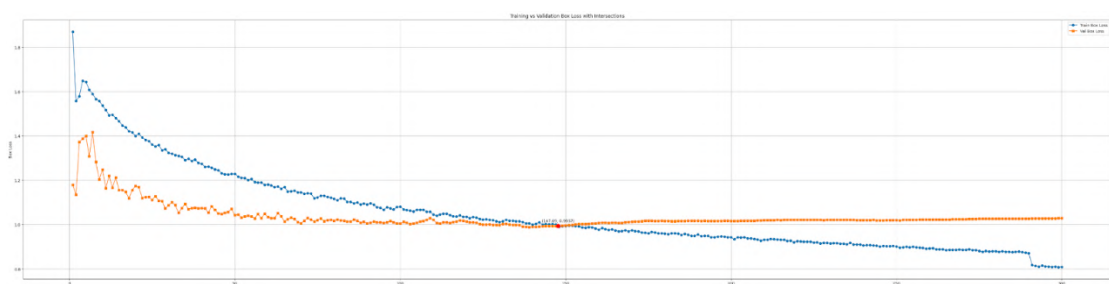
4.1 การทดลองหาจำนวน Epoch ที่ดีที่สุดในแต่ละโมเดล

ทางคณะผู้จัดทำได้ทำการทดสอบการเทรนทุกโมเดลกับแต่ละ Dataset เพื่อหาจำนวน Epoch ที่มากพอสำหรับแต่ละโมเดล และทำการเทรนแต่ละโมเดลด้วยจำนวน Epoch ดังกล่าว

หลังจากทำการเทรนโมเดลเสร็จสิ้น คณะผู้จัดทำได้พิจารณาจากค่า Box Loss และ Class Loss ที่มีความสำคัญต่อประสิทธิภาพของโมเดลมากที่สุด ด้วยเหตุนี้คณะผู้จัดทำจึงทำการเทรนโมเดลโดยที่บันทึก Weight ของโมเดลทุก Epoch และพิจารณาจากกราฟ Loss ได้แก่ Validation Loss และ Training Loss ของ Box Loss และ Class Loss จากนั้นพิจารณาจากจุดที่ตัดกันระหว่าง Validation Loss และ Training Loss โดยจะนำ Weight ในช่วงจุดตัดของ Loss ทั้งสองมาทำการวัดประสิทธิภาพด้วยชุด Test ชุดเดียวกันเพื่อหา Epoch ที่ดีที่สุด



รูป 4.1 กราฟ Class Loss ของ YOLOv8 กับ Flickr Logos-47



รูป 4.2 กราฟ Box Loss ของ YOLOv8 กับ Flickr Logos-47

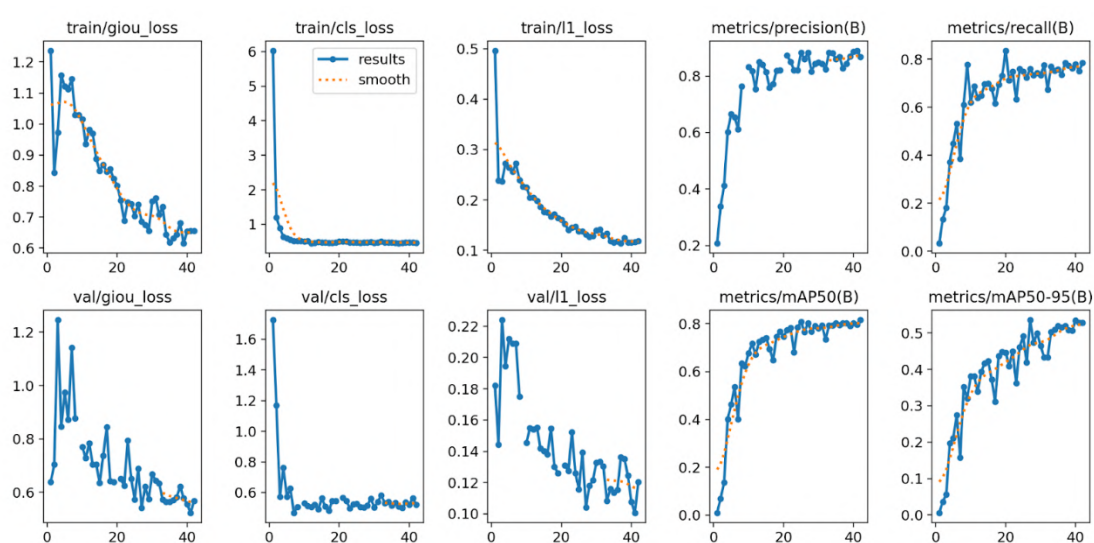
จากภาพดังกล่าวจะเห็นได้ว่า ในกราฟ Class Loss ค่าของ Training Loss กับ Validation Loss เริ่มตัดกันที่ Epoch 35 และในกราฟ Box Loss ค่าของ Training Loss กับ Validation Loss เริ่มตัดกันที่ Epoch 147 ดังนั้นช่วงของการหาค่า Epoch ที่ดีที่สุดของโมเดลอยู่ในช่วง 35 ถึง 147 โดยจะใช้ค่า

mAP50 ที่ทดสอบกับชุด Test ซึ่งกระบวนการดังกล่าวจะเป็นวิธีที่จะใช้ในการหาโมเดลที่ดีที่สุดของแต่ละโมเดลในทุกโมเดลการตรวจจับโลโก้

4.2 การทดลองเทรน Detection Model หลังเทรนกับ 22-Class Thai Logos

4.2.1 การเทรน YOLOv8 กับ 22-Class Thai Logos

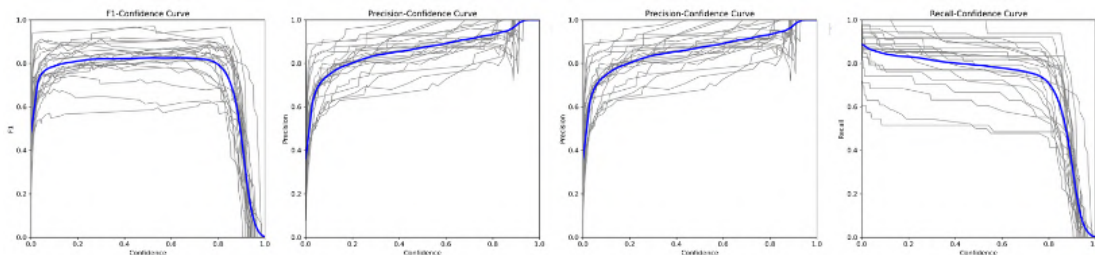
ทำการเทรน YOLOv8 ขนาด X ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 68.8 ล้านพารามิเตอร์กับ 22-Class Thai Logos โดยได้ Epoch ที่ 76 เป็น Epoch ที่ดีที่สุดเมื่อทดสอบกับชุด Test ซึ่งได้ค่า mAP50 อยู่ที่ 0.8516 และมีค่าความล่าช้าอยู่ที่ 13.3 มิลลิวินาที



รูป 4.3 กราฟแสดงค่า Metrics ในแต่ละ Epoch ของการเทรน YOLOv8 กับ 22-Class Thai Logos



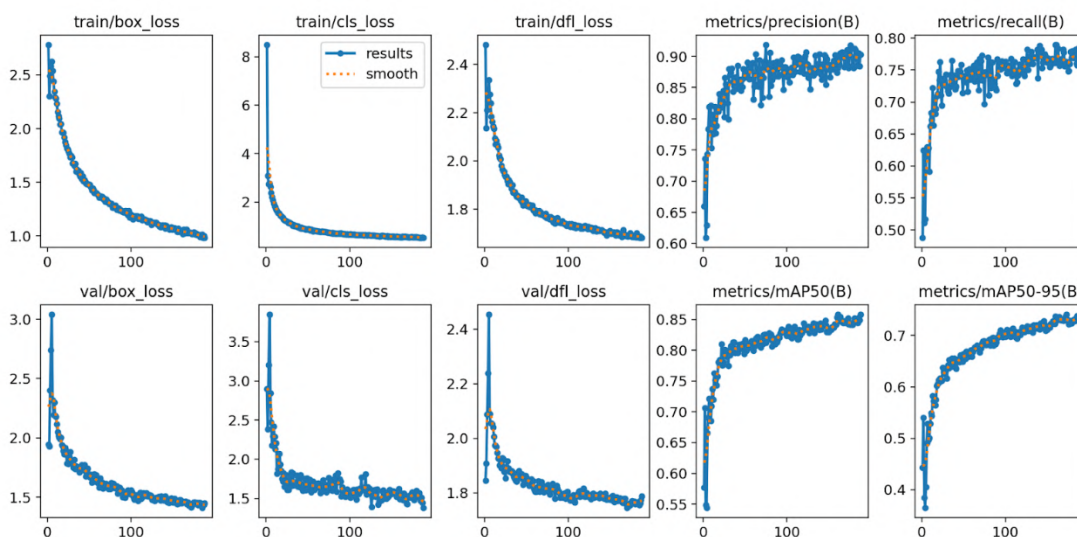
รูป 4.4 ตัวอย่างการทำนายกับชุด Test ของ YOLOv8 กับ 22-Class Thai Logos



รูป 4.5 กราฟแสดงค่า Metric ในการทำนายกับชุด Test ของ YOLOv8 กับ 22-Class Thai Logos

4.2.2 การเทรน YOLOv10 กับ 22-Class Thai Logos

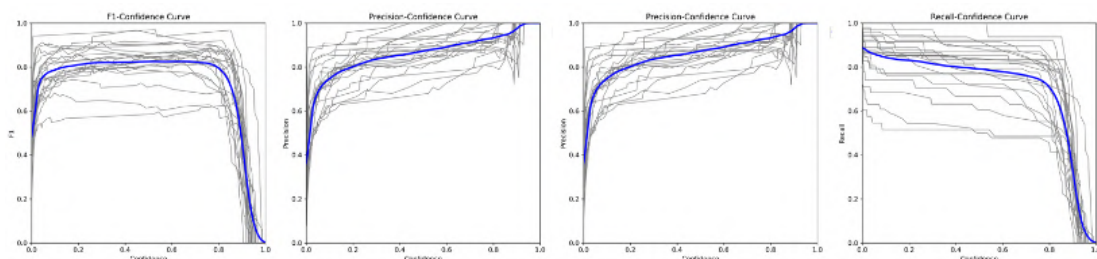
ทำการเทรน YOLOv10 ขนาด X ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 31.6 ล้านพารามิเตอร์กับ 22-Class Thai Logos โดยได้ Epoch ที่ 173 เป็น Epoch ที่ดีที่สุดเมื่อทดสอบกับชุด Test ซึ่งได้ค่า mAP50 อยู่ที่ 0.8569 และมีค่าความล่าช้าอยู่ที่ 14.1 มิลลิวินาที



รูป 4.6 กราฟแสดงค่า Metrics ในแต่ละ Epoch ของการเทรน YOLOv10 กับ 22-Class Thai Logos



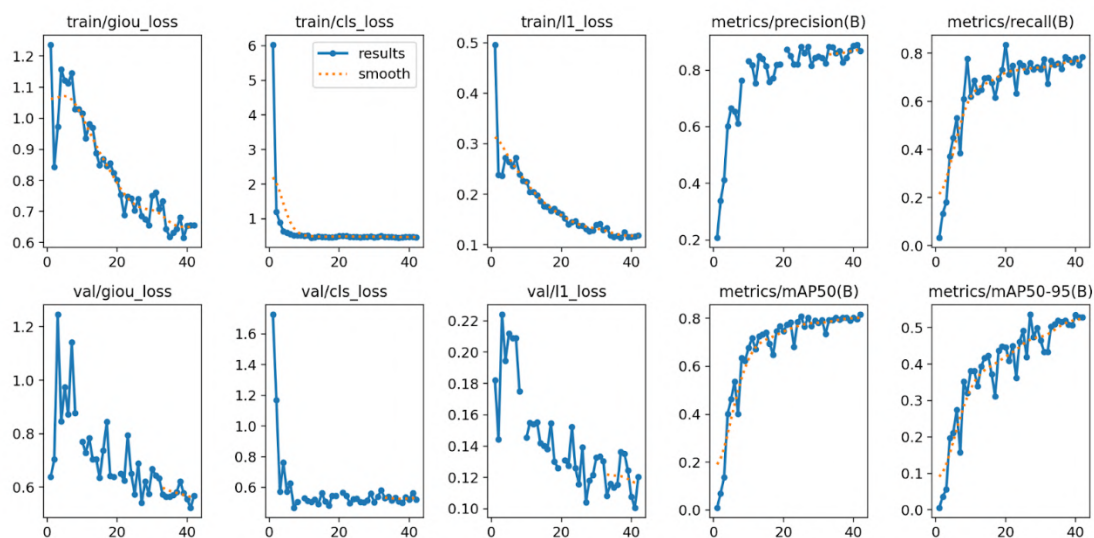
รูป 4.7 ตัวอย่างการทำนายกับชุด Test ของ YOLOv10 กับ 22-Class Thai Logos



รูป 4.8 กราฟแสดงค่า Metric ในการทำนายกับชุด Test ของ YOLOv10 กับ 22-Class Thai Logos

4.2.3 การเทรน RT-DETR กับ 22-Class Thai Logos

ทำการเทรน RT-DETR ขนาด X ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 65.5 ล้านพารามิเตอร์กับ 22-Class Thai Logos โดยได้ Epoch ที่ 42 เป็น Epoch ที่ดีที่สุดเมื่อทดสอบกับชุด Test ซึ่งได้ค่า mAP50 อยู่ที่ 0.8447 และมีค่าความล่าช้าอยู่ที่ 26.62 มิลลิวินาที

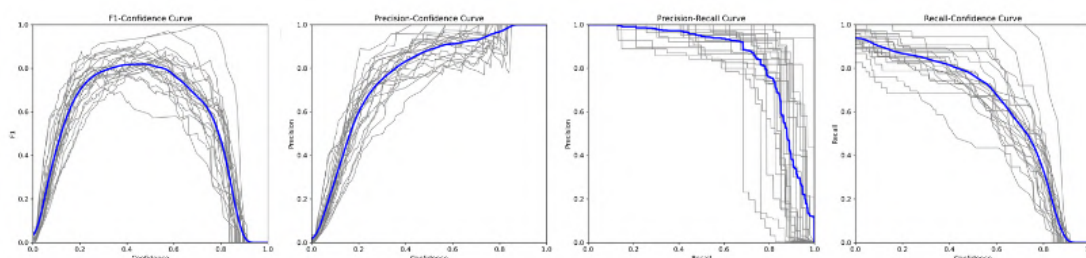


รูป 4.9 กราฟแสดงค่า Metrics ในแต่ละ Epoch ของการเทรน RT-DETR กับ

22-Class Thai Logos



รูป 4.10 ตัวอย่างการทำนายกับชุด Test ของ RT-DETR กับ 22-Class Thai Logos



รูป 4.11 กราฟแสดงค่า Metric ในการทำนายกับชุด Test ของ RT-DETR กับ 22-Class Thai Logos

4.2.4 ผลการทดลองเทรน Detection Model กับ 22-Class Thai Logos

ผลลัพธ์ของการเทรนโมเดล YOLOv8, YOLOv10, และ RT-DETR ด้วย 22-Class Thai Logos โดยได้ทำการทดสอบกับชุด Test ที่ทำการแบ่งไว้สำหรับทดสอบประสิทธิภาพซึ่งเป็นภาพชุดเดียวกันและได้ผลลัพธ์ประสิทธิภาพของโมเดล ดังนี้

ตาราง 4.1 ผลการทดสอบ Detection Model หลังเทรนกับ 22-Class Thai Logos

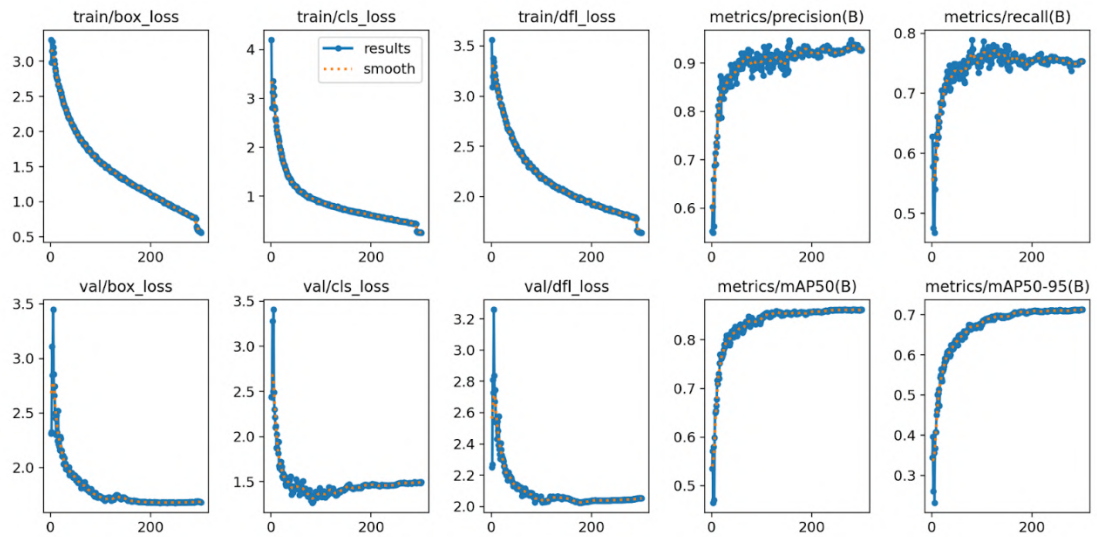
Dataset	Model		epochs	mAP50	param(m)	runtime (ms)
22-Class Thai	YOLOv8	YOLOv8-X	76	0.8516	68.2	13.3
Logos	YOLOv10	YOLOv10-X	173	0.8569	31.6	14.1
	RT-DETR	RT-DETR-X	42	0.8447	65.5	26.62

จากตารางที่ 4.1 โมเดล YOLOv10-X ได้ค่า mAP50 ที่ดีที่สุดคือ 0.8569 และจำนวน Parameter ของ YOLOv10-X มีอยู่ที่ 31.6 m ซึ่งนับว่าเป็นโมเดลที่มีขนาดเล็กที่สุดเมื่อเทียบกับโมเดลตัวอื่น ๆ ดังนั้นจึงสามารถสรุปได้ว่า ในด้านของความแม่นยำกับ 22-Class Thai Logos โมเดล YOLOv10-X ทำได้ดีที่สุดรวมทั้งขนาดที่เล็กโมเดล และส่วนในด้านความเร็วในการทำนาย YOLOv8 ทำได้ดีกว่าโมเดลตัวอื่น

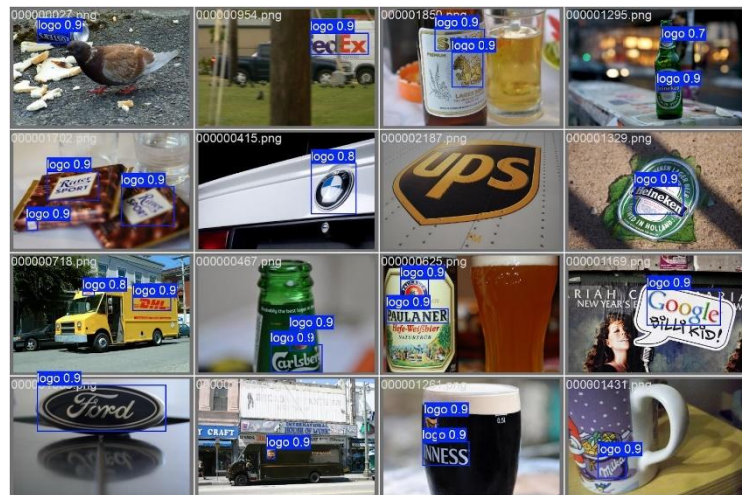
4.3 การทดลองเทรน Detection Model กับ FlickrLogos-47 Dataset

4.3.1 การเทรน YOLOv8 กับ FlickrLogos-47 Dataset

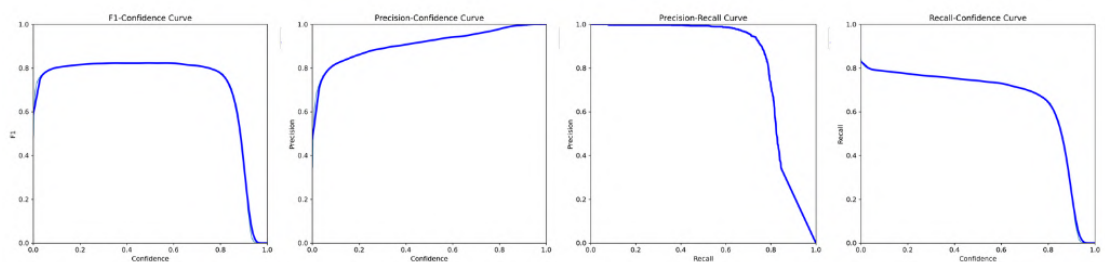
ทำการเทรน YOLOv8 ขนาด X ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 68.8 ล้านพารามิเตอร์กับ FlickrLogos-47 Dataset โดยได้ Epoch ที่ 69 เป็น Epoch ที่ดีที่สุดเมื่อทดสอบกับชุด Test ซึ่งได้ค่า mAP50 อยู่ที่ 0.835 และมีค่าความล่าช้าในการทำนายอยู่ที่ 14 มิลลิวินาที



รูป 4.12 กราฟแสดงค่า Metrics ในแต่ละ Epoch ของการเทรน YOLOv8 กับ FlickrLogos-47



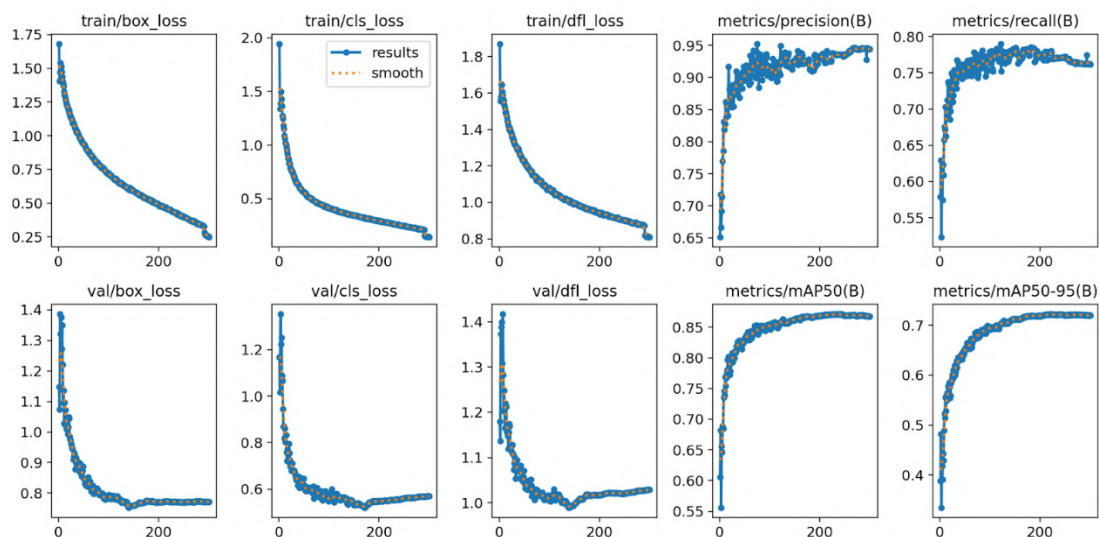
รูป 4.13 ตัวอย่างการทำนายกับชุด Test ของ YOLOv8 กับ FlickrLogos-47



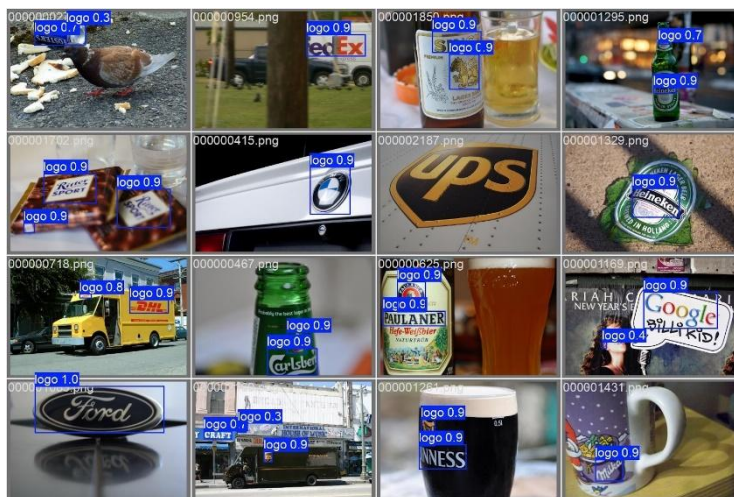
รูป 4.14 กราฟแสดงค่า Metric ในการทำนายกับชุด Test ของ YOLOv8 กับ FlickrLogos-47

4.3.2 การเทรน YOLOv10 กับ FlickrLogos-47 Dataset

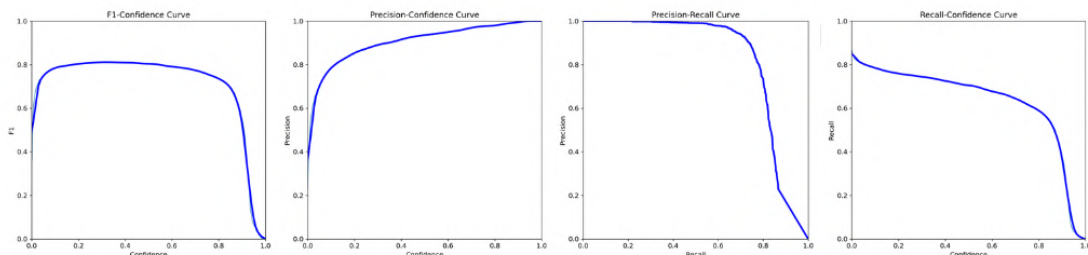
ทำการเทรน YOLOv10 ขนาด X ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 31.6 ล้านพารามิเตอร์กับ FlickrLogos-47 Dataset โดยได้ Epoch ที่ 47 เป็น Epoch ที่ดีที่สุดเมื่อทดสอบกับชุด Test ซึ่งได้ค่า mAP50 อยู่ที่ 0.829 และมีค่าความล่าช้าในการทำนายอยู่ที่ 14.7 มิลลิวินาที



รูป 4.15 กราฟแสดงค่า Metrics ในแต่ละ Epoch ของการเทรน YOLOv10 กับ FlickrLogos-47



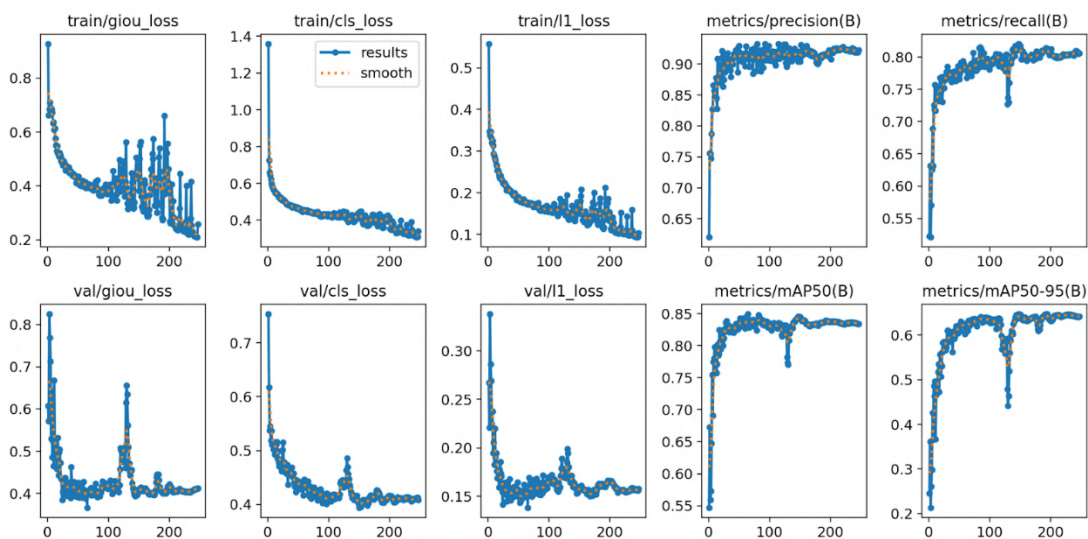
รูป 4.16 ตัวอย่างการทำนายกับชุด Test ของ YOLOv10 กับ FlickrLogos-47



รูป 4.17 กราฟแสดงค่า Metric ในการทำนายกับชุด Test ของ YOLOv10 กับ FlickrLogos-47

4.3.3 การเทรน RT-DETR กับ FlickrLogos-47 Dataset

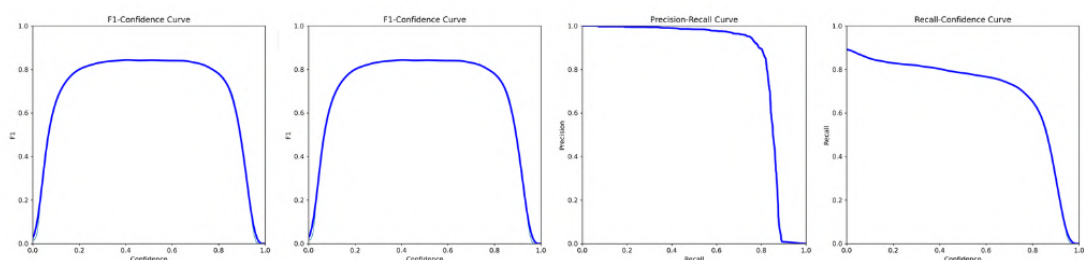
ทำการเทรน RT-DETR ขนาด X ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 65.5 ล้านพารามิเตอร์กับ FlickrLogos-47 Dataset โดยได้ Epoch ที่ 83 เป็น Epoch ที่ดีที่สุดเมื่อทดสอบกับชุด Test ซึ่งได้ค่า mAP50 อยู่ที่ 0.836 และมีค่าความล่าช้าในการทำนายอยู่ที่ 26 มิลลิวินาที



รูป 4.18 กราฟแสดงค่า Metrics ในแต่ละ Epoch ของการเทรน RT-DETR กับ FlickrLogos-47



รูป 4.19 ตัวอย่างการทำนายกับชุด Test ของ RT-DETR กับ FlickrLogos-47



รูป 4.20 กราฟแสดงค่า Metric ในการทำนายกับชุด Test ของ RT-DETR กับ FlickrLogos-47

4.3.4 ผลการทดลองเทรน Detection Model กับ FlickrLogos-47 Dataset

ผลลัพธ์ของการเทรนโมเดล YOLOv8, YOLOv10, และ RT-DETR ด้วย FlickrLogos-47 Dataset โดยได้ทำการทดสอบกับชุด Test ที่ทำการแบ่งไว้สำหรับทดสอบประสิทธิภาพซึ่งเป็นภาพชุดเดียวกัน และได้ผลลัพธ์ประสิทธิภาพของโมเดล ดังนี้

ตาราง 4.2 ผลการทดสอบ Detection Model หลังเทรนกับ FlickrLogos-47 Dataset

Dataset	Model		epochs	mAP50	param(m)	runtime (ms)
FlickrLogos-47	YOLOv8	YOLOv8-X	69	0.835	68.2	14
	YOLOv10	YOLOv10-X	47	0.829	31.6	14.7
	RT-DETR	RT-DETR-X	83	0.836	65.5	26

จากตารางที่ 4.2 โมเดล RT-DETR-X ได้ค่า mAP50 ที่ดีที่สุดคือ 0.836 และ YOLOv10-X ให้ค่า Runtime อยู่ที่ 15.01 ms ซึ่งมีความล่าช้าน้อยที่สุด นอกจากนี้จำนวน Parameter ของ YOLOv10-X มีอยู่ที่ 31.6 m ซึ่งนับว่าเป็นโมเดลที่มีขนาดเล็กที่สุดเมื่อเทียบกับโมเดลตัวอื่น ๆ

จากตารางดังกล่าวสรุปได้ว่า ในด้านของความแม่นยำต่อ FlickrLogos-47 Dataset โมเดล RT-DETR-X ทำได้ดีที่สุดและในด้านของความเร็วในการทำนายและขนาดที่เล็กโมเดล YOLOv10-X ทำได้ดีที่สุด

4.4 การทดลองเทรน Classification model โดยใช้ Pre-trained CNN model

4.4.1 การทดลอง Hyperparameter tuning classification model กับ 22-Class Thai

Logos

ทำการเทรน YOLOv8 ขนาด X ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 68.8 ล้านพารามิเตอร์กับ 22-Class Thai Logos โดยได้ Epoch ที่ 76 เป็น Epoch ที่ดีที่สุดเมื่อทดสอบกับชุด Test ซึ่งได้ค่า mAP50 อยู่ที่ 0.8516 และมีค่าความล่าช้าอยู่ที่ 13.3 มิลลิวินาที

4.4.1.1 การ Hyperparameter tuning EfficientNetV2 กับ 22-Class Thai Logos

ทำการเทรน EfficientNetV2 ขนาด B0 ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 7.1 ล้านพารามิเตอร์กับ 22-Class Thai Logos โดยหลังจากการทำ Hyperparameter tuning ได้พารามิเตอร์ที่ดีที่สุดคือ learning_rate เท่ากับ 0.003 และ dropout_rate เท่ากับ 0.1 เมื่อทดสอบกับชุด Test ได้ค่า Accuracy อยู่ที่ 0.9613

4.4.1.2 การ Hyperparameter tuning InceptionV3 กับ 22-Class Thai Logos

ทำการเทรน InceptionV3 ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 23.9 ล้านพารามิเตอร์กับ 22-Class Thai Logos โดยหลังจากการทำ Hyperparameter tuning ได้พารามิเตอร์ที่ดีที่สุดคือ learning_rate เท่ากับ 0.003 และ dropout_rate เท่ากับ 0.2 เมื่อทดสอบกับชุด Test ได้ค่า Accuracy อยู่ที่ 0.9517

4.4.1.3 การ Hyperparameter tuning InceptionResNetV2 กับ 22-Class Thai

Logos

ทำการเทรน InceptionResNetV2 ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 55.9 ล้านพารามิเตอร์กับ 22-Class Thai Logos โดยหลังจากการทำ Hyperparameter tuning ได้พารามิเตอร์ที่ดีที่สุดคือ learning_rate เท่ากับ 0.003 และ dropout_rate เท่ากับ 0.3 เมื่อทดสอบกับชุด Test ได้ค่า Accuracy อยู่ที่ 0.9421

4.4.1.4 การ Hyperparameter tuning ConvNext กับ 22-Class Thai Logos

ทำการเทรน ConvNext ขนาด Tiny ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 28 ล้านพารามิเตอร์กับ 22-Class Thai Logos โดยหลังจากการทำ Hyperparameter tuning ได้

พารามิเตอร์ที่ดีที่สุดคือ learning_rate เท่ากับ 0.003 และ dropout_rate เท่ากับ 0.3 เมื่อทดสอบกับชุด Test ได้ค่า Accuracy อยู่ที่ 0.8841

4.4.1.5 การ Hyperparameter tuning MobileNetV3 กับ 22-Class Thai Logos

ทำการเทรน MobileNetV3 ขนาด Large ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 5.9 ล้านพารามิเตอร์กับ 22-Class Thai Logos โดยหลังจากการทำ Hyperparameter tuning ได้พารามิเตอร์ที่ดีที่สุดคือ learning_rate เท่ากับ 0.01 และ dropout_rate เท่ากับ 0.1 เมื่อทดสอบกับชุด Test ได้ค่า Accuracy อยู่ที่ 0.9633

4.4.1.6 ผลการทดลอง Hyperparameter tuning classification model กับ 22-Class Thai Logos

จากผลการเปรียบเทียบผลที่ได้หลังจากการทำ Hyperparameter tuning หากพิจารณา Test_Accuracy จะเห็นได้ว่าโมเดลที่มีประสิทธิภาพดีที่สุด 3 อันดับแรกคือ MobileNetV3, EfficientNetV2 และ InceptionV3 ตามลำดับ

ตาราง 4.3 ผลการทดสอบ Hyperparameter tuning classification model กับ 22-Class Thai Logos

Dataset	Model	param (M)	learning_rate	dropout_rate	Test_Accuracy
22-Class Thai Logos	EfficientNetV2	7.1	0.003	0.1	0.9613
	InceptionV3	23.9	0.003	0.2	0.9517
	InceptionResNetV2	55.9	0.003	0.3	0.9421
	ConvNeXt	28	0.003	0.3	0.8841
	MobileNetV3	5.9	0.01	0.1	0.9633

4.4.2 การทดลอง Hyperparameter tuning classification model กับ FlickrLogos-47

4.4.2.1 การ Hyperparameter tuning EfficientNetV2 กับ FlickrLogos-47

ทำการเทรน EfficientNetV2 ขนาด B0 ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 7.1 ล้านพารามิเตอร์กับ FlickrLogos-47 โดยหลังจากการทำ Hyperparameter tuning ได้พารามิเตอร์ที่ดีที่สุดคือ learning_rate เท่ากับ 0.003 และ dropout_rate เท่ากับ 0.1 เมื่อทดสอบกับชุด Test ได้ค่า Accuracy อยู่ที่ 0.8608

4.4.2.2 การ Hyperparameter tuning InceptionV3 กับ FlickrLogos-47

ทำการเทรน InceptionV3 ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 23.9 ล้านพารามิเตอร์กับ FlickrLogos-47 โดยหลังจากการทำ Hyperparameter tuning ได้พารามิเตอร์ที่ดีที่สุด

คือ learning_rate เท่ากับ 0.001 และ dropout_rate เท่ากับ 0.1 เมื่อทดสอบกับชุด Test ได้ค่า Accuracy อยู่ที่ 0.8251

4.4.2.3 การ Hyperparameter tuning InceptionResNetV2 กับ FlickrLogos-47

ทำการเทรน InceptionResNetV2 ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 55.9 ล้านพารามิเตอร์กับ FlickrLogos-47 โดยหลังจากการทำ Hyperparameter tuning ได้พารามิเตอร์ที่ดีที่สุดคือ learning_rate เท่ากับ 0.003 และ dropout_rate เท่ากับ 0.2 เมื่อทดสอบกับชุด Test ได้ค่า Accuracy อยู่ที่ 0.8177

4.4.2.4 การ Hyperparameter tuning ConvNext กับ FlickrLogos-47

ทำการเทรน ConvNext ขนาด Tiny ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 28 ล้านพารามิเตอร์กับ FlickrLogos-47 โดยหลังจากการทำ Hyperparameter tuning ได้พารามิเตอร์ที่ดีที่สุดคือ learning_rate เท่ากับ 0.001 และ dropout_rate เท่ากับ 0.1 เมื่อทดสอบกับชุด Test ได้ค่า Accuracy อยู่ที่ 0.885

4.4.2.5 การ Hyperparameter tuning MobileNetV3 กับ FlickrLogos-47

ทำการเทรน MobileNetV3 ขนาด Large ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 5.9 ล้านพารามิเตอร์กับ FlickrLogos-47 โดยหลังจากการทำ Hyperparameter tuning ได้พารามิเตอร์ที่ดีที่สุดคือ learning_rate เท่ากับ 0.01 และ dropout_rate เท่ากับ 0.1 เมื่อทดสอบกับชุด Test ได้ค่า Accuracy อยู่ที่ 0.8666

4.4.2.6 ผลการทดลอง Hyperparameter tuning classification model กับ

FlickrLogos-47

จากผลการเปรียบเทียบผลที่ได้หลังจากการทำ Hyperparameter tuning หากพิจารณา Test_Accuracy จะเห็นได้ว่าโมเดลที่มีประสิทธิภาพดีที่สุด 3 อันดับแรกคือ ConvNext, MobileNetV3 และ EfficientNetV2 ตามลำดับ

ตาราง 4.4 ผลการทดสอบ Hyperparameter tuning classification model กับ FlickrLogos-47

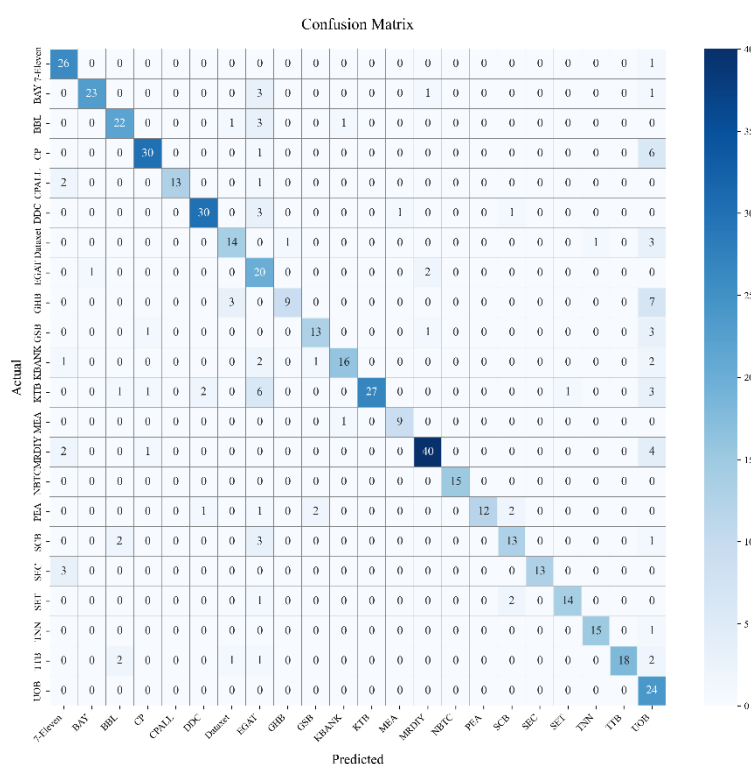
Dataset	Model	param (M)	learning_rate	dropout_rate	Test_Accuracy
FlickrLogos-47	EfficientNetV2	7.1	0.003	0.1	0.8608
	InceptionV3	23.9	0.001	0.1	0.8251
	InceptionResNetV2	55.9	0.003	0.2	0.8177
	ConvNeXt	28	0.003	0.1	0.885
	MobileNetV3	5.9	0.01	0.1	0.8666

4.4.3 การทดลอง Fine-tuning classification model กับ 22-Class Thai Logos

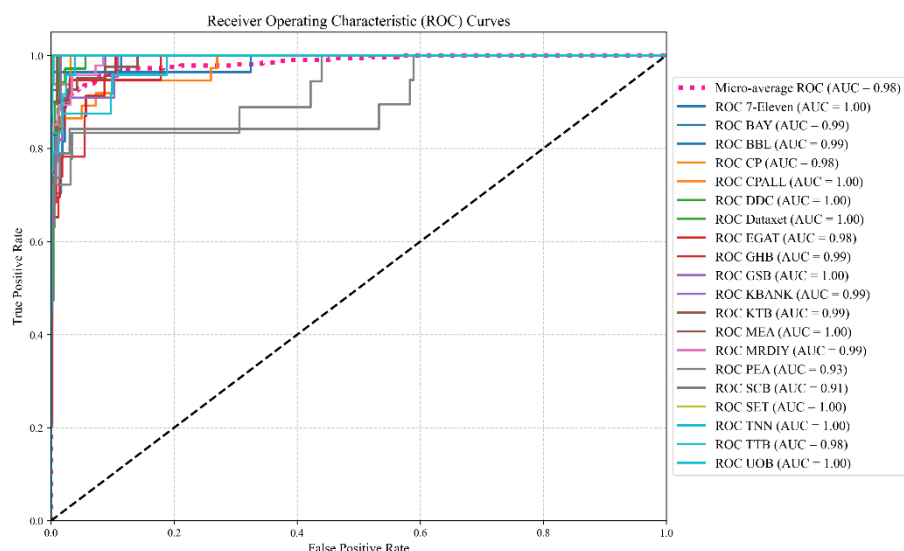
หลังจากการทดลอง Hyperparameter tuning กับ CNN ทั้ง 5 ตัวกับ 22-Class Thai Logos ภายใต้ Classification head เดียวกัน โดยเลือก 3 ตัวที่ดีที่สุดจากค่า Accuracy ด้วยเหตุผลนี้จึงเลือก EfficientNetV2, InceptionResNetV2 และ MobileNetV3 มาทำการ Fine-tuning เพื่อปรับแต่ง Layer ของ Classification head ให้มีประสิทธิภาพสูงสุดต่อ 22-Class Thai Logos

4.4.3.1 การทดลอง Fine-tuning EfficientNetV2 กับ 22-Class Thai Logos

ทำการ Fine-tuning EfficientNetV2 กับ 22-Class Thai Logos โดยมีการปรับแต่ง Layer ในส่วน Classification head ซึ่งได้ค่า Accuracy เท่ากับ 0.9652 เมื่อทดสอบกับชุด Test



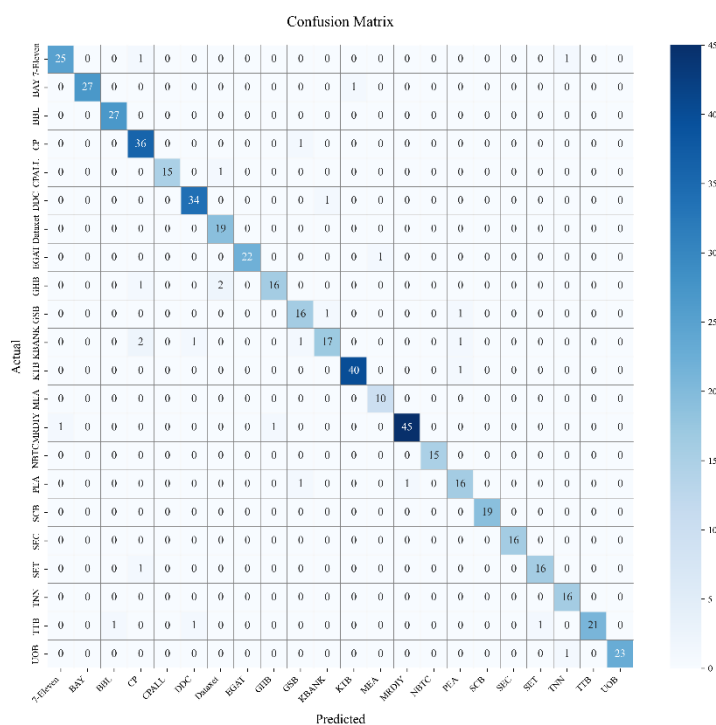
รูป 4.21 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ EfficientNetV2 กับ 22-Class Thai Logos



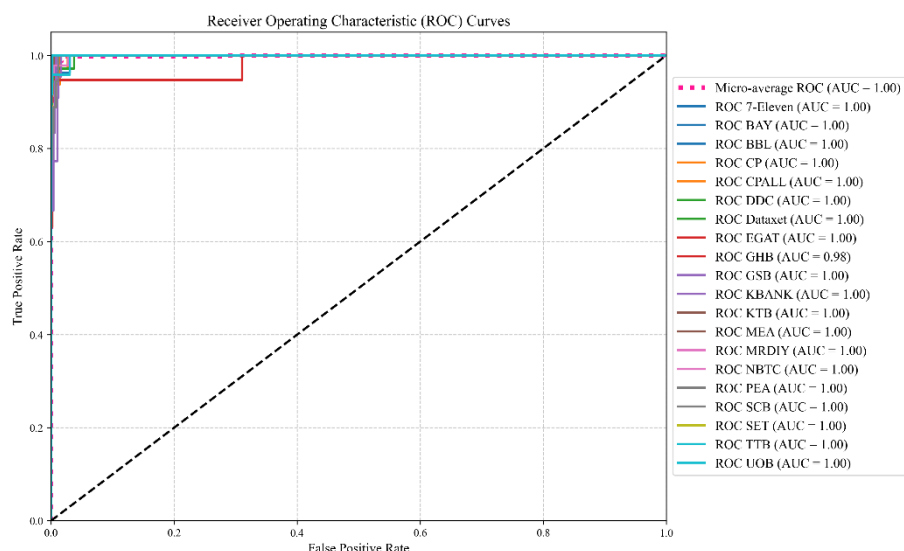
รูป 4.22 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ EfficientNetV2 กับ 22-Class Thai Logos

4.4.3.2 การทดลอง Fine-tuning InceptionResNetV2 กับ 22-Class Thai Logos

ทำการ Fine-tuning InceptionResNetV2 กับ 22-Class Thai Logos โดยมีการปรับแต่ง Layer ในส่วน Classification head ซึ่งได้ค่า Accuracy เท่ากับ 0.9517 เมื่อทดสอบกับชุด Test



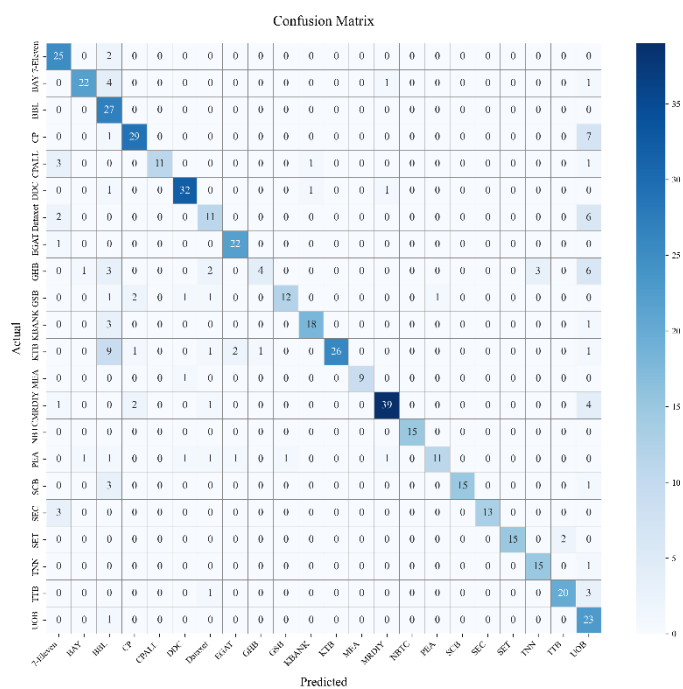
รูป 4.23 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ InceptionResNetV2 กับ 22-Class Thai Logos



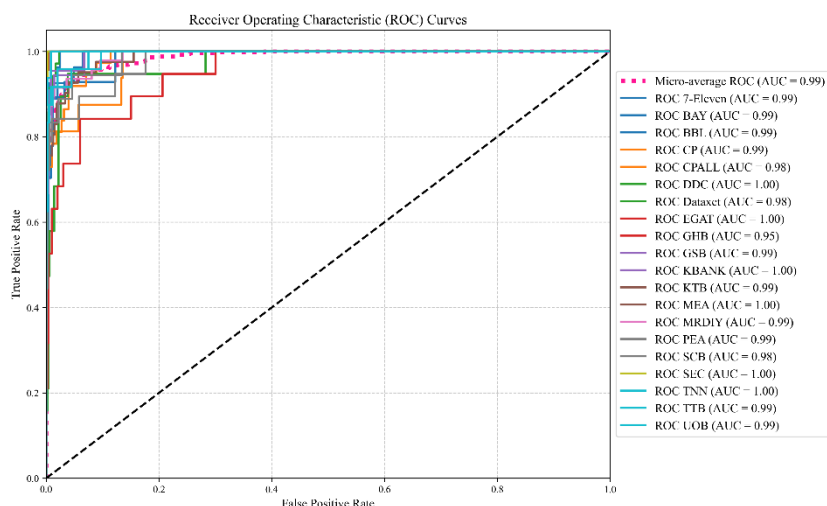
รูป 4.24 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ InceptionResNetV2 กับ 22-Class Thai Logos

4.4.3.3 การทดลอง Fine-tuning MobileNetV3 กับ 22-Class Thai Logos

ทำการ Fine-tuning MobileNetV3 กับ 22-Class Thai Logos โดยมี การปรับแต่ง Layer ในส่วน Classification head ซึ่งได้ค่า Accuracy เท่ากับ 0.9633 เมื่อทดสอบกับชุด Test



รูป 4.25 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ MobileNetV3 กับ 22-Class Thai Logos



รูป 4.26 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ MobileNetV3 กับ 22-Class Thai Logos

4.4.3.4 ผลการทดลอง Fine-tuning classification model กับ 22-Class Thai

Logos

หลังจากทำการ Fine-tuning classification model กับ 22-Class Thai Logos ทำให้ได้ค่า Accuracy เมื่อทดสอบกับชุด Test เพิ่มขึ้นประมาณ 0.01-0.02 โดยได้ค่า Accuracy ของแต่ละโมเดล ดังนี้

ตาราง 4.5 ผลการทดสอบ Fine-tuning classification model กับ 22-Class Thai Logos

Dataset	Model	Param (M)	Test_Accuracy
22-Class Thai Logos	EfficientNetV2	7.1	0.9652
	InceptionV3	23.9	0.9517
	MobileNetV3	5.9	0.9633

จากตารางข้างต้น แสดงให้เห็นว่าในส่วนของการ Pre-trained model ที่ทำการเลือกมา EfficientNetV2 ได้ค่า Accuracy เมื่อทดสอบกับชุด Test มากที่สุดอยู่ที่ 0.9652 ซึ่งมีจำนวนพารามิเตอร์จำนวน 7.1 ล้านพารามิเตอร์

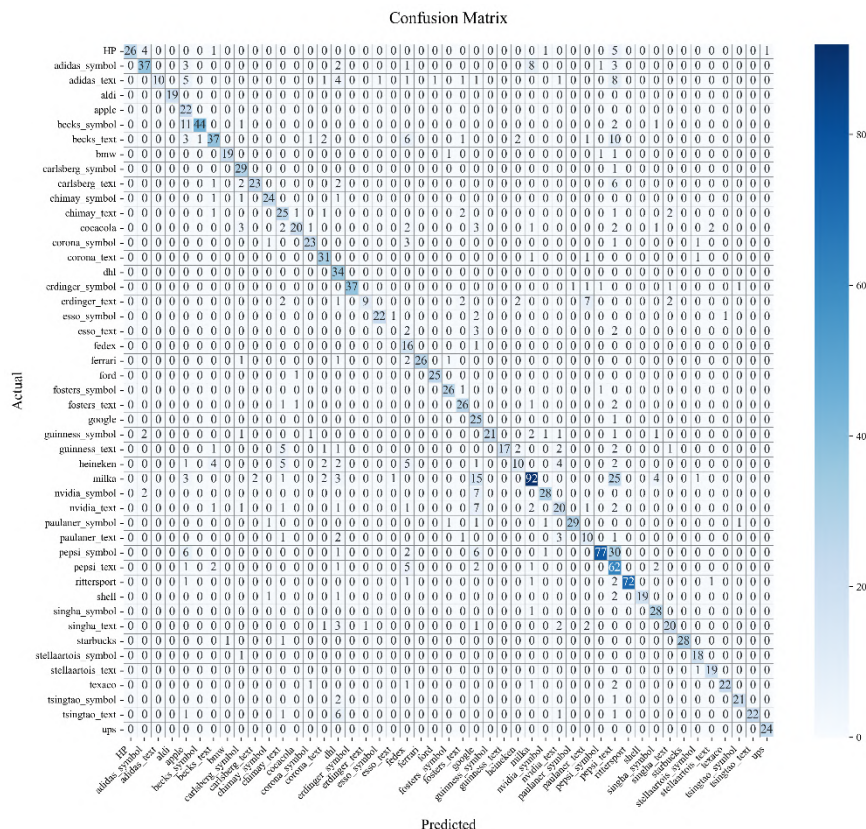
4.4.4 การทดลอง Fine-tuning classification model กับ FlickrLogos-47

หลังจากการทดลอง Hyperparameter tuning กับ CNN ทั้ง 5 ตัวกับ FlickrLogos-47 ภายใต้ Classification head เดียวกัน โดยเลือก 3 ตัวที่ดีที่สุดจากค่า Accuracy ด้วยเหตุผลนี้จึงเลือก EfficientNetV2, ConvNext และ MobileNetV3 มาทำการ Fine-tuning เพื่อปรับแต่ง Layer ของ Classification head ให้มีประสิทธิภาพสูงสุดต่อ FlickrLogos-47

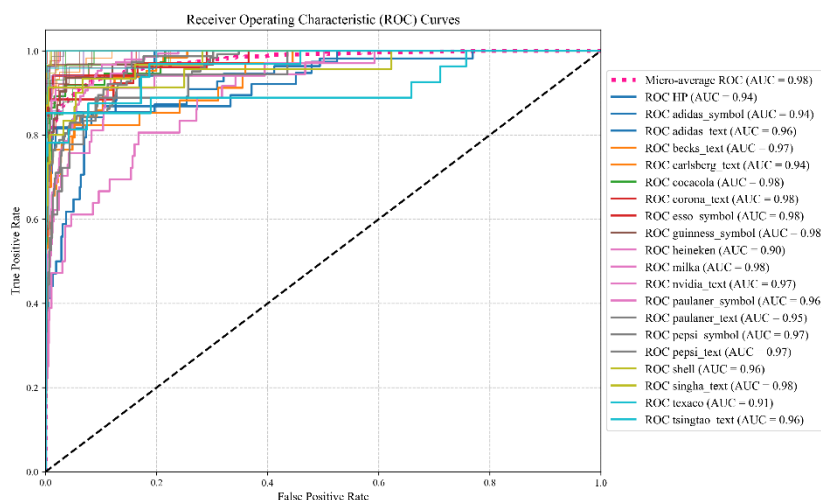
4.4.4.1 การทดลอง Fine-tuning EfficientNetV2 กับ FlickrLogos-47

ทำการ Fine-tuning MobileNetV3 กับ FlickrLogos-47 โดยมีการปรับแต่ง

Layer ในส่วน Classification head ซึ่งได้ค่า Accuracy เท่ากับ 0.8746 เมื่อทดสอบกับชุด Test



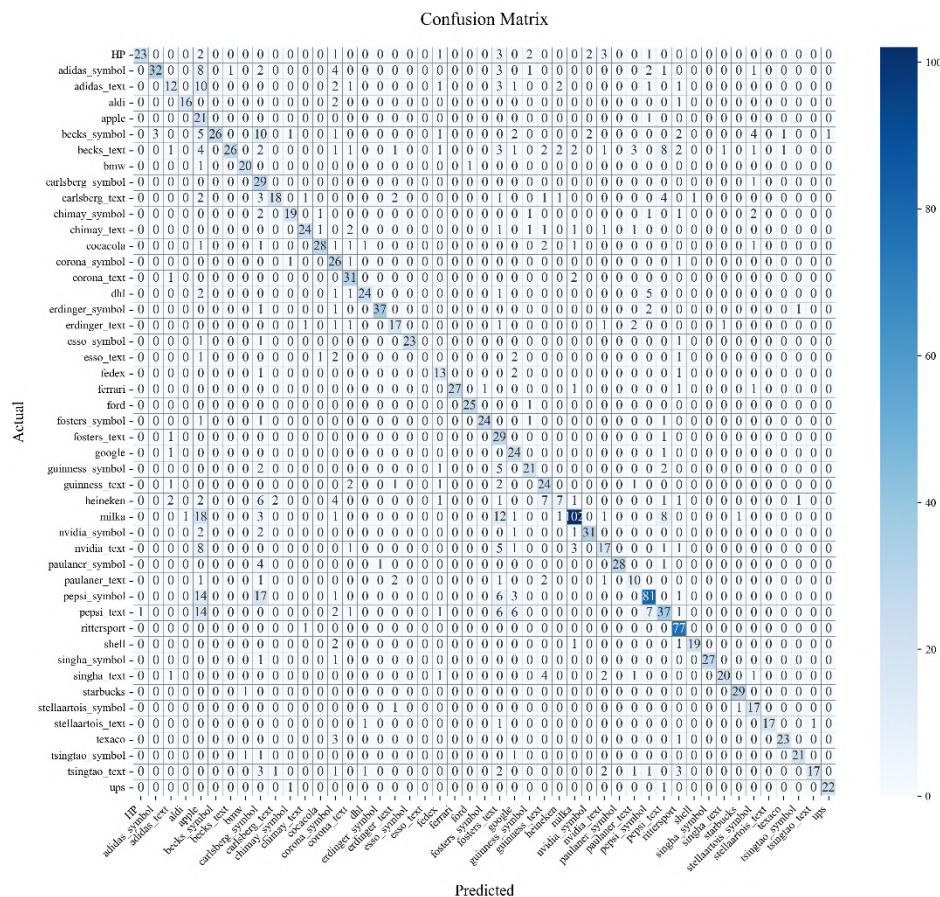
รูป 4.27 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ EfficientNetV2 กับ FlickrLogos-47



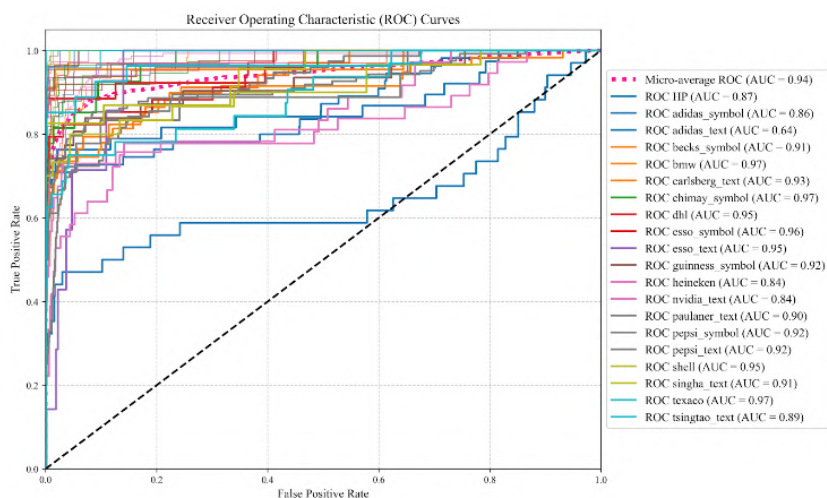
รูป 4.28 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ EfficientNetV2 กับ FlickrLogos-

4.4.4.2 การทดลอง Fine-tuning ConvNext กับ FlickrLogos-47

ทำการ Fine-tuning MobileNetV3 กับ FlickrLogos-47 โดยมีการปรับแต่ง Layer ในส่วน Classification head ซึ่งได้ค่า Accuracy เท่ากับ 0.8924 เมื่อทดสอบกับชุด Test



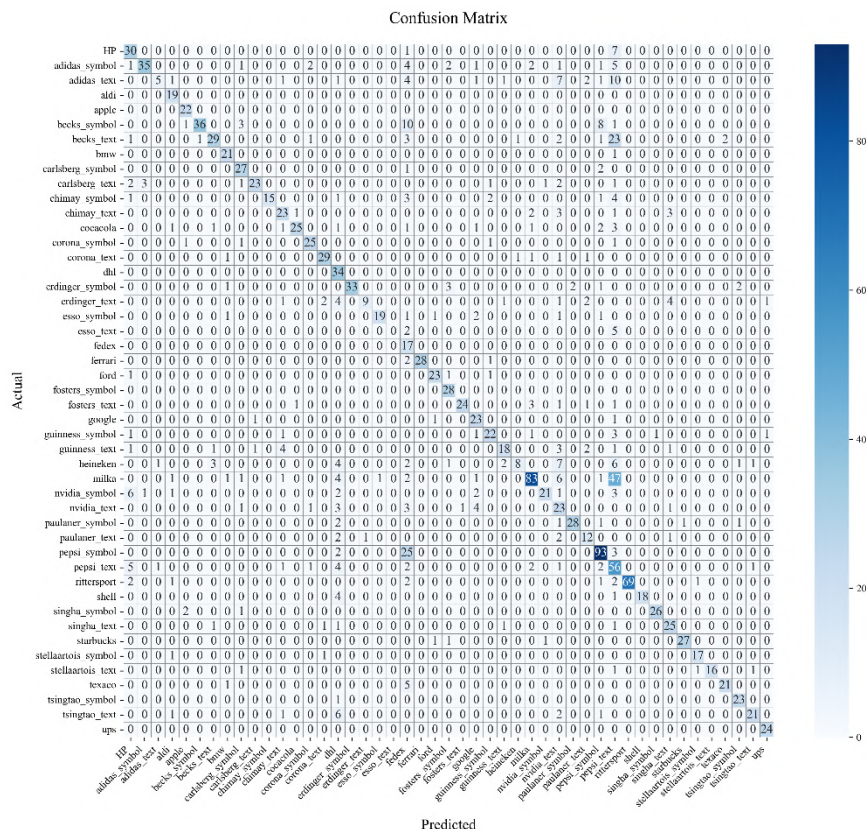
รูป 4.29 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ ConvNext กับ FlickrLogos-47



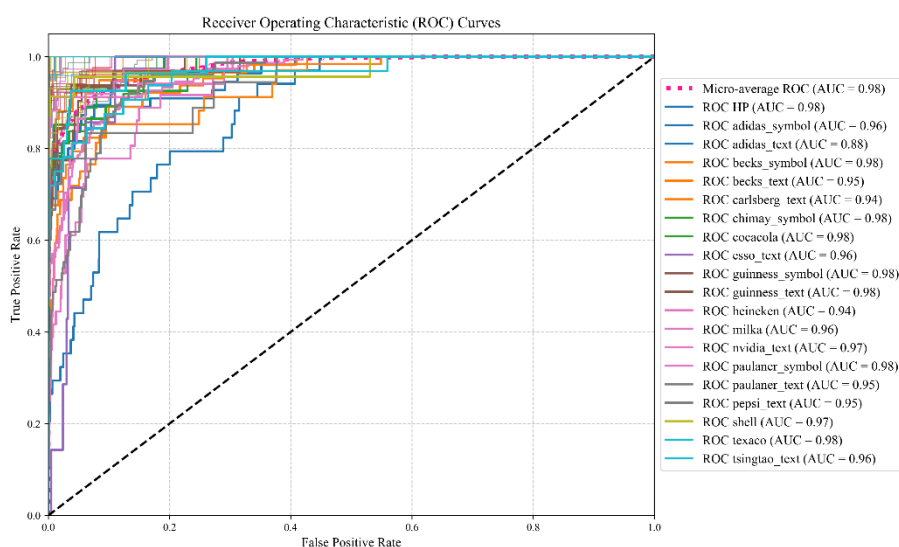
รูป 4.30 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ ConvNext กับ FlickrLogos-47

4.4.4.3 การทดลอง Fine-tuning MobileNetV3 กับ FlickrLogos-47

ทำการ Fine-tuning MobileNetV3 กับ FlickrLogos-47 โดยมีการปรับแต่ง Layer ในส่วน Classification head ซึ่งได้ค่า Accuracy เท่ากับ 0.8694 เมื่อทดสอบกับชุด Test



รูป 4.31 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ MobileNetV3 กับ FlickrLogos-47



รูป 4.32 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ MobileNetV3 กับ FlickrLogos-47

4.4.4 ผลการทดลอง Fine-tuning classification model กับ FlickrLogos-47

หลังจากทำการ Fine-tuning classification model กับ FlickrLogos-47 ทำให้ได้ค่า Accuracy เมื่อทดสอบกับชุด Test เพิ่มขึ้นประมาณ 0.01-0.02 โดยได้ค่า Accuracy ของแต่ละโมเดล ดังนี้

ตาราง 4.6 ผลการทดสอบ Fine-tuning classification model กับ FlickrLogos-47

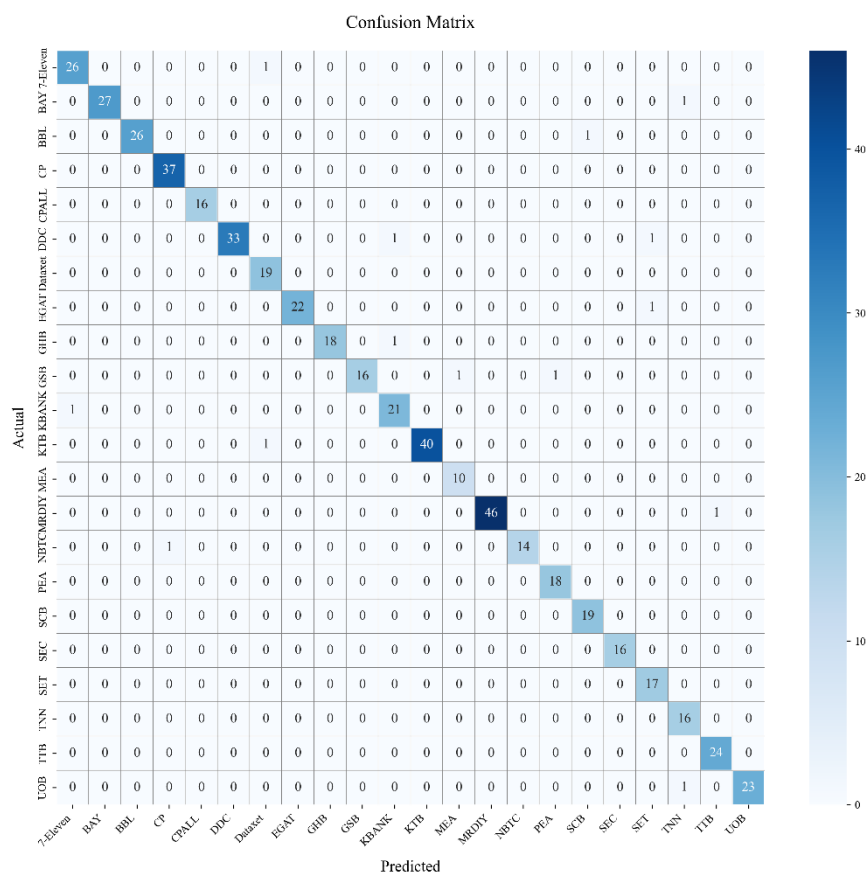
Dataset	Model	Param (M)	Test_Accuracy
FlickrLogos-47	EfficientNetV2	7.1	0.8746
	ConvNeXt	28	0.8924
	MobileNetV3	5.9	0.8694

จากตารางข้างต้น แสดงให้เห็นว่าในส่วนของ Pre-trained model ที่ทำการเลือกมา ConvNeXt ได้ค่า Accuracy เมื่อทดสอบกับชุด Test มากที่สุดอยู่ที่ 0.9652 ซึ่งมีจำนวนพารามิเตอร์มากถึง 28 ล้านพารามิเตอร์

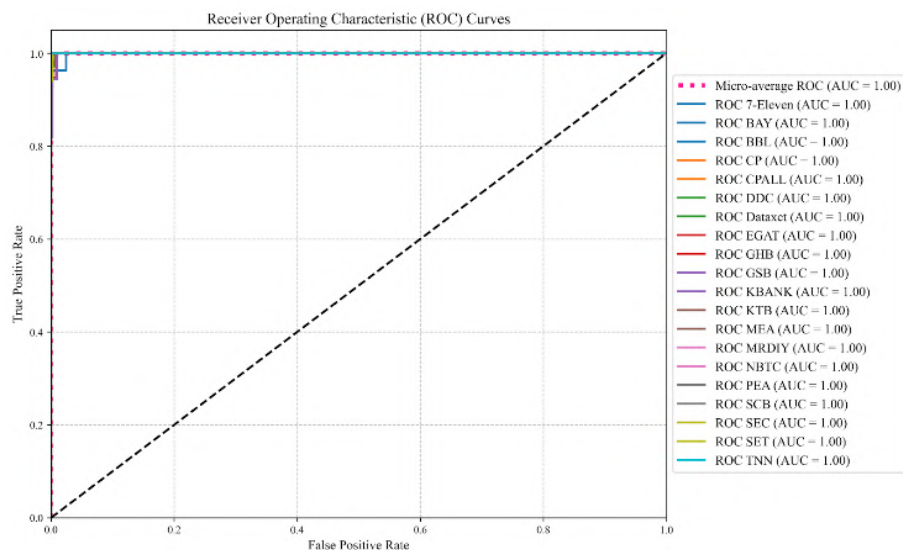
4.5 การทดลองเทรน Classification model โดยใช้ Custom CNN model

4.5.1 การทดลองเทรน Custom CNN model กับ 22-Class Thai Logos

ทำการเทรน Custom CNN ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 0.66 ล้านพารามิเตอร์กับ 22-Class Thai Logos ซึ่งได้ค่า Accuracy อยู่ที่ 0.9787 เมื่อทดสอบกับชุด Test



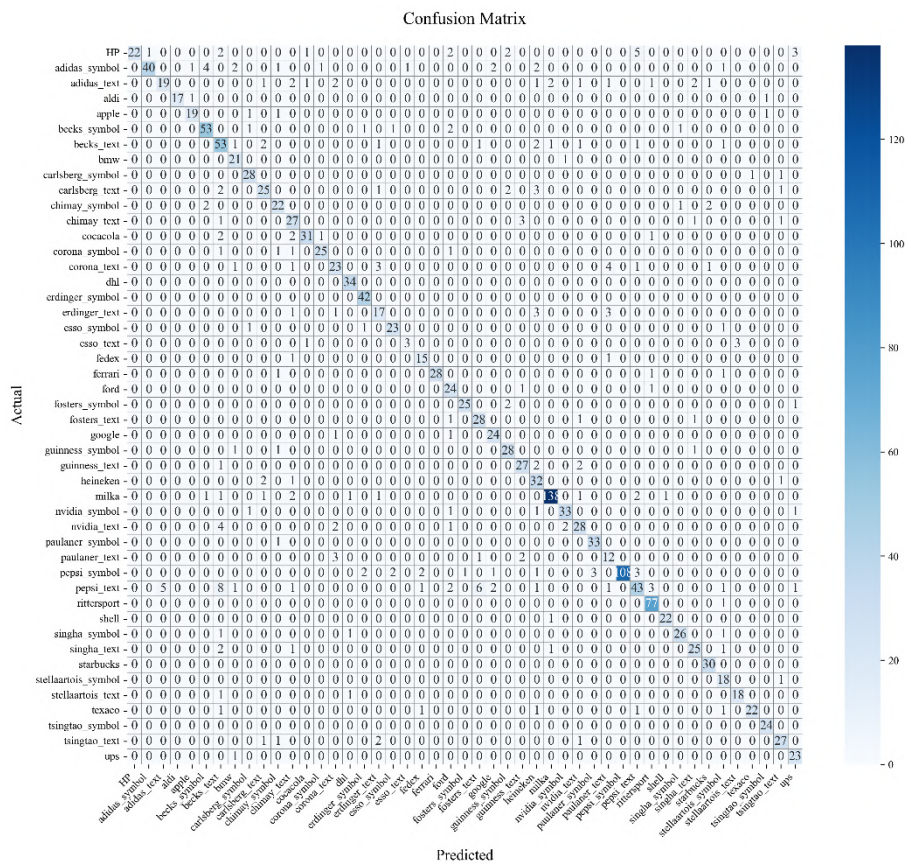
รูป 4.33 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ Custom CNN กับ 22-Class Thai Logos



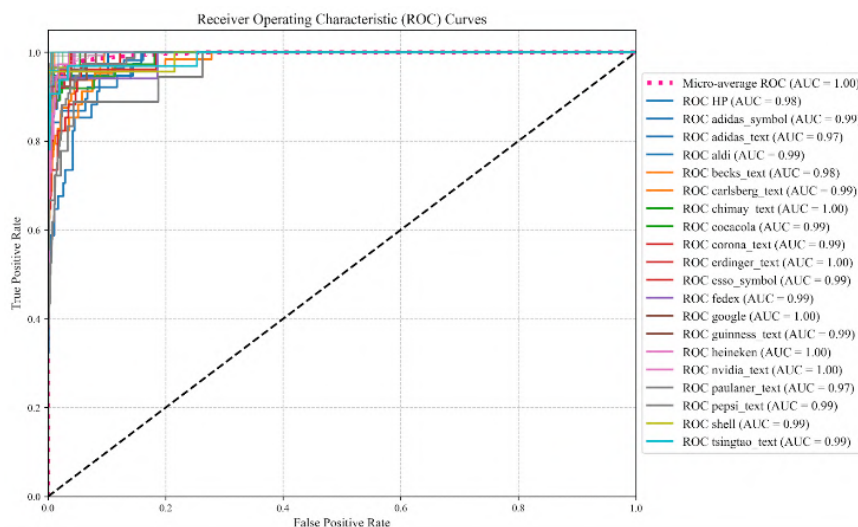
รูป 4.34 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ Custom CNN กับ 22-Class Thai Logos

4.5.2 การทดลองเทรน Custom CNN model กับ FlickrLogos-47

ทำการเทรน Custom CNN ซึ่งมีจำนวนพารามิเตอร์ทั้งหมด 0.66 ล้านพารามิเตอร์กับ FlickrLogos-47 ซึ่งได้ค่า Accuracy อยู่ที่ 0.8642 เมื่อทดสอบกับชุด Test



รูป 4.35 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ Custom CNN กับ FlickrLogos-47



รูป 4.36 กราฟแสดง ROC Curve การทำนายกับชุด Test ของ Custom CNN กับ FlickrLogos-47

4.5.3 การทดลองเทรน Custom CNN model

หลังจากการทดสอบ Custom CNN model ผ่านข้อมูลในชุด Test ได้ผลลัพธ์ออกมาดังนี้

ตาราง 4.7 ผลการทดสอบการเทรน Custom CNN กับ FlickrLogos-47 และ 22-Class Thai Logos

Model	Param (M)	Dataset	Test_Accuracy
Custom CNN	0.66	22-Class Thai Logos	0.9787
		FlickrLogos-47	0.8642

จากตารางข้างต้น แสดงให้เห็นถึงค่า Accuracy ของ Custom CNN model ต่อ Dataset ทั้งสอง ซึ่งสังเกตได้ว่า Custom CNN model ได้ค่า Accuracy เมื่อทดสอบกับ 22-Class Thai Logos ในชุด Test สูงถึง 0.9787 ซึ่งมีค่ามากกว่า Pre-trained model หลังจากทำการ Fine tune และได้พารามิเตอร์ที่น้อยกว่า Pre-trained model หลังจากทำการ Fine tune เช่นกัน และเมื่อทดสอบกับ FlickrLogos-47 ในชุด Test ได้ค่า Accuracy ใกล้เคียง Pre-trained model แต่ใช้พารามิเตอร์เพียง 0.66 ล้านพารามิเตอร์

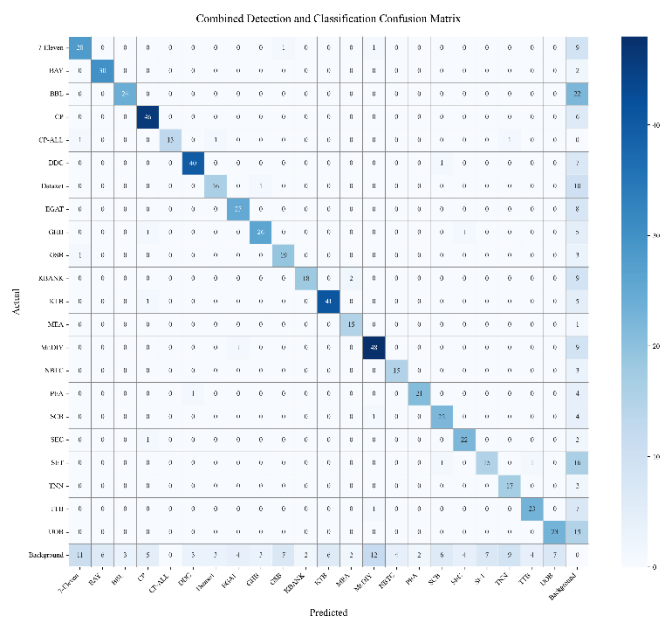
4.6 การทดสอบประสิทธิภาพโดยรวมของ Detection model และ Classification model

หลังจากที่มีการเทรน Detection model และ Classification model ทั้งหมด ทางคณะผู้จัดทำจึงได้ทำการวัดประสิทธิภาพโดยรวมผ่านข้อมูลชุด Test โดยได้เลือก YOLOv8 มาเป็นโมเดลหลักสำหรับการตรวจจับโลโก้ เพื่อเปรียบเทียบประสิทธิภาพของ Classification model สำหรับการจำแนกประเภทของโลโก้

4.6.1 การทดสอบประสิทธิภาพของ Classification model กับ 22-Class Thai Logos เมื่อผ่าน YOLOv8

4.6.1.1 EfficientNetV2

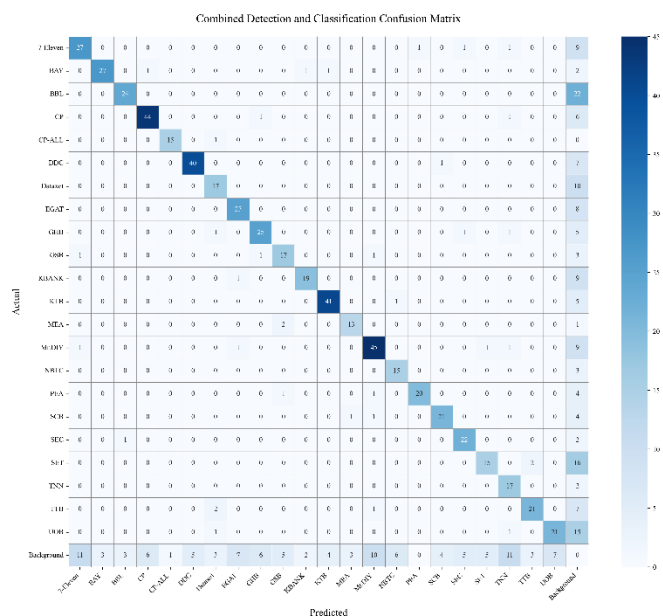
ทำการทดสอบโดยใช้ข้อมูลชุด Test กับ YOLOv8 ซึ่งได้ค่า mAP50 อยู่ที่ 0.8516 ค่าความล่าช้าอยู่ที่ 29 มิลลิวินาทีเมื่อผ่าน YOLOv8 และมีค่า F1 score เท่ากับ 0.8140 เมื่อผ่าน YOLOv8 จะนำ Object ที่เป็น True positive มาผ่าน EfficientNetV2 และได้ค่า Accuracy เท่ากับ 0.9647 ซึ่งมีความล่าช้าอยู่ที่ 65 มิลลิวินาทีเมื่อผ่าน EfficientNetV2



รูป 4.37 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ EfficientNetV2 กับ 22-Class Thai Logos

4.6.1.2 InceptionV3

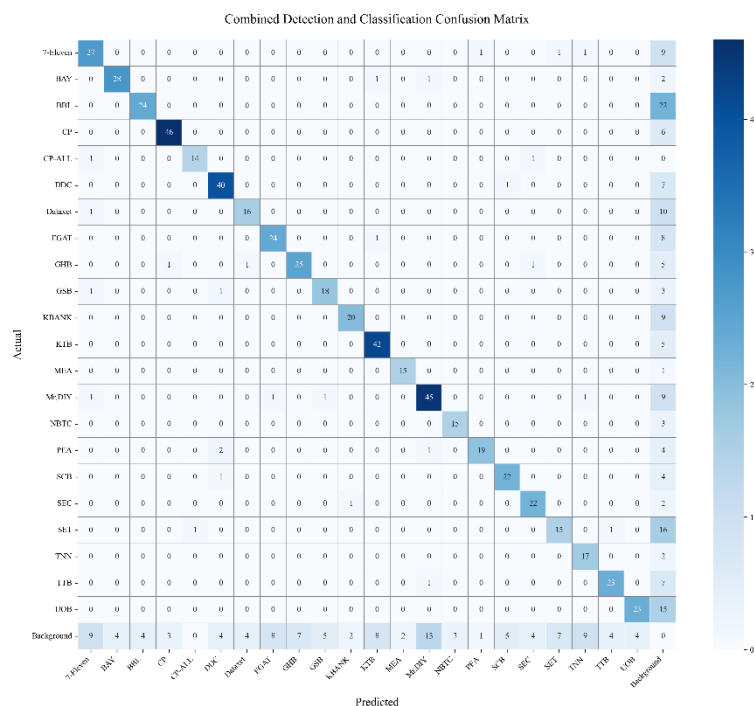
ทำการทดสอบโดยใช้ข้อมูลชุด Test กับ YOLOv8 ซึ่งได้ค่า mAP50 อยู่ที่ 0.8516 ค่าความล่าช้าอยู่ที่ 29 มิลลิวินาทีเมื่อผ่าน YOLOv8 และมีค่า F1 score เท่ากับ 0.8140 เมื่อผ่าน YOLOv8 จะนำ Object ที่เป็น True positive มาผ่าน InceptionV3 และได้ค่า Accuracy เท่ากับ 0.9365 ซึ่งมีความล่าช้าอยู่ที่ 83 มิลลิวินาทีเมื่อผ่าน InceptionV3



รูป 4.38 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ InceptionV3 กับ 22-Class Thai Logos

4.6.1.3 MobileNetV3

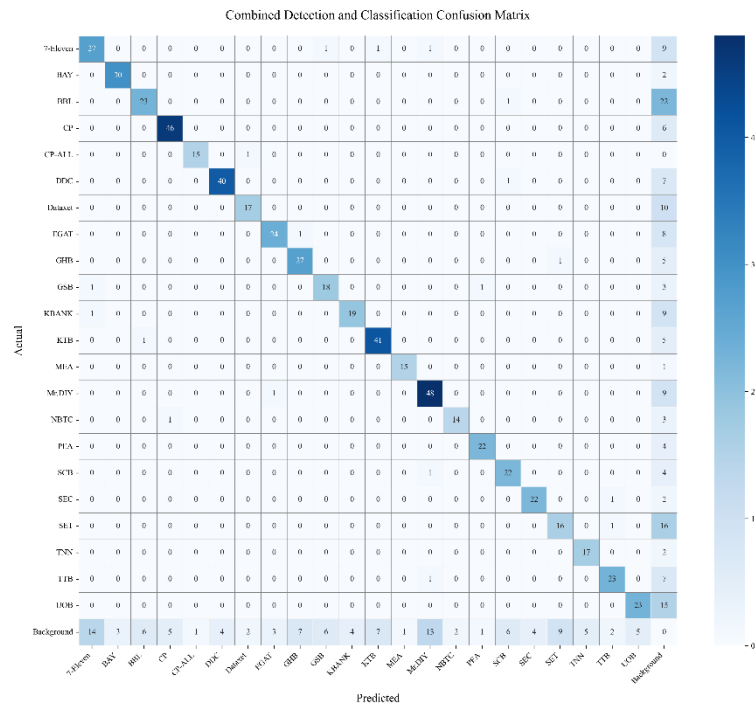
ทำการทดสอบโดยใช้ข้อมูลชุด Test กับ YOLOv8 ซึ่งได้ค่า mAP50 อยู่ที่ 0.8516 ค่าความล่าช้าอยู่ที่ 29 มิลลิวินาทีเมื่อผ่าน YOLOv8 และมีค่า F1 score เท่ากับ 0.8140 เมื่อผ่าน YOLOv8 จะนำ Object ที่เป็น True positive มาผ่าน MobileNetV3 และได้ค่า Accuracy เท่ากับ 0.9524 ซึ่งมีความล่าช้าอยู่ที่ 56 มิลลิวินาทีเมื่อผ่าน MobileNetV3



รูป 4.39 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ MobileNetV3 กับ 22-Class Thai Logos

4.6.1.4 Custom CNN

ทำการทดสอบโดยใช้ข้อมูลชุด Test กับ YOLOv8 ซึ่งได้ค่า mAP50 อยู่ที่ 0.8516 ค่าความล่าช้าอยู่ที่ 29 มิลลิวินาทีเมื่อผ่าน YOLOv8 และมีค่า F1 score เท่ากับ 0.8140 เมื่อผ่าน YOLOv8 จะนำ Object ที่เป็น True positive มาผ่าน Custom CNN และได้ค่า Accuracy เท่ากับ 0.9683 ซึ่งมีความล่าช้าอยู่ที่ 41 มิลลิวินาทีเมื่อผ่าน Custom CNN



รูป 4.40 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ Custom CNN กับ 22-Class Thai Logos

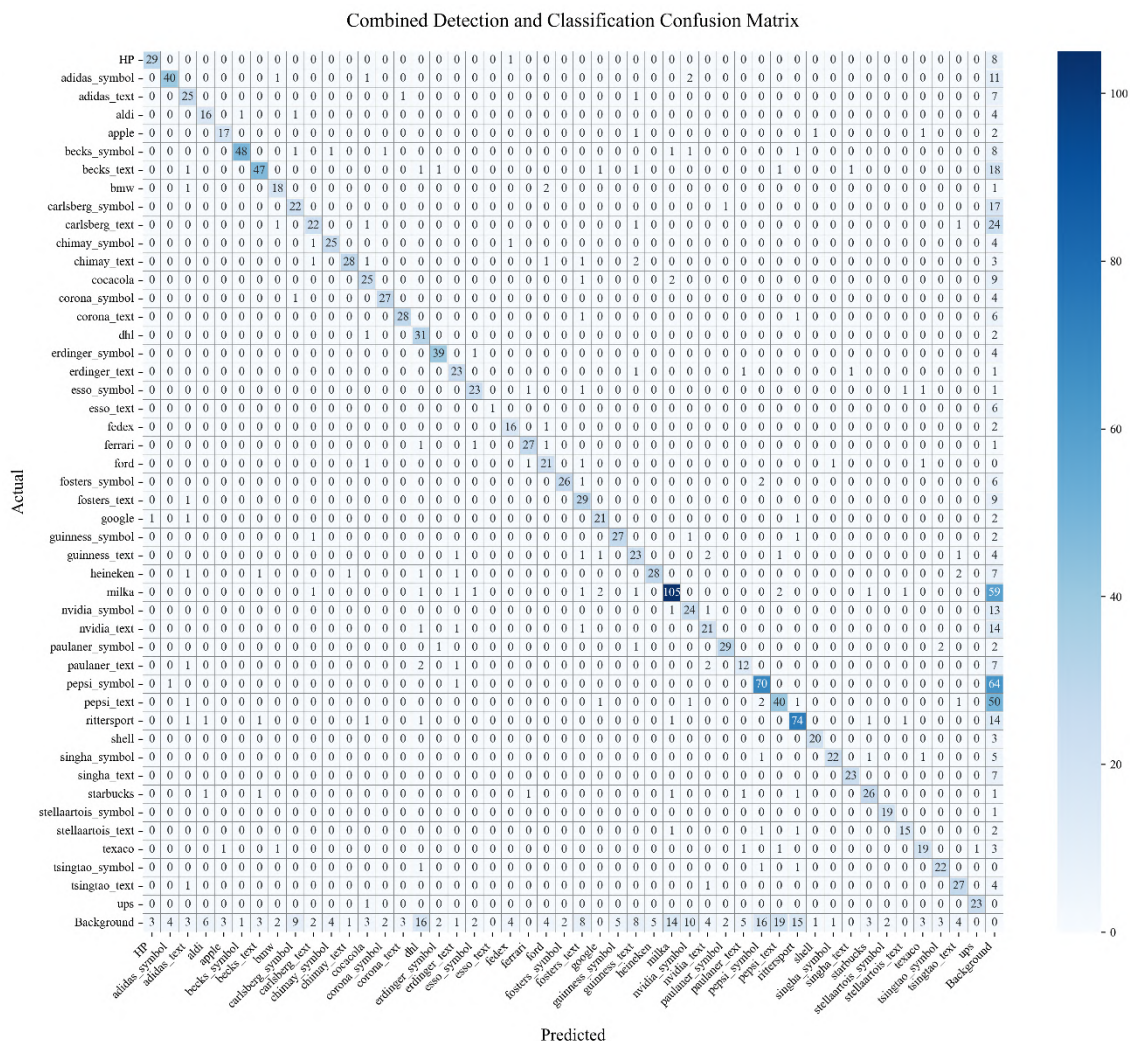
4.6.1.5 ผลการทดสอบประสิทธิภาพของ Classification model กับ 22-Class Thai Logos เมื่อผ่าน YOLOv8

ทดสอบประสิทธิภาพของ YOLOv8 และ Classification model กับ 22-Class Thai Logos ได้ผลลัพธ์ออกมา ดังนี้

ตาราง 4.8 ผลการทดสอบประสิทธิภาพของ Classification model กับ 22-Class Thai Logos เมื่อผ่าน YOLOv8

Detection Model	Classification Model	Classification Runtime (ms)	Precision	Recall	F1 score	Test Accuracy
YOLOv8	EfficientNetV2	65	0.9659	0.9647	0.9645	0.9647
	InceptionV3	83	0.9402	0.9365	0.9369	0.9365
	MobileNetV3	56	0.9532	0.9523	0.9521	0.9524
	Custom CNN	41	0.9687	0.9682	0.9682	0.9683

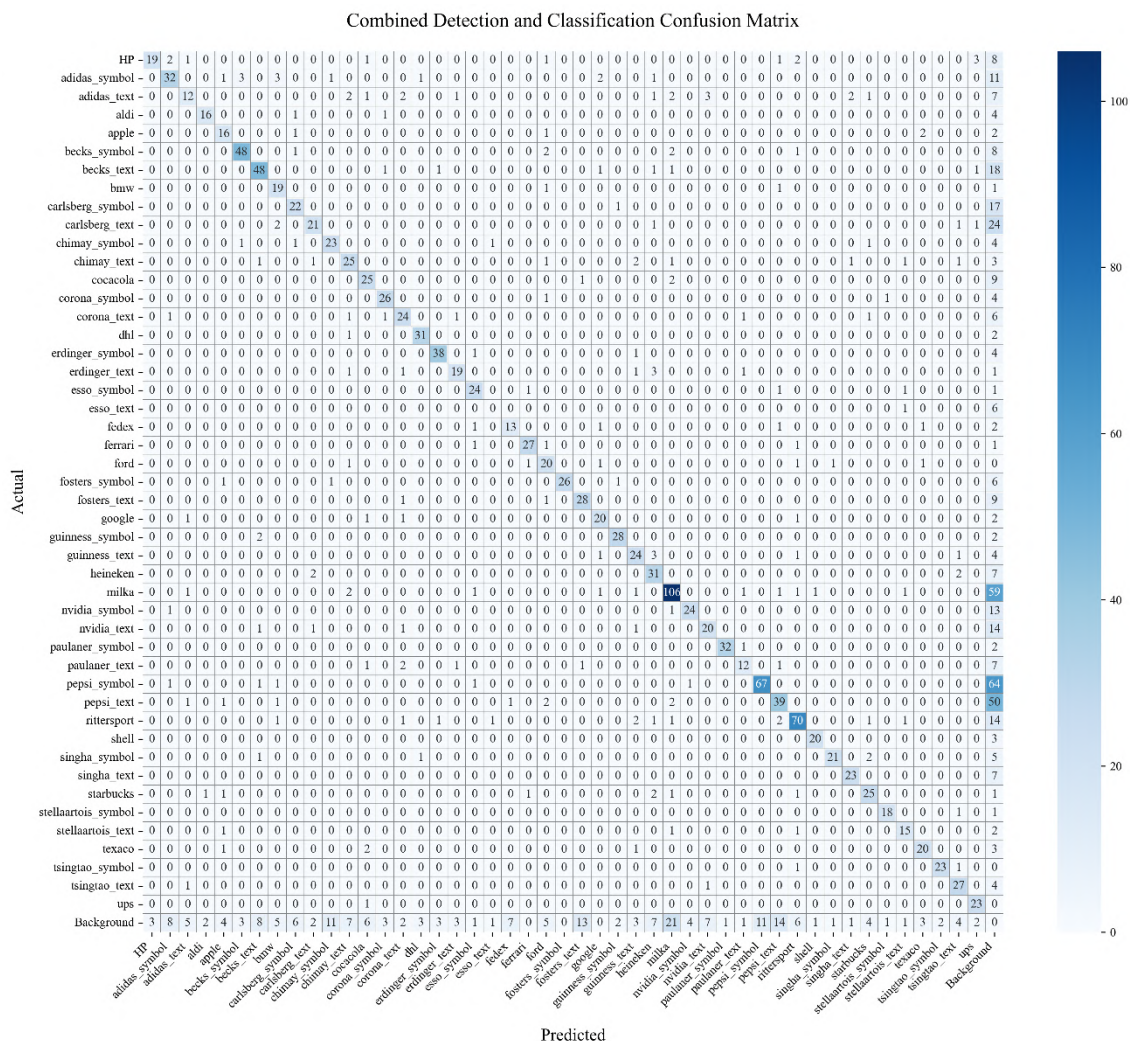
จากตารางข้างต้น แสดงให้เห็นว่า เมื่อใช้ YOLOv8 เป็น Detection model จะได้ค่า mAP50 อยู่ที่ 0.8516 ซึ่งความล่าช้าอยู่ที่ 29 มิลลิวินาที และใช้ Custom CNN เป็น



รูป 4.43 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ MobileNetV3 กับ FlickrLogos-47

4.6.2.4 Custom CNN

ทำการทดสอบโดยใช้ข้อมูลชุด Test กับ YOLOv8 ซึ่งได้ค่า mAP50 อยู่ที่ 0.835 ค่าความล่าช้าอยู่ที่ 27 มิลลิวินาทีเมื่อผ่าน YOLOv8 และมีค่า F1 score เท่ากับ 0.8605 เมื่อผ่าน YOLOv8 จะนำ Object ที่เป็น True positive มาผ่าน Custom CNN และได้ค่า Accuracy เท่ากับ 0.8605 ซึ่งมีความล่าช้าอยู่ที่ 41 มิลลิวินาทีเมื่อผ่าน Custom CNN



รูป 4.44 กราฟแสดง Confusion matrix การทำนายกับชุด Test ของ YOLOv8 และ Custom CNN กับ FlickrLogos-47

4.6.2.5 ผลการทดสอบประสิทธิภาพของ Classification model กับ FlickrLogos-47 เมื่อผ่าน YOLOv8

ทดสอบประสิทธิภาพของ YOLOv8 และ Classification model กับ FlickrLogos-47 ได้ผลลัพธ์ออกมา ดังนี้

ตาราง 4.9 ผลการทดสอบประสิทธิภาพของ Classification model กับ FlickrLogos-47 เมื่อผ่าน

YOLOv8

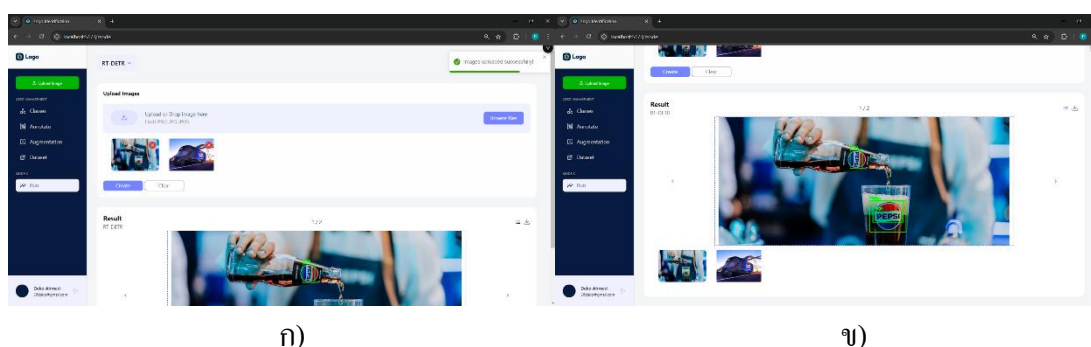
Detection Model	Classification Model	Classification Runtime (ms)	Precision	Recall	F1 score	Test Accuracy
YOLOv8	EfficientNetV2	65	0.8650	0.8605	0.8603	0.8605
	ConvNeXt	105	0.9012	0.8959	0.8965	0.8959
	MobileNetV3	56	0.8590	0.8525	0.8533	0.8525
	Custom CNN	41	0.8662	0.8605	0.8603	0.8605

จากตารางข้างต้น แสดงให้เห็นว่า เมื่อใช้ YOLOv8 เป็น Detection model จะได้ค่า mAP50 อยู่ที่ 0.8516 ซึ่งความล่าช้าอยู่ที่ 29 มิลลิวินาที และใช้ ConvNeXt เป็น Classification model ทำการทดสอบโดยใช้ 22-Class Thai Logos ชุด Test ได้ค่า Accuracy ออกมาดีที่สุดในที่ 0.8959 และมีความล่าช้าในการจำแนกมากที่สุดซึ่งอยู่ที่ 83 มิลลิวินาทีซึ่งมากกว่า Custom CNN ถึง 2 เท่า ซึ่ง Custom CNN ได้ค่า Accuracy ออกมาอยู่ที่ 0.8605 ซึ่งเทียบเท่า EfficientNetV2 แต่มีจำนวนพารามิเตอร์ที่น้อยกว่า

4.7 การทดสอบเว็บแอปพลิเคชัน

4.7.1 การทดสอบระบบการทำนายผลจากโมเดล

สำหรับการเรียกใช้โมเดลผ่านหน้าเว็บแอปพลิเคชัน พบว่าสามารถเรียกใช้โมเดลได้อย่างถูกต้อง อีกทั้งยังสามารถแสดงผลที่ได้จากการทำนายได้ ดังนี้



รูป 4.45 หน้าทำนายผลจากโมเดล

- ก) อัปโหลดภาพเพื่อทำนายผล
- ข) ผลลัพธ์ที่ได้จากโมเดล

4.7.2 การทดสอบการใช้งาน API

ทำการทดสอบฟังก์ชันการทำงานระหว่าง Frontend และ Backend ทำให้เห็นถึงผลลัพธ์ของ API แต่ละเส้น โดยแบ่ง API ในการทดสอบออกเป็น 2 ส่วนได้แก่ ระบบการจัดการโลโก้ (Logo Management System) และระบบการทำนายผล (Logo Prediction System)

ตาราง 4.10 ผลการทดสอบระบบ Logo Management

Method	API	Description
POST	/api/images	อัปโหลดรูปภาพไปยัง MinIO
GET	/api/images	ดึงข้อมูลรูปภาพทั้งหมด
DELETE	/api/images	ลบรูปใน Bucket ทั้งหมด
GET	/api/class	ดึงข้อมูลคลาสที่มีอยู่ทั้งหมด
POST	/api/class	สร้างคลาสใหม่
DELETE	/api/class	ลบคลาสเฉพาะที่กำหนด
GET	/api/annotated-images	ดึงข้อมูลรูปภาพที่ Annotated
POST	/api/annotated-images	สร้างหรือแก้ไขรูปภาพที่ Annotated
GET	/api/dataset	ดึงข้อมูลจำนวนรูปภาพและจำนวนคลาสของ Dataset
GET	/api/dataset/download	ดาวน์โหลดชุดข้อมูลในรูปแบบ ZIP file

ตาราง 4.11 ผลการทดสอบระบบ Logo Management

Method	API	Description
POST	/api/model/run-realtime	อัปโหลดรูปภาพเพื่อทำนายผลและรับข้อมูลกลับเป็นรูปภาพที่โมเดลทำนายผล
GET	/api/scrape?keyword={keyword}&start_date={start_date}&end_date={end_date}	ดึงข้อมูลรูปภาพจากโซเชียลมีเดีย ได้แก่ x.com ตามพารามิเตอร์ที่กำหนด และภาพที่ได้เป็นการทำนายผลจากโมเดล

บทที่ 5

บทสรุปและข้อเสนอแนะ

5.1 สรุปผลการพัฒนา

คณะผู้จัดทำได้ทำการพัฒนาโมเดล Machine Learning เพื่อใช้สำหรับการตรวจจับและจำแนกโลโก้จากภาพ โดยใช้ชุดข้อมูลสาธารณะได้แก่ FlickrLogos-47 และชุดข้อมูลที่ทำกรเก็บรวบรวมขึ้นมา ได้แก่ 22-Class Thai Logos โดยนำชุดข้อมูลดังกล่าวมาทำการเทรนกับโมเดล Logo Detection ทั้งหมด 3 โมเดล ได้แก่ YOLOv8, YOLOv10, และ RT-DETR ซึ่งได้ทำการวัดประสิทธิภาพความแม่นยำจากค่า mAP50 (Mean Average Precision) และวัดประสิทธิภาพด้านเวลาโดยการเปรียบเทียบเวลาที่ใช้ในการประมวลผลของรูปภาพ 1 รูป ในโมเดลนั้น ๆ จากนั้นจึงทำการเทรนโมเดล Logo Classification ซึ่งเป็นโมเดล Pre-Trained 5 โมเดล ได้แก่ EfficientNetV2, InceptionV3, InceptionResNetV2, ConvNeXt, MobileNetV3, และ Custom CNN ที่ทางคณะผู้จัดทำได้ทำการพัฒนาขึ้น โดยทำการวัดประสิทธิภาพความแม่นยำจากค่า Accuracy และวัดประสิทธิภาพด้านเวลาจากการเปรียบเทียบเวลาที่ใช้ในการประมวลผลของโลโก้ 1 โลโก้ในรูปของแต่ละโมเดล

ผลการทดสอบ Detection Model ในชุดข้อมูลทั้งสองชุดข้อมูลนั้นมีขนาดและวิธีการเตรียมข้อมูลต่างกัน แต่จะได้ว่าทั้งในชุดข้อมูล FlickrLogos-47 และในชุดข้อมูล 22-Class Thai Logos โมเดลที่มีประสิทธิภาพดีที่สุดคือ YOLOv8 โดยพิจารณาจากประสิทธิภาพโดยรวมของความแม่นยำและความเร็วในการประมวลผล

ผลการทดสอบ Classification Model ในชุดข้อมูล FlickrLogos-47 ได้โมเดลที่ดีที่สุด 4 โมเดล ได้แก่ ConvNeXt, Custom CNN, EfficientNetV2, และ MobileNetV3 ตามลำดับ ซึ่ง FlickrLogos-47 เป็นชุดข้อมูลที่มีขนาดกลางทำให้ ConvNeXt มีประสิทธิภาพในด้านความแม่นยำมากที่สุดซึ่งได้ค่า Accuracy ที่ 0.8959 เนื่องจากเป็นสถาปัตยกรรมที่ต้องการข้อมูลจำนวนมากแต่แลกมากับขนาดของโมเดลที่ค่อนข้างใหญ่และใช้เวลาในการทำนายค่อนข้างมากเช่นกัน แต่หากพิจารณาจากประสิทธิภาพโดยรวม พบว่า Custom CNN มีประสิทธิภาพในการทำ Logo Classification ได้ดีเทียบเท่า EfficientNetV2 ซึ่งได้ Accuracy ที่ 0.8605 แต่มีจำนวนพารามิเตอร์ที่น้อยกว่าถึง 12 เท่า ในส่วนของชุดข้อมูล 22-Class Thai Logos ได้โมเดลที่ดีที่สุด 4 โมเดล ได้แก่ Custom CNN, MobileNetV3, EfficientNetV2, และ InceptionV3 ตามลำดับ โดยโมเดลที่มีประสิทธิภาพดีที่สุดคือ Custom CNN เมื่อพิจารณาจากทั้งความแม่นยำที่ให้ Accuracy สูงที่สุดที่ 0.9682 และในด้านขนาดที่เล็กกว่ามากเมื่อเทียบกับโมเดลอื่น ๆ

นอกจากนี้ เพื่อเพิ่มความสะดวกของโครงงาน ทางคณะผู้จัดทำจึงได้พัฒนาเว็บแอปพลิเคชันที่สามารถจำลองการทำ Social Monitoring ผ่านฟีดเจอร์ Scraping Data ทำให้ผู้ใช้สามารถตรวจสอบโล

โลโก้ที่อยู่ในรูปจากคีเวิร์ดที่ได้ทำการค้นหา ผ่านโมเดล Logo Detection และโมเดล Logo Classification ที่ทางคณะผู้จัดทำได้พัฒนาขึ้น อีกทั้งยังสามารถทดสอบการใช้งานทั้งหมดที่กล่าวมาได้อย่างสะดวกผ่านอินเทอร์เน็ต

5.2 ปัญหาและแนวทางการแก้ไข

- 1) การหา Weight ที่เหมาะสมที่สุดของโมเดล ผู้พัฒนาได้ใช้วิธีตั้งค่า Early Stop แต่จากการศึกษาเพิ่มเติมพบว่า ใน Epoch ที่เกิด Early Stop นั้น ค่า Validation Loss และ Training Loss ยังไม่เกิดจุดตัดกัน แสดงให้เห็นว่าโมเดลยังสามารถพัฒนาประสิทธิภาพต่อไปได้ เนื่องจากการทำ Early Stop ของ Library มีการพิจารณาค่า Loss 3 ประเภท ได้แก่ Box Loss, Class Loss, และ DFL Loss ซึ่งมีค่า Weight ที่ไม่สามารถปรับแก้ได้ ผู้พัฒนาจึงตัดสินใจไม่นำ DFL Loss มาพิจารณา เนื่องจากไม่ใช่ Metrics หลักในการประเมินผล
- 2) การวัดผลประสิทธิภาพของกระบวนการทำ Logo Identification ซึ่งมีทั้งส่วนของการทำงาน Logo Detection และ Logo Classification ไม่สามารถวัดผลทั้งสองกระบวนการรวมกันได้ อย่างชัดเจน จึงทำการแยกวัดประสิทธิภาพในแต่ละกระบวนการ โดยใช้ mAP50 ในการวัดประสิทธิภาพของ Logo Detection และใช้ Accuracy ในการวัดประสิทธิภาพของ Logo Classification
- 3) ชุดข้อมูล 22-Class Thai Logos มีจำนวนรูปภาพที่น้อยเมื่อเทียบกับ FlickrLogos-47 ทำให้บางโมเดลไม่สามารถเรียนรู้ได้ดีพอ จึงได้มีการ Augmentation เพื่อเพิ่มจำนวนภาพในชุดข้อมูล 22-Class Thai Logos

5.3 แนวทางการพัฒนาต่อ

เนื่องจากคณะผู้จัดทำมีความประสงค์ให้นำโมเดล Logo Detection ไปใช้งาน Social Monitoring หมายความว่า Logo Detection จะเป็นโมเดลที่เรียนรู้ลักษณะ (Feature) ของโลโก้ทั่วไป (Generic Logo) ซึ่งจำเป็นต้องใช้ข้อมูลจำนวนมากในการเทรนโมเดลเพื่อให้โมเดลสามารถแยกแยะโลโก้จากรูปภาพได้อย่างชัดเจนและแม่นยำ ดังนั้นหากต้องการที่จะพัฒนา Logo Detection อาจมีการนำ Dataset ที่เกี่ยวกับโลโก้ทั้งในรูปแบบของ Dataset ที่เปิดสาธารณะและ Dataset ที่เก็บรวบรวมเอง มาสร้างเป็น Dataset ที่ครอบคลุมกับโลโก้ที่ปรากฏอยู่ในโลกปัจจุบันให้ได้มากที่สุด

จากผลทดสอบของโมเดล Custom CNN ที่เป็นโมเดล Classification มีประสิทธิภาพสูงเมื่อเทียบกับขนาดที่เล็ก แต่ในโมเดล Detection ยังมีประสิทธิภาพไม่ดีพอ ในกรณีกรณีที่โมเดล Detection ตรวจจับโลโก้ไม่เจอถึงแม้จะเป็นรูปที่มีโลโก้ปรากฏอยู่ ส่งผลให้โมเดล Classification ไม่ได้จำแนกโลโก้ที่ควรจะถูกจำแนก ซึ่งทำให้ค่า True Positive ลดลงจากที่ควรจะเป็น โดยในการพัฒนาโมเดล Logo Detection ให้ดีขึ้นนั้นควรทำการรวบรวมชุดข้อมูลเพิ่มเติม เพื่อให้ชุดข้อมูลมี

ขนาดที่ใหญ่ขึ้นจะทำให้โมเดลเกิดการเรียนรู้ที่มากขึ้น และควรทำการค้นคว้างานวิจัยเพิ่มเติมเกี่ยวกับโมเดล Computer Vision ที่ถูกพัฒนาขึ้นมาใหม่และนำมาปรับใช้กับการทำ Logo Detection

บรรณานุกรม

- Sujuan H., Jiacheng L., Weiqing M., Qiang H., ..., Yanna Z. 2023. **Deep Learning for Logo Detection: A Survey**. [Online].Available : <https://dl.acm.org/doi/10.1145/3611309>
- Qimao Y., Huili C., & Qiwei D. 2023. **Comparative Analysis of Deep Learning Models for Brand Logo Classification in Real-World Scenarios**. [Online].Available : <https://arxiv.org/abs/2305.12242>
- Mark H., & Suren K. 2023. **Image-Text Pre-Training for Logo Recognition**. [Online].Available : https://openaccess.thecvf.com/content/WACV2023/papers/Hubenthal_Image-Text_Pre-Training_for_Logo_Recognition_WACV_2023_paper.pdf
- Wenyu L., Yian Z., Shangliang X., Jinman W., Guanzhong W., Cheng C., ..., & Yi L. 2023. **DETRs Beat YOLOs on Real-time Object Detection**. [Online].Available : <https://arxiv.org/abs/2304.08069>
- Ao W., Hui C., Lihao L., Kai C., Zijia L., Lungong H., &Guiguang D. 2024. **YOLOv10: Real-Time End-to-End Object Detection**. [Online].Available : <https://arxiv.org/pdf/2405.14458>
- Muhammad Hussain. 2024. **YOLOV5, YOLOV8 AND YOLOV10: THE GO-TO DETECTORS FOR REAL-TIME VISION**. [Online].Available : https://www.researchgate.net/publication/381961120_YOLOv5_YOLOv8_and_YOLOv10_The_Go-To_Detectors_for_Real-time_Vision
- Natthawat Phongchit. 2018. **Convolutional Neural Network (CNN) คืออะไร**. [Online].Available : <https://medium.com/@natthawatphongchit/มาลองดูวิธีการคิดของ-cnn-กัน-e3f5d73eebaa>
- Niklas Donges. 2022. **A Comprehensive Guide to Transfer Learning in Data Science**. [Online].Available : <https://builtin.com/data-science/transfer-learning>

Kanyawee Pornsawangdee. 2021. **ทำความเข้าใจ Transformer Architecture แบบง่ายๆกันเถอะ.**

[Online].Available : <https://kamkanyawee.medium.com/transformer-architecture-39cce176a2d6>

Panaya Sudta. 2023. **นักการตลาดกับ AI: Confusion Matrix และหลักการประเมินประสิทธิภาพ**

ML Model. [Online].Available : <https://www.everydaymarketing.co/business-and-marketing-case-study/ai/confusion-matrix-ml-evaluation/>

Andrian Rosebrock. 2016. **Intersection over Union (IoU) for object detection.**

[Online].Available : <https://pyimagesearch.com/2016/11/07/intersection-over-union-iou-for-object-detection/>

Everingham M., Van Gool L., Williams C., Winn J., & Zisserman A. 2010. **The Pascal Visual Object Classes (VOC) Challenge.** International Journal of Computer Vision, 88(2), 303–338. [Online].Available : <https://doi.org/10.1007/s11263-009-0275-4>

Seachaos. 2021. **Get Heatmap from CNN (Convolution Neural Network), AKA CAM.**

[Online].Available : <https://tree.rocks/get-heatmap-from-cnn-convolution-neural-network-aka-grad-cam-222e08f57a34>

ป๊อิ่ง แซ่อึ้ง. 2020. **Class Activation Map ใ้เดียวในการแกะรอยว่าบริเวณไหนในภาพที่ส่งผลต่อคำตอบใน Neural Network.** [Online].Available : <https://beeying.medium.com/class-activation-map-ใ้เดียวในการแกะรอยว่าบริเวณไหนในภาพที่ส่งผลต่อคำตอบใน-neural-network-d1a2b44e7aca>

The Neural Engineer. 2019. **มาฝึกทำ Image Augmentation กันเถอะ — AI for Eyes #2 —**

ตรวจสอบเบ้าหวานในดวงตา ภาค2. [Online].Available : <https://thaikeras.com/2019/workshop-image-augmentation-ai-for-eyes-2/>

Lauren Sell 2022. **Label Studio is a modern, multi-modal data annotation tool.**

[Online].Available : <https://github.com/HumanSignal/labelImg>

Glenn J., AyushExcel. 2023. **Ultralytics YOLOv8 Docs.** [Online].Available :

<https://docs.ultralytics.com/>

- Nicolas C., Francisco M., Gabriel S., Nicolas U., ..., & Sergey Z. 2020. **End-to-End Object Detection with Transformers**. [Online].Available : <https://arxiv.org/abs/2005.12872>
- Glenn Jocher. 2023. **Baidu's RT-DETR: A Vision Transformer-Based Real-Time Object Detector**. [Online].Available : <https://docs.ultralytics.com/models/rtdetr/>
- Ke D., Chengjie Z., Yihan R., & Yuzhi L., 2020. **MobileNetV2 Model for Image Classification**. [Online].Available : <https://ieeexplore.ieee.org/document/9422058>
- Christian S., Sergey I., & Vincent V. 2016. **Inception-v4, Inception-ResNet and the Impact of Residual Connections on Learning**. [Online].Available : <https://arxiv.org/pdf/1602.07261.pdf>
- Mingxing T., & Quoc V. 2020. **EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks**. [Online].Available : <https://arxiv.org/pdf/1905.11946.pdf>
- Gaudenz Boesch. 2024. **Vision Transformers (ViT) in Image Recognition – 2024 Guide**. [Online].Available : <https://viso.ai/deep-learning/vision-transformer-vit/>
- Zhuang L., Hanzi M., Chao-Yuan W., Christoph F., Trevor D., & Saining X. 2022. **A ConvNet for the 2020s**. [Online].Available : <https://arxiv.org/abs/2201.03545>
- ทินกร ม้าลายทอง, ปพนัน ธรรมเจริญพร. 2021. **เพิ่มประสิทธิภาพของ Machine Learning Model ด้วย Hyperparameter Optimization**. [Online].Available : <https://bdi.or.th/big-data-101/machine-learning-model-hyperparameter-optimization/>
- Chris K.,Aaron Z.,Frank V.,Jacob C. 2017. **Gesture Recognition for Robotic Control Using Deep Learning**. [Online].Available : <https://www.researchgate.net/publication/32008413910>
- Juan C Olamendy. 2023. **Understanding ReLU, LeakyReLU, and PReLU: A Comprehensive Guide**. [Online].Available : <https://medium.com/@juanc.olamendy/understanding-relu-leakyrelu-and-prelu-a-comprehensive-guide-20f2775d3d64>

Surapong Kanoktipsatharporn. 2019. **Vanishing Gradient Problem คืออะไร**. [Online].Available : <https://www.bualabs.com/archives/1789/what-is-vanishing-gradient-problem-fix-vanishing-gradient-problem-with-xavier-initialization-kaiming-initialization-neural-network-ep-2/>

Wipawas T. 2021. **จะพามาทำความรู้จักกับ Hyperparameter Tuning กัน**. [Online].Available : <https://wipawas-t.medium.com/%E0%B8%88%E0%B8%B0%E0%B8%9E%E0%B8%B2%E0%B8%A1%E0%B8%B2%E0%B8%97%E0%B8%B3%E0%B8%84%E0%B8%A7%E0%B8%B2%E0%B8%A1%E0%B8%A3%E0%B8%B9%E0%B9%89%E0%B8%88%E0%B8%B1%E0%B8%81%E0%B8%81%E0%B8%B1%E0%B8%9A-hyperparameter-%E0%B8%81%E0%B8%B1%E0%B8%99-9c9985ac76f1>

Mail'Suesarn Wilainuch. 2021. **Introduced to the Convolutional Block Attention Module (CBAM)**. [Online].Available : <https://medium.com/super-ai-engineer/introduced-to-the-convolutional-block-attention-module-cbam-19f61a2c98b>

Taha Samavati. 2021. **Squeeze-and-Excitation block explained**. [Online].Available : <https://medium.com/@tahasamavati/squeeze-and-excitation-explained-387b5981f249>

Pasith Thanapatpisarn. 2022. **[Fundamental Data Analytics & Data Scientist EP.21] วัดประสิทธิภาพของโมเดลด้วย Evaluation Metrics Part 2**. [Online].Available : <https://datascihaeng.medium.com/evaluation-matrix-part2-a7d83aea6537>

ภาคผนวก
บทความวิจัย

ก.1 ต้นฉบับบทความวิจัย

Compact-sized Logo Monitoring Model

Kietikul Jearanaitanakij¹, Surasak Tansudjaroenyang¹, Suppawit Prasurat¹, Napat Yordsawad¹, Supitcha Pulsiri¹, Tiwat Kullawan² and Chankit Yongpiyakul²

¹Department of Computer Engineering, School of Engineering, King Mongkut's Institute of Technology Ladkrabang, Bangkok 10520 THAILAND

²Dataxet Limited, Bangkok, 10330, Thailand

Abstract

Logo monitoring is a process that involves tracking and analyzing the presence and usage of a brand's logo across various platforms. In general, it comprises two major steps: logo detection and logo classification. Logo detection identifies all logo images and their locations in the picture while logo classification maps those logo images into the corresponding brands. The main challenge in logo monitoring is the system must detect and classify logo images with acceptable high accuracy within a reasonable short period. Our proposed method detects generic logos from an image using YoloV8 and classifies logos extracted from the previous step using a custom convolutional neural network. The experimental results on the Flicker and Thai logo datasets indicate a competitive performance of the proposed method among other pretrained CNNs. Moreover, the proposed model has a much more compact size and faster execution compared to other models. Our research contributes in two folds. First, we present a small logo monitoring model that can still achieve prediction accuracy comparable to other methods. Second, our proposed model attains a short execution time and is suitable for real-time deployment.

Keywords: Logo monitoring, Logo detection, Logo classification, Thai logo, Deep learning

1. Introduction

Logo monitoring involves tracking and analyzing a brand's reputation online. It gives you visibility of current and possible future threats to your brand's reputation that you can correct quickly. This process includes monitoring social media, forums, and other platforms where your brand is mentioned or discussed. It also helps in maintaining a positive brand image and customer trust and detecting counterfeit products or unauthorized use of a brand's logo.

Related research papers on logo monitoring are reviewed in chronological order as follows. Romberg et al. (2011) proposed a scalable approach for logo recognition in an image. They encoded and indexed the basic structures, such as edges and triangles, observed from the local features in the logo image. These encoded structures are used in analyzing the similarity between logos. They also introduced a benchmark called FlickrLogo consisting of 32-class logos and non-logo images. The experimental results on their dataset reflect a high precision, but a fair recall. Sahbi et al. (2013) presented a system capable of localizing and recognizing numerous instances of various reference logos within images. Reference logos and test images are perceived as clusters of local features, such as interest points, regions, etc., They are matched by minimizing a mixed energy function of the quality of matching between two features, a regularization criterion, and a neighborhood criterion. The experimental results were evaluated on MICC-Logos and FlickrLogos-27 benchmarks show a significant improvement over other state-of-the-art models. Hoi et al. (2015) introduced LOGO-Net, a comprehensive logo image repository designed for detecting logos and recognizing brands from actual product images in real-world scenarios. It consists of two datasets: "logos-18," which includes 18 logo classes from 10 brands, amounting to 16,043 logo images, and "logos-160," covering 160 logo classes across 100 brands, totaling 130,608 logo images. They also performed thorough empirical assessments on cutting-edge deep learning methods focusing on an application to large-scale logo detection and brand recognition tasks. Bianco et al. (2017) presented a deep learning-based approach to logo recognition. Their recognition process consists of a recall-focused region-based Convolutional Neural Network (CNN) proposed by Girshick et al. (2016) and another CNN specially trained for logo classification. Their top-performing solution surpassed other existing methods when evaluated on both the original FlickrLogos-32 dataset and its enlarged version. Su et al. (2017) devised an innovative algorithm to create synthetic context logos within training images, enhancing the model's resilience to unfamiliar background distractions and yielding superior logo detection performance. They also introduced the TopLogo-10 dataset for logo detection, comprising images of the top 10 most popular clothing brands captured within a visually rich context. Liao et al. (2017) suggested a mutual enhancement method to boost logo detection by leveraging information from simultaneously occurring logos. Their proposed mutual enhancement technique improves prominent proposals with weaker ones. Experiments conducted on the FlickrLogos-32 and FlickrLogos-27 datasets demonstrate that their approach achieves superior accuracy compared to existing methods. Liu et al. (2019) created an extensive vehicle logo benchmark to address real-world challenges like occlusions, low image resolutions, and fluctuating lighting conditions. They employed the YOLOv3 deep learning model as a baseline method and reported its performance on the VLD 1.0 dataset. YOLOv3 attained the highest mean average precision of 79.99% at IOU 0.25. Kuznetsov and Savchenko (2020) tackled an ambiguity in logo detection arising from numerous occlusions and logo view alterations caused by uniform wrinkles during hockey games. An experiment was carried out using the labeled NHL teams dataset, where they tested a variety of descriptor calculation algorithms, including SIFT, SURF, ORB, BRISK, FREAK, AKAZE, and DELF. Their findings revealed that the SIFT algorithm demonstrated the highest precision, whereas BRISK exhibited the highest recall. Wang et al. (2020) introduced the Discriminative Region Navigation and Augmentation Network (DRNA-Net). This network is designed to identify and enlarge the most informative regions for logo classification. The DRNA-Net is primarily composed of four sub-networks: the navigator, the teacher, the region-oriented data augmentation, and the scrutinizer. Initially, the navigator sub-network identifies regions in an image based on their information content. The teacher sub-network then evaluates these regions to select the most logo-relevant ones. These selected regions are augmented by the region-oriented data augmentation sub-network to identify more logo-related areas. Finally, the scrutinizer sub-network combines the features of these augmented regions with the entire image to create a unified feature for logo prediction.

Carvalho et al. (2021) put forward a self-learning and automatic detection technique designed to process TV broadcast content to identify channel logos and advertising segments, without the need for prior data about the channel or content. Histogram of Oriented Gradients method was utilized to extract logo features from potential regions. Their approach outperformed another renowned advertisement detection method (Gomes et al., 2017) when tested on a dataset comprising multiple Portuguese and international TV channels. Sahel et al. (2021) devised a method for logo detection that employs multiple convolutional neural networks. They leveraged pre-trained object detection models to boost the performance of logo detection tasks, particularly when only a subset of labeled training images captured in a real-world context is available. This approach yielded excellent results in logo detection. The study was conducted on the FlickrLogos-32 dataset, a widely used public dataset for detecting logos and recognizing brands from images of real-world products. The evaluation of the model focused on both the efficiency of its construction and its accuracy. Palecek and J. Chaloupka (2021) introduced a method for logo detection and identification from images using the scaled YOLOv4 technique. They tested the logo detector on widely used benchmarks, including FlickrLogos-32, Logos-32plus, TopLogo-10, and QMULOpenLogo. Their methodology demonstrated exceptional performance in terms of both average precision and speed. Wang et al. (2022) presented LogoDet-3K, a comprehensive logo detection dataset that stands out as the largest of its kind with complete annotations. It boasts 3,000 distinct logo categories, approximately 200,000 manually annotated logo objects, and 158,652 images. The extensive coverage and diversity of logo categories and annotated objects in LogoDet-3K set a more demanding standard for logo detection, surpassing existing datasets in complexity. Furthermore, the team introduced Logo-Yolo, a robust baseline method. This approach enhances the cutting-edge YOLOv3 framework by integrating Focal loss and CIoU loss, making it suitable for large-scale logo detection. Li et al. (2022) introduced a multi-task learning framework called SeeTek, which is grounded in deep learning and scene text recognition. By integrating deep metric learning with text recognition into a unified model, SeeTek has demonstrated significant enhancements in performance. Hubenthal and Kumar (2023) proposed a novel method to enhance logo recognition models. Their approach involved pre-training with image-text paired samples, which notably boosted the performance of a visual embedder used for logo retrieval, particularly for classes dominated by text. Additionally, they created a comprehensive public logo dataset named OpenLogoDet3K47, which combines LogoDet3K, QMUL-OpenLogo, and FlickrLogos-47. Hosseini et al. (2024) proposed a comprehensive approach to quantify the attention drawn by brand logos on packaging designs. The process begins with accurate logo detection using YOLOv8, applied to two notable datasets: FoodLogoDet-1500 and LogoDet-3K. Following this, they model user visual attention using a unique saliency prediction model, specifically designed for the context of packaging. Finally, they combine logo detection and saliency map creation to yield a comprehensive score for brand attention. Their approach surpasses existing models in terms of both logo detection and saliency map prediction.

In this study, we introduce a compact logo classification model leveraging Convolutional Neural Network (CNN) architecture. The proposed model is distinguished by its significantly reduced size and enhanced execution speed. Our research makes two notable contributions to the field. Firstly, we present a streamlined logo monitoring model that maintains prediction accuracy on par with existing methods. This compact model is particularly advantageous for applications requiring efficient resource utilization. Secondly, the proposed model exhibits a markedly shorter execution time, rendering it highly suitable for real-time deployment scenarios. The rest of this paper is organized as follows. Section 2 provides the fundamental knowledge necessary to understand the research. Section 3 presents the logo monitoring framework and the proposed logo classification model. The experimental results and visual

evaluations of various models are shown in Section 4. Lastly, the summary of our research and possible future works are given in Section 5.

2. Related Fundamentals

YOLO Object Detection

YOLO (You Only Look Once) is a pioneering object detection algorithm introduced by Redmon et al. (2016). Unlike traditional methods that employ a multi-stage process, YOLO reframes object detection as a single regression problem, predicting bounding boxes and class probabilities directly from full images in one evaluation. This approach significantly enhances processing speed, achieving real-time detection capabilities. YOLO's architecture utilizes a single convolutional neural network (CNN) to simultaneously predict multiple bounding boxes and their associated class probabilities, streamlining the detection process. Its efficiency and accuracy have made YOLO a cornerstone in computer vision, widely adopted for various applications ranging from autonomous driving to real-time video analysis.

Real-Time Detection Transformer

Real-Time Detection Transformer (RT-DETR), proposed by Zhao et al. (2024), represents a significant advancement in the field of object detection, leveraging the power of transformer architectures to achieve high accuracy and efficiency. Unlike traditional convolutional neural networks (CNNs), RT-DETR utilizes self-attention mechanisms to capture global context and dependencies within an image, which enhances its ability to detect objects with varying scales and occlusions. This model is particularly notable for its real-time performance, making it suitable for applications requiring immediate processing, such as autonomous driving, surveillance, and robotics. The integration of transformers in RT-DETR not only improves detection precision but also reduces computational complexity, thereby offering a robust solution for real-time object detection challenges.

Intersection over Union

Intersection over Union (IoU) is a widely used metric in computer vision, particularly in object detection and segmentation tasks. It measures the overlap between two bounding boxes or regions, providing a quantitative assessment of the accuracy of predicted object locations. IoU is defined as the ratio of the area of overlap between the predicted bounding box and the ground truth bounding box to the area of their union. Mathematically, it is expressed as:

$$IoU = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

This metric ranges from 0 to 1, where 1 indicates perfect overlap and 0 indicates no overlap. IoU is crucial in evaluating object detection models. It determines whether a predicted bounding box is a true positive or a false positive. Typically, a threshold (e.g., 0.5) is set, and predictions with IoU above this threshold are considered correct detections.

Mean Average Precision

Mean Average Precision (mAP) is a crucial evaluation metric in information retrieval and computer vision, particularly in object detection tasks (Beitzel et al., 2009). It provides a comprehensive measure of a model's precision across different recall levels, making it a standard for assessing the performance of detection algorithms. mAP for a set of queries or classes can be expressed as follows:

$$mAP = \frac{1}{n} \sum_i^n AP_i$$

Where n is the number of queries or classes and AP_i is the average precision of the i -th query or class. mAP evaluates how well a model detects objects across various categories. It considers both precision (the ratio of true positive detections to the total number of detections) and recall (the ratio of true positive detections to the total number of actual objects). High mAP values indicate that the model performs well in detecting objects accurately and consistently.

MobileNetV3

MobileNetV3 (Howard et al., 2019) is a state-of-the-art convolutional neural network architecture for mobile and edge devices. It builds upon the success of its predecessors, MobileNetV1 and MobileNetV2, by incorporating several advancements such as using a new activation function called hard-swish and an improved network search technique. These enhancements enable MobileNetV3 to achieve higher accuracy and efficiency while maintaining a low computational cost, making it ideal for real-time applications on resource-constrained devices.

InceptionV3

InceptionV3 (Szegedy et al., 2016) is a highly efficient convolutional neural network architecture that builds upon the success of its predecessors, InceptionV1 and InceptionV2. It introduces several improvements, such as factorized convolutions, and uses label smoothing, which enhances its performance and accuracy in image classification tasks. InceptionV3 is known for its ability to achieve high accuracy with relatively low computational cost, making it suitable for a wide range of applications, including real-time image recognition.

EfficientNetV2

EfficientNetV2 (Tan and Le, 2021) is an advanced convolutional neural network architecture designed to achieve high performance with improved efficiency. It builds upon the original EfficientNet by incorporating several enhancements, such as a new scaling method and the use of progressive learning, which allows the model to train faster and more accurately. EfficientNetV2 models are particularly effective in image classification tasks, achieving state-of-the-art results on various benchmarks while maintaining a balance between accuracy and computational cost.

ConvNeXt

ConvNeXt is a modern convolutional neural network architecture designed to bridge the gap between traditional convolutional networks and vision transformers. Introduced by Liu et al. (2022), ConvNeXt reexamines the design principles of convolutional networks, incorporating several key innovations from vision transformers to enhance performance. The architecture retains the simplicity and efficiency of convolutional networks while integrating advanced features such as depthwise convolutions, inverted bottlenecks, and layer normalization. These enhancements enable ConvNeXt to achieve state-of-the-art results on various benchmarks, including ImageNet classification, COCO object detection, and ADE20K semantic segmentation. The design and performance improvements of ConvNeXt highlight its potential as a robust and scalable architecture for diverse computer vision tasks.

3. Proposed Model

A framework for logo monitoring consists of two main components: Logo detection and Logo classification, as illustrated in Figure 1. The Logo detection module processes the original image to identify the location of logos. This module draws bounding boxes around objects likely to be logos and assigns a confidence score to each detection. Objects with confidence scores exceeding a predefined threshold are then passed to the next component, i.e., a logo classification.

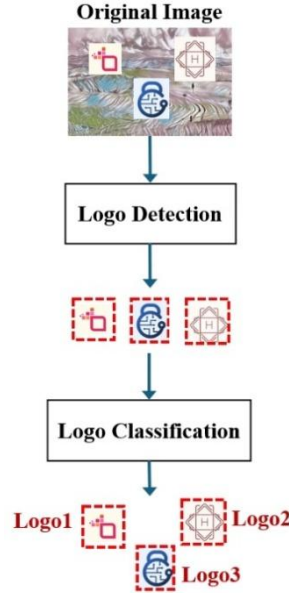


Figure 1: Logo Monitoring Framework

To achieve the optimal result, we compare the results of logo detection using two models: YoloV8 and RT-DETR. For the logo classification, we evaluated the logo classification of four pretrained CNN models (EfficientNetV2, ConvNeXt, InceptionV3, MobileNetV3), as an architecture shown in Figure 2, against the proposed CNN model. The proposed CNN model in Figure 3 comprises 4 main blocks followed by 2 dense and output layers. The most important block is the Convolutional Block Attention Module (CBAM). It enhances CNN by refining feature maps through channel and spatial attention mechanisms, leading to improved feature representation and accuracy. It is lightweight, efficient, and versatile, making it suitable for real-time applications and easy integration into existing architectures. CBAM helps models generalize better to new data, reducing the risk of overfitting and boosting overall performance.

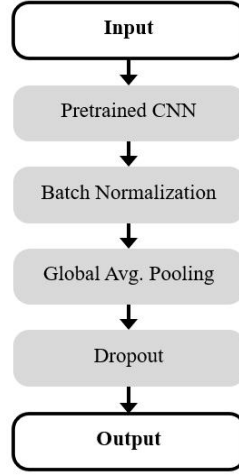


Figure 2: Transfer-learning architecture for logo classification using the pretrained CNN

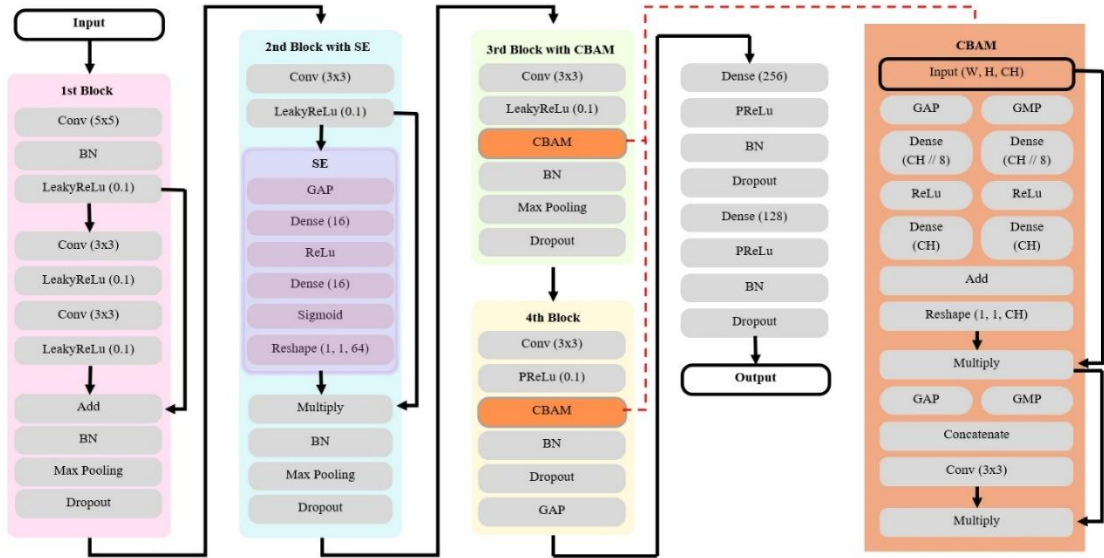


Figure 3: Proposed Logo Classification Model

4. Experiments and Comparisons

The experiments are conducted on two datasets, FlickrLogos-47 and 22-class Thai logos. FlickrLogos-47 is designed to reflect real-world scenarios without synthetic contexts, making it highly relevant for practical applications and testing in realistic environments. 22-class Thai logos are composed of the logo images of 22 Thai companies and government agencies manually collected from the internet. Each dataset is partitioned into 50% for the training set, 20% for the validation set, and 30% for the test set. Image augmentation methods used for both datasets are rotate, horizontal flip, shear, scale, contrast, and Gaussian noise.

FlickrLogos-47

The FlickrLogos-47 dataset (Romberg et al., 2011) is an extensive collection of real-world images curated for the evaluation of logo detection and recognition systems. It extends the earlier FlickrLogos-32 dataset by re-annotating the same image corpus to rectify missing annotations and incorporate additional logo classes. Comprising 47 distinct logo categories,

each depicted in various contexts and environments, this dataset significantly enhances its applicability for training and evaluating robust machine learning models. There are 47 logo classes: Adidas (Symbol), Adidas (Text), Aldi, Apple, Becks (Symbol), Becks (Text), BMW, Carlsberg (Symbol), Carlsberg (Text), Chimay (Symbol), Chimay (Text), Coca-Cola, Corona (Symbol), Corona (Text), DHL, Erdinger (Symbol), Erdinger (Text), Esso (Symbol), Esso (Text), Fedex, Ferrari, Ford, Foster's (Symbol), Foster's (Text), Google, Guinness (Symbol), Guinness (Text), Heineken, HP, Milka (Symbol), Milka (Text), Nvidia (Symbol), Nvidia (Text), Paulaner (Symbol), Paulaner (Text), Pepsi (Symbol), Pepsi (Text), Ritter Sport, Shell, Singha (Symbol), Singha (Text), Starbucks, Stella Artois (Symbol), Stella Artois (Text), Texaco, Tsingtao (Symbol) Tsingtao (Text) and UPS. The detailed annotations, which include bounding boxes for each logo instance, facilitate precise object detection tasks. The diversity and complexity of the images within FlickrLogos-47 make it an invaluable resource for advancing research in logo detection, brand monitoring, and related computer vision applications. Sample images from the FlickrLogos-47 dataset are shown in Figure 4.



Figure 4: Sample images of the FlickrLogos-47 dataset

22-Class Thai Logos

We have collected images from the internet featuring logos of prominent companies and government agencies in Thailand, categorized into 22 classes: DDC, GSB, MEA, GHB, PEA, SCB, EGAT, BBL, NBTC, TTB, CP, KTB, CP-ALL, UOB, 7-Eleven, BAY, SET, KBANK, SEC, Dataxet, MR.DIY, and TNN. Each class comprises 50-60 cropped logo images, totaling 1,170 cropped logo images. Figure 5 presents sample images of the logos for each class in the dataset.



Figure 5: Sample images of the 22-Class Thai logos dataset

Experimental results

We conduct all experiments on a machine with the following specifications: Intel Core i5-13500 RAM 64 GB with NVIDIA RTX 4000 ADA RAM 20 GB. The experiment is divided into two parts: selecting the model that can most accurately detect generic logos and their bounding boxes and selecting the CNN model that can best classify the logo's class.

Logo Detection

The experimental results in Table 1 indicate that RT-DETR slightly outperforms both YOLO models in terms of efficiency. However, the runtime of RT-DETR is approximately 30-40% longer than that of the YOLO models for the following reasons. First, a transformer-based architecture of RT-DETR which involves more complex computations compared to the convolutional neural networks (CNNs) used in YOLO models. Second, YOLOv8 and YOLOv10 have been optimized for real-time performance, focusing on reducing latency and computational cost. Finally, the multi-scale feature processing employed by RT-DETR to enhance its accuracy results in additional computational overhead. Therefore, we prefer using the YOLO model over RT-DETR.

Table 1: Runtime and precision comparison for logo detection

Dataset	Model	Runtime (msec.)	mAP50
Flickr Logos 47	YOLOv8	13.5	0.8539
	YOLOv10	14.1	0.8514
	RT-DETR	19.0	0.8759
22-Class Thai Logos	YOLOv8	14.0	0.8569
	YOLOv10	14.9	0.8303
	RT-DETR	19.5	0.8600

Logo Classification

We collected small images obtained from bounding boxes during the logo detection phase, using YoLOv8, and used them to train CNN models. We selected four pretrained CNN models: EfficientNetV2 (size B0), ConvNeXt (size tiny), MobileNetV3 (size large), and InceptionV3, and chose the top three of them which attain the highest accuracy to present the results. Additionally, we experimented with the proposed logo classification model shown in Figure 3. The experimental results of the test set are presented in Table 2. The experimental results indicate that the proposed model can predict logo images with accuracy comparable to other pretrained CNN models but with significantly less prediction time. The proposed model is task-specific and includes only the essential layers and parameters required for the logo classification task.

Table 2: Runtime and test accuracy comparison for logo classification

Dataset	Classification Model	Param (Million)	Avg. Prediction time (msec.)	Test Accuracy	Precision	Recall	F1-Score
Flickr Logos 47	EfficientNetV2	7.1	62	0.8606	0.8650	0.8606	0.8603
	ConvNeXt	28	105	0.8959	0.9012	0.8959	0.8966
	MobileNetV3	5.9	54	0.8526	0.8590	0.8526	0.8534
	Proposed	0.66	41	0.8606	0.8663	0.8606	0.8603
22-Class Thai Logos	EfficientNetV2	7.1	65	0.9647	0.9659	0.9647	0.9645
	InceptionV3	23.9	83	0.9365	0.9403	0.9365	0.9370
	MobileNetV3	5.9	56	0.9524	0.9533	0.9524	0.9522
	Proposed	0.66	41	0.9683	0.9687	0.9683	0.9682

Figures 6 and 7 present the confusion matrices for the proposed model on the Flickr Logos 47 and 22-Class Thai Logos datasets. The model accurately predicts most classes along the diagonal of both datasets. However, certain classes within the Flickr Logos 47 dataset, including Pepsi (symbol), Milka, and Pepsi (text), require improvement. Similarly, the BBL, SET, and UOB classes within the 22-Class Thai Logos dataset exhibit potential for enhancement. Figures 8 and 9 depict the Receiver Operating Characteristic (ROC) curves, which assess the model's performance across all possible classification thresholds, providing a comprehensive analysis of the model's behavior. The proposed model generates curves that are closer to the top-left corner, indicating its robust capability in logo prediction.

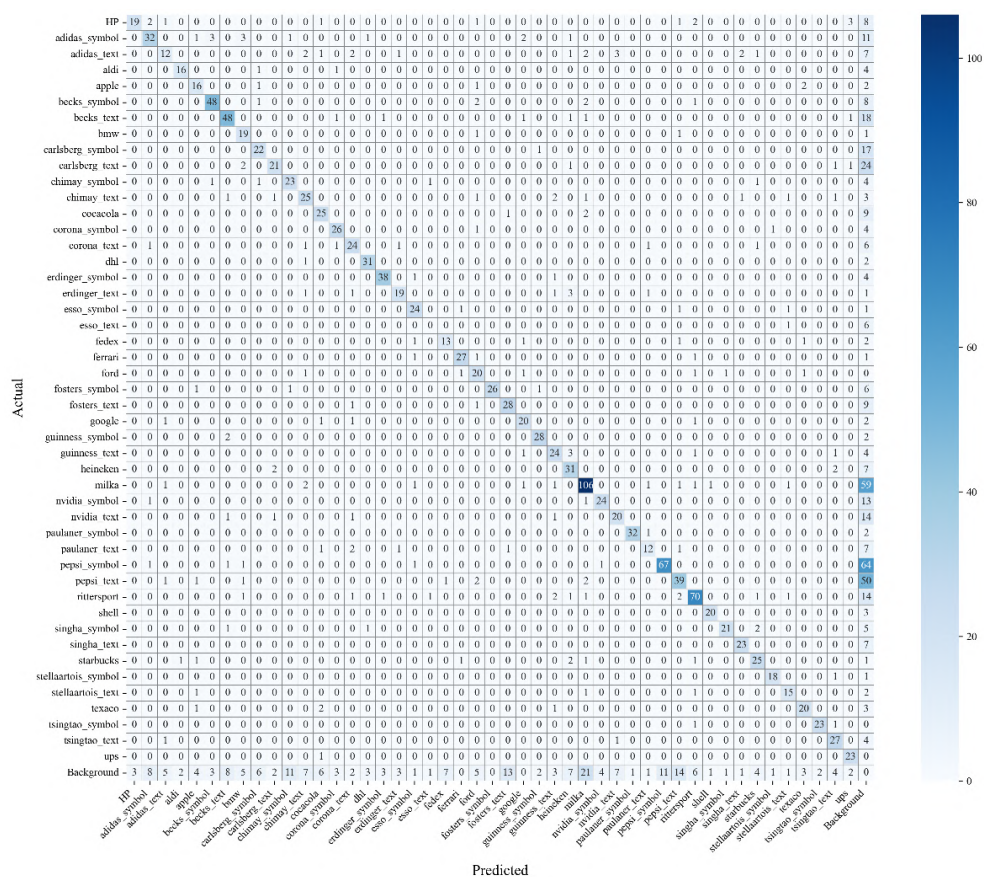


Figure 6 Confusion matrix of the proposed model on Flickr Logos 47.

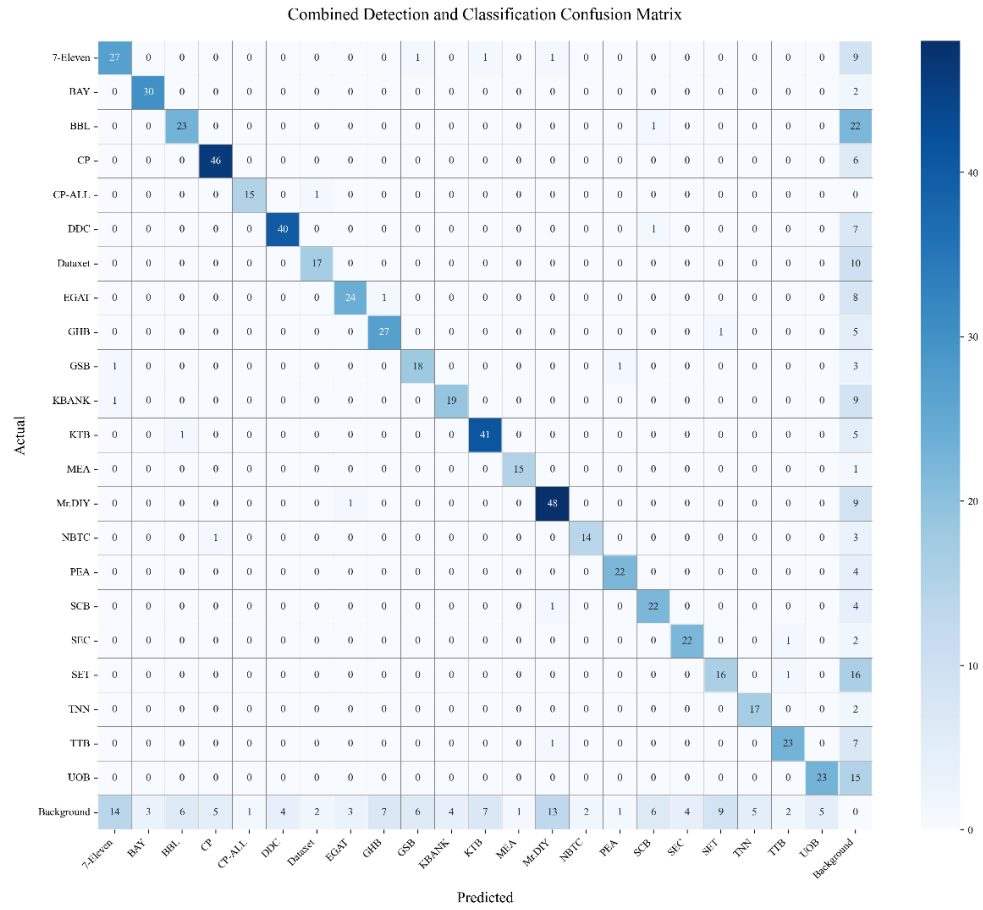


Figure 7 Confusion matrix of the proposed model on 22-Class Thai Logos.

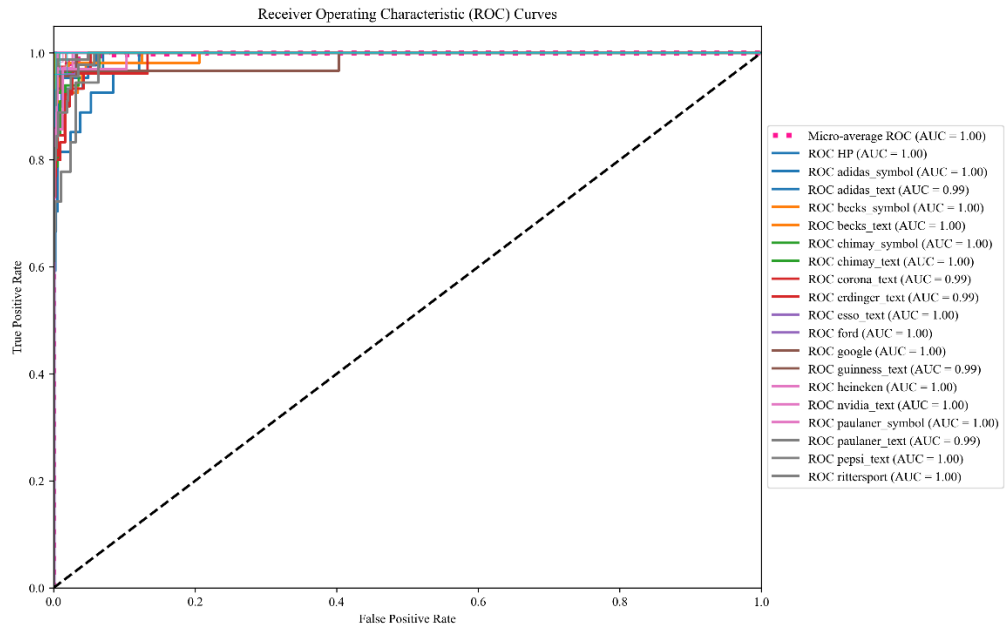


Figure 8 ROC graph of the proposed model on Flickr Logos 47.

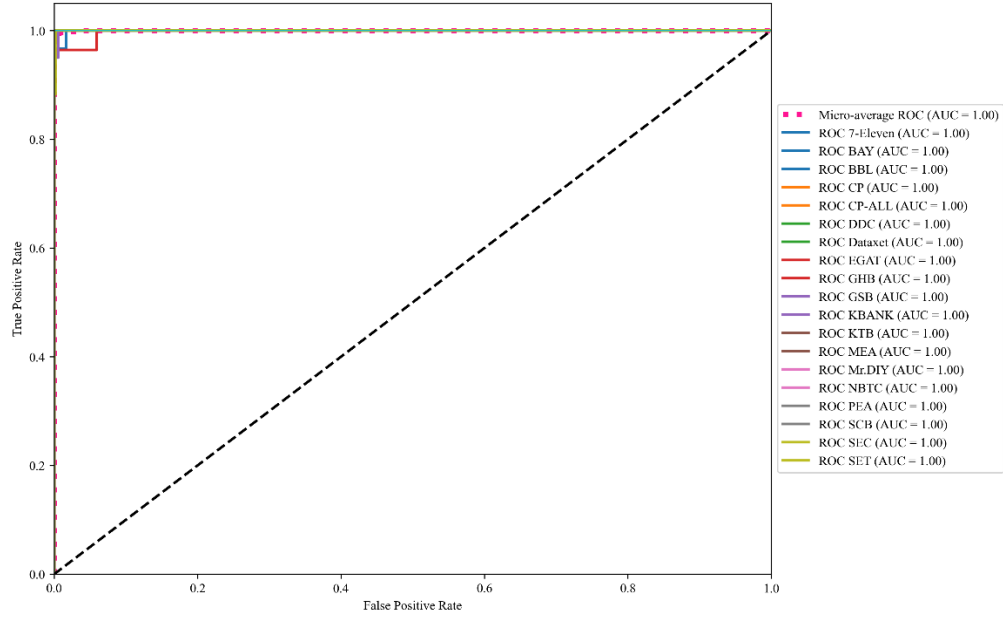


Figure 9 ROC graph of the proposed model on 22-Class Thai Logos.

To visualize the results of logo monitoring models, Figure 6 compares the results of four hybrid models (YOLOv8 + CNN). YOLOv8 is a finetuned version for detecting the generic logos in the dataset. Since the sample images in Figure 6 are relatively straightforward, most models can correctly predict the logos. However, the Milka logo is slightly tilted, causing most models to mispredict it. Only the proposed model correctly predicted the Milka logo.

Figure 11 illustrates examples of logo predictions on challenging images from the FlickrLogos-47 dataset. In addition to being captured at oblique angles, some images exhibit low brightness and contain dirt on the logos. Most CNN models manage to detect the logos but predict them incorrectly. Only the proposed model accurately predicts the logos in all images.

This observation underscores the robustness of the proposed model in handling various adversities such as low brightness, oblique angles, and occlusions. The superior performance of the proposed model can be attributed to its low confidence threshold and the subsequent use of a CNN for refined logo classification. These findings highlight the importance of model adaptability in enhancing the accuracy of logo detection and classification tasks.



Figure 10 Logo results on easy images in the FlickrLogos-47 dataset

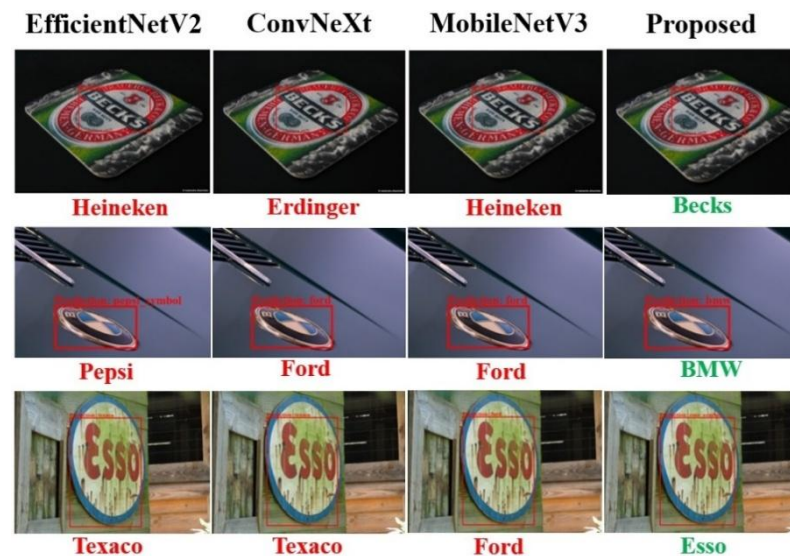


Figure 11 Logo results on hard images in the FlickrLogos-47 dataset

Figure 8 presents examples of logo predictions from images in the 22-Class Thai Logos dataset. This dataset contains nearly half the number of classes compared to the FlickrLogos-47 dataset. Most of the hybrid models, except for the proposed model, exhibit a relatively high number of incorrect predictions. This may be attributed to the intricate and similar details present in Thai

logo images across different classes. Consequently, using transfer learning without retraining weights in the pretrained CNN model might reduce the ability to accurately predict these details.

The experimental results of the proposed model are notably impressive, as it accurately predicted almost all logos, except for the text logos of MR.DIY and PEA. The SET logo on the podium is considered a challenging case, as no model was able to predict it correctly. This difficulty may be due to its proximity to flowers, which likely caused confusion among the models, resulting in confidence scores falling below the threshold.



Figure 12 Logo results in the 22-Class Thai Logos dataset

5. Conclusion

A comprehensive framework for logo monitoring encompasses two primary components: generic logo detection and logo classification. We proposed a compact logo classification model based on Convolutional Neural Network (CNN) architecture. Experimental evaluations on the FlickrLogos-47 and Thai logo datasets reveal that our proposed method achieves competitive testing accuracy compared to other pretrained CNN models. The findings indicate that integrating a compact and efficient logo classification model can significantly enhance the feasibility of real-time logo monitoring systems. Future research may extend this framework to include a wider variety of logo datasets and further optimize the model's architecture to improve its performance and applicability in diverse real-world scenarios. Our research offers two primary contributions. Firstly, we introduce a small logo monitoring model that maintains

prediction accuracy comparable to existing methods. Secondly, our proposed model demonstrates a shorter execution time, making it highly suitable for real-time deployment.

References

- [1] S. Romberg, L. G. Pueyo, R. Lienhart, and V. Z. Roelof, “Scalable logo recognition in real-world images,” in *Proceedings of the 1st ACM International Conference on Multimedia Retrieval*, 2011, pp. 1–8.
- [2] H. Sahbi, L. Ballan, G. Serra, and A. Bimbo, “Context-dependent logo matching and recognition,” *IEEE Transactions on Image Processing*, vol. 22, no. 3, pp. 1018–1031, 2013.
- [3] S. C. H. Hoi, X. Wu, H. Liu, Y. Wu, H. Wang, H. Xue, and Q. Wu, “Logo-Net: Large-scale deep logo detection and brand recognition with deep region-based convolutional networks,” *arXiv:1511.02462*, 2015.
- [4] S. Bianco, M. Buzzelli, D. Mazzini, and R. Schettini, “Deep learning for logo recognition,” *Neurocomputing*, vol. 245, pp. 23–30, 2017.
- [5] R. Girshick, J. Donahue, T. Darrell, J. Malik, Region-based convolutional networks for accurate object detection and segmentation, *Pattern Analysis and Machine Intelligence*, *IEEE Transactions on* 38 (1) (2016) 142–158.
- [6] H. Su, X. Zhu, and S. Gong, “Deep learning logo detection with data expansion by synthesising context,” in *IEEE Winter Conference on Applications of Computer Vision*, 2017, pp. 530–539.
- [7] Y. Liao, X. Lu, C. Zhang, Y. Wang, and Z. Tang, “Mutual enhancement for detection of multiple logos in sports videos,” in *IEEE International Conference on Computer Vision*, 2017, pp. 4856–4865.
- [8] J. Liu, F. Shen, M. Wei, Y. Zhang, H. Zeng, J. Zhu, and C. Cai, “A large-scale benchmark for vehicle logo recognition,” in *4th International Conference on Image, Vision and Computing*, 2019, pp. 479–483.
- [9] A. Kuznetsov and A. V. Savchenko, “A new sport teams logo dataset for detection tasks,” in *International Conference on Computer Vision and Graphics*, 2020, pp. 87–97.
- [10] J. Wang, W. Min, S. Hou, S. Ma, Y. Zheng, H. Wang, and S. Jiang, “Logo-2K+: A large-scale logo dataset for scalable logo classification,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020, pp. 6194–6201.
- [11] P. Carvalho, A. Pereira, and P. Viana, “Automatic TV logo identification for advertisement detection without prior data,” *Applied Sciences*, vol. 11, no. 16, pp. 74–94, 2021.
- [12] Gomes, A.; Queluz, M.P.; Pereira, F. Automatic detection of TV commercial blocks: A new approach based on digital on-screen graphics classification. In *Proceedings of the 2017 11th International Conference on Signal Processing and Communication Systems (ICSPCS)*, Surfers Paradise, Australia, 13–15 December 2017; pp. 1–6.
- [13] S. Sahel, M. Alsahafi, M. Alghamdi, and T. Alsubait, “Logo detection using deep learning with pretrained CNN models,” *Engineering, Technology & Applied Science Research*, vol. 11, no. 1, pp. 6724–6729, 2021.

- [14] K. Palecek and J. Chaloupka, "Logo detection and identification in system for audio-visual broadcast transcription," in 44th International Conference on Telecommunications and Signal Processing, 2021, pp. 357–360.
- [15] J. Wang, W. Min, S. Hou, S. Ma, Y. Zheng, and S. Jiang, "LogoDet-3K: A large-scale image dataset for logo detection," *ACM Transactions on Multimedia Computing, Communications, and Applications*, vol. 18, no. 1, pp. 1–19, 2022.
- [16] C. Li, I. Fehervari, X. Zhao, I. Macedo, and S. Appalaraju, "SeeTek: Very large-scale open-set logo recognition with text-aware metric learning," in *IEEE/CVF Winter Conference on Applications of Computer Vision*, 2022, pp. 587–596.
- [17] M. Hubenthal and S. Kumar, "Image-Text Pre-Training for Logo Recognition," 2023 *IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, Waikoloa, HI, USA, 2023, pp. 1145–1154, doi: 10.1109/WACV56688.2023.00120.
- [18] Hosseini, A., Hooshanfar, K., Omrani, P., Toosi, R., Toosi, R., Ebrahimian, Z., & Akhaee, M.A. (2024). Brand Visibility in Packaging: A Deep Learning Approach for Logo Detection, Saliency-Map Prediction, and Logo Placement Analysis. *ArXiv*, abs/2403.02336.
- [19] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 779–788). <https://doi.org/10.1109/CVPR.2016.91>
- [20] Beitzel, S.M., Jensen, E.C., Frieder, O. (2009). MAP. In: LIU, L., ÖZSU, M.T. (eds) *Encyclopedia of Database Systems*. Springer, Boston, MA. https://doi.org/10.1007/978-0-387-39940-9_492
- [21] Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., & Chen, J. (2024). DETRs beat YOLOs on real-time object detection. *arXiv*. <https://doi.org/10.48550/arXiv.2304.08069>
- [22] Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., ... & Le, Q. V. (2019). Searching for MobileNetV3. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 1314–1324.
- [23] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the Inception architecture for computer vision. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2818–2826.
- [24] Tan, M., & Le, Q. V. (2021). EfficientNetV2: Smaller models and faster training. *Proceedings of the 38th International Conference on Machine Learning*, 139, 10096–10106.
- [25] Wang, J., Min, W., Hou, S., Ma, S., Zheng, Y., & Jiang, S. (2022). LogoDet-3K: A large-scale image dataset for logo detection. *ACM Transactions on Multimedia Computing, Communications, and Applications*, 18(1), Article 21. <https://doi.org/10.1145/3466780>
- [26] Romberg, S., Garcia Pueyo, L., Lienhart, R., & van Zwol, R. (2011). Scalable logo recognition in real-world images. *Proceedings of the ACM International Conference on Multimedia Retrieval (ICMR)*. <https://doi.org/10.1145/1991996.1992031>
- [27] Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). A ConvNet for the 2020s. *arXiv preprint arXiv:2201.03545*.